

~~ORACLE ADVANCED CUSTOMER SERVICES~~

~~华泰证券~~

~~Oracle数据库故障和性能诊断指导手册~~

~~ORACLE工程师:~~ ~~万斌~~

~~报告生成日期:~~ ~~2022-09-20~~

~~现场支持起始日期:~~

~~现场支持结束日期:~~

~~现场支持总时间(小时):~~

~~Timesheet编号:~~



目录

1	文档概述	6
1.1	目的	6
1.2	适用范围	6
1.3	使用对象	6
2	常用基础管理	6
2.1	用户管理	6
2.1.1	检查 oracle 用户是否被锁或者密码是否过期:	6
2.1.2	显示指定数据库用户的信息和权限	7
2.1.3	查询某一个用户登录失败次数	8
2.1.4	解锁 oracle 用户	8
2.1.5	用户密码永不过期(按需)	8
2.2	表空间管理	9
2.2.1	检查表空间使用率	9
2.2.2	表空间扩容	10
2.2.3	undo 表空间使用率高	12
2.2.4	临时表空间使用率高	13
2.2.5	sysaux 表空间使用率高	14
2.2.6	system 表空间使用率高	14
2.3	归档日志管理	15
2.3.1	归档目录满	15
2.3.2	查看最近几天内归档日志量	15
3	性能诊断	15
3.1	等待事件	15
3.1.1	检查系统中存在的等待事件	15
3.1.2	检查系统正执行的 SQL 对应的等待事件	16
3.1.3	定位引起等待的源头 session	16
3.2	消耗资源的 SESSION	18
3.2.1	查看消耗 CPU 最为严重的 session	18
3.2.2	查看消耗 DB time 最高的 session	18
3.2.3	查看等待最为严重的 Session	19
3.3	锁或 LATCH	19
3.3.1	检查系统是否出现锁或者堵塞	19
3.3.2	目前锁等待的语句的 session 号以及语句	23
3.3.3	检查行锁 (enq: TX - row lock contention)	24
3.3.4	查看被锁的对象信息	24
3.3.5	快速定位锁源头 session 和解锁	25
3.4	语句执行慢	27
3.4.1	查看当前执行 SQL 的平均资源消耗	27
3.4.2	查看当前长时间运行的 SQL	27
3.4.3	查看耗费性能的 SQL	28

3.4.4	查询目前正在运行的耗时最长的 10 个 sql 语句	29
3.4.5	查看未使用绑定变量的 sql 语句	29
3.4.6	通过查询 ash 报告中某一会话在某一时间内的执行语句及执行时长	29
3.4.7	查看当前占用大量 TEMP 表空间的 SQL	30
3.4.8	查看当前占用大量 UNDO 表空间的 SQL	30
3.5	执行计划	31
3.5.1	查询某个 sql 语句最后一次执行的执行计划	31
3.5.2	查询某个 SQL 语句历史执行计划是否改变	31
3.5.3	执行计划变更分析	32
3.5.4	绑定执行计划	38
3.5.5	使用 sqlhc 查看执行计划情况	42
3.6	统计信息	43
3.6.1	查询表的统计信息的最新时间和行数	43
3.6.2	手动收集统计信息	43
3.6.3	给予普通用户授予统计信息更新的权限	44
3.6.4	自动收集统计信息任务	44
3.7	SGA/PGA 使用	46
3.7.1	查看 SGA 各组件大小	46
3.7.2	查看 shared pool 中占用内存较多的组件	46
3.7.3	查看 shared pool 中组件大小的历史信息	46
3.7.4	查看 SGA 各组件大小变动记录	46
3.7.5	查看 shared pool 中各子池各组件大小	47
3.7.6	查看当前 PGA 设定最大值	48
3.7.7	查看实例当前使用的 PGA 总大小	48
3.7.8	查看当前占用 PGA 较大的 session	48
4	常见等待事件分析	50
4.1	LOG FILE SYNC	50
4.1.1	Log file sync	50
4.2	LOG BUFFER SPACE	54
4.2.1	Log buffer space	54
4.3	BUFFER BUSY WAITS	56
4.3.1	Buffer busy waits	56
4.4	ROW CACHE LOCK	58
4.4.1	Row cache lock	58
4.5	DB FILE SCATTERED READ	61
4.5.1	Db file scattered read	61
4.6	争用等待	62
4.6.1	library cache lock/pin	62
4.6.2	read by other session	68
4.6.3	latch: cache buffers chains	70
4.7	ENQUEUE 等待	73
4.7.1	enq: TX - row lock contention	73
4.7.2	enq: TX - index contention	78
4.7.3	enq: TX - allocate ITL entry	80

4.7.4	enq: SQ - contention/enq: SV - contention(DFS lock handle).....	84
4.7.5	enq: US - contention.....	87
5	故障处理.....	89
5.1	常见故障处理及信息收集.....	89
5.1.1	ORA-04031	89
5.1.2	ORA-600	92
5.1.3	ORA-7445	92
5.1.4	性能问题.....	94
5.1.5	实例进程数满 ORA-00020 错误.....	95
5.1.6	长事务回滚问题.....	97
5.1.7	监听器 hang	99
5.1.8	数据文件有坏块.....	100
5.1.9	服务器 CPU 使用率高.....	102
5.1.10	服务器内存使用率高.....	103
5.1.11	IO 性能缓慢.....	104
5.2	数据库 HANG 故障诊断.....	108
5.2.1	HANG 分析方法.....	108
5.2.2	注意事项	108
5.2.3	收集 HangAnalyze 信息.....	108
5.2.4	收集 SystemState 信息（在 Oracle support 建议下收集）.....	108
5.2.5	收集 ErrorStack 信息（在 Oracle support 建议下收集）	109
5.2.6	收集相关日志信息.....	109
6	高可用集群.....	110
6.1	RAC 集群	110
6.1.1	数据库实例和集群启停.....	110
6.1.2	RAC 集群资源状态检测.....	114
6.1.3	RAC 集群资源状态检测.....	115
6.1.4	集群服务状态检查.....	116
6.1.5	集群 alert 日志检查.....	116
6.1.6	集群 crsd 日志检查.....	117
6.1.7	集群 cssd 日志检查.....	117
6.1.8	ASM 日志检查.....	117
6.1.9	数据库 alert 日志检查.....	117
6.1.10	Clusterware 集群日志总结.....	117
6.1.11	RACDiag 信息收集.....	119
6.2	DATA GUARD	120
6.2.1	查看 DG 是否同步.....	120
6.2.2	日志应用问题 MRP 进程异常.....	121
6.2.3	MRP 进程正常，但 DG 有 GAP	125
7	常用工具或方法.....	127
7.1	主机资源类工具.....	127
7.1.1	OSWatcher 工具.....	127

7.2	SQL 优化类工具	130
7.2.1	SQLHC 工具.....	130
7.2.2	SQL Tuning advisor 工具.....	131
7.2.3	SQL Monitor 工具.....	135
7.2.4	SQLT 工具.....	138
7.3	日志收集类工具.....	140
7.3.1	TFA 工具.....	140
7.3.2	ADRCI 工具.....	140
7.4	数据库巡检类工具.....	144
7.4.1	OraCheck 工具.....	144
7.4.2	oratop 工具.....	144
8	常用信息收集方法	147
8.1	收集 AWR 报告.....	147
8.2	收集 ASH 报告.....	148
8.3	收集 ASH DUMP 数据.....	148
8.4	收集 ADDM 信息.....	149
8.5	标准 TRACE 收集/ALERT 信息收集	149
8.6	TFA 日志收集	149
8.7	OS 日志信息搜集.....	150
8.8	OSWATCHER 日志收集	150
8.9	HANGANALYZE 信息搜集.....	150
8.10	SYSTEMSTATE 信息搜集.....	151
8.11	ERRORSTACK 信息收集	153
8.12	10046 事件	154
8.13	10053 事件	155
9	附加内容 – 相关故障解决分析参考	157
9.1	ORA-4031.....	157
9.2	SQL 锁问题处理	159
9.2.1	查看被锁对象信息.....	159
9.2.2	查看会话阻塞信息.....	160
9.2.3	解锁	160

1 文档概述

1.1 目的

通过 oracle 数据库故障和性能诊断作业指导书，实现 oracle 数据库的故障信息收集和性能信息收集的。

- 提高数据库日常运维效率；
- 减少数据库运维操作的出错率；
- 加快数据库应急故障处理速度；
- 降低数据库维护操作技术门槛。

1.2 适用范围

本手册适用于在 Oracle 数据库环境下故障诊断及排查。

1.3 使用对象

本手册使用对象为 Oracle 数据库的的运维 DBA 及相关人员。

2 常用基础管理

2.1 用户管理

2.1.1 检查 oracle 用户是否被锁或者密码是否过期：

使用DBA用户登录查询：

```
SQL>set linesize 1000
```

```
SQL>select username,account_status,lock_date,EXPIRY_DATE,PROFILE from dba_users  
where username = upper('&1');
```

说明：&1代表用户名

```
XTJK          OPEN          DEFAULT
SQL> set verify off
SQL> set linesize 1000
SQL> select username,account_status,lock_date,EXPIRY_DATE,PROFILE from dba_users where username = upper('&1');
Enter value for 1: xtjk

USERNAME          ACCOUNT_STATUS          LOCK_DATE          EXPIRY_DATE          PROFILE
-----
XTJK              OPEN                    NULL               NULL                 DEFAULT
SQL>
```

2.1.2 显示指定数据库用户的信息和权限

一个 oracle 数据库用户拥有系统权限，角色权限和对象权限：

```
SET LINES 200 PAGES 1000
alter session set nls_date_format='YYYYMMDD HH24:MI:SS' ;

COL USERnm NEW_VALUE USERNM NOPRINT ;
col profile new_value profile format a20
col usernm format a16
SELECT UPPER('&username') as usernm from dual;

PROMPT
PROMPT Base Info:

SELECT USERNAME,
       ACCOUNT_STATUS,
       DEFAULT_TABLESPACE,
       TEMPORARY_TABLESPACE,
       PROFILE ,
       CREATED
FROM DBA_USERS
where username = '&usernm';

prompt =====
prompt
prompt granted role:

col granted_role format a20
col admin_option format a20
col default_role format a16
COL GRANTOR      FORMAT A16
COL TABLE_NAME  FORMAT A24
col PRIVILEGE    format a30
select granted_role,
       admin_option,
       default_role
from dba_role_privs
where grantee = '&usernm';

prompt =====
prompt
prompt granted object PRIVS:

select grantor,
```

```

        table_name,
        privilege
from dba_tab_privs
where grantee = '&usernm'
    and owner <> '&usernm'
ORDER BY GRANTOR, TABLE_NAME;

prompt =====
prompt
prompt granted SYS PRIVS:
SELECT PRIVILEGE,
        ADMIN_OPTION
FROM DBA_SYS_PRIVS
WHERE GRANTEE = '&usernm' ;

prompt =====
prompt
prompt PROFILE WITH THE USER
SELECT * FROM DBA_PROFILES WHERE PROFILE = '&PROFILE' ;
undefine usernm;
undefine profile;

```

功能:

- 用户基本信息、用户被授予的角色、用户拥有的系统级权限、用户拥有的非自身对象的权限、用户 profile 信息;

2.1.3 查询某一个用户登录失败次数

用sys或system用户登录查询:
select lcount from user\$ where name=upper('&l');
说明: &l代表用户名

2.1.4 解锁 oracle 用户

使用system或sys用户登录解锁:
alter user &l account unlock;
说明: &l代表用户名

2.1.5 用户密码永不过期（按需）

Oracle 用户密码永不过期需要通过设置用户所属的 profile。

使用DBA用户登录查询用户所属profile:
set linesize 1000
SELECT USERNAME, ACCOUNT_STATUS, PROFILE, CREATED FROM DBA_USERS where
username=upper('&usernm');
查询profile的密码策略:


```
set linesize 1000
col profile for a30
col limit for a30
select profile,resource_name,resource_type,profile from dba_profiles order by 1;
其中PASSWORD_LIFE_TIME表示密码到期时长，如果要设置成用户过期：
oracle用户登录
sqlplus / as sysdba
alter profile profile名 limit password_life_time unlimited;
```

2.2 表空间管理

2.2.1 检查表空间使用率

```
使用DBA用户登录查询：
SET PAGESIZE 999
SET LINESIZE 110
TTITLE LEFT "Tablespace Information" SKIP 1 -
      LEFT "===== "
SET HEAD ON
SET FEEDBACK ON
BREAK ON REPORT
COMPUTE SUM LABEL 'Total Spaces' OF total_m ON REPORT
COMPUTE SUM LABEL 'Total Spaces' OF free_m ON REPORT
COMPUTE SUM LABEL 'Total Spaces' OF used_m ON REPORT
col tablespace format a25
col ext_mgt format a8
col seg_mgt format a8
col status format a7
set feedback off
select b.tablespace_name tablespace,
       b.total_m,
       b.free_m,
       b.used_m,
       b.used_pct
from
dba_tablespaces a,
(select
  d.tablespace_name tablespace_name,
  round((d.sumbytes/1024/1024),2) total_m,
  round(decode(f.sumbytes,null,0,f.sumbytes)/1024/1024,2) free_m,
  round(((d.sumbytes-decode(f.sumbytes,null,0,f.sumbytes))/1024/1024),2) used_m,
  round((d.sumbytes-decode(f.sumbytes,null,0,f.sumbytes))*100/d.sumbytes,2)
  used_pct
```

```

from
  (select
    tablespace_name,    sum(bytes) sumbytes
    from dba_free_space  group by tablespace_name) f,
  (select tablespace_name,    sum(bytes) sumbytes
    from dba_data_files    group by tablespace_name) d
where f.tablespace_name(+) = d.tablespace_name
order by d.tablespace_name) b
where a.tablespace_name=b.tablespace_name
union all
select b.tablespace_name tablespace,
       b.total_m,
       b.free_m,
       b.used_m,
       b.used_pct
from
dba_tablespaces a,
(select
  d.tablespace_name tablespace_name,
  round((d.sumbytes/1024/1024), 2) total_m,
  round((d.sumbytes/1024/1024), 2) -
round(decode(f.sumbytes, null, 0, f.sumbytes)/1024/1024, 2) free_m,
  round(decode(f.sumbytes, null, 0, f.sumbytes)/1024/1024, 2) used_m,
  round(decode(f.sumbytes, null, 0, f.sumbytes)*100/d.sumbytes, 2) used_pct
from
  (select
    tablespace_name,    sum(bytes_used) sumbytes
    -- sum(bytes_cached) sumbytes
    from v$temp_extent_pool  group by tablespace_name) f,
  (select tablespace_name,    sum(bytes) sumbytes
    from dba_temp_files    group by tablespace_name) d
where f.tablespace_name(+) = d.tablespace_name
order by d.tablespace_name) b
where a.tablespace_name=b.tablespace_name order by 5;
TTITLE OFF

```

功能:

- 检查空间使用率（包括永久表空间、临时表空间和 Undo 表空间）；
- 当表空间使用比例达到或超过 80%时，该表空间便应该着手进行扩容；

2.2.2 表空间扩容

Oracle 表空间扩容可通过 `resize` 已存在的文件大小或者新建一个文件，而数据表空间和临时表空间扩容命令行有略微不同：

先查询数据库的所有文件等信息：

```
set lines 200 pages 1000
set head on
set feedback on
col file_type format a16
col file_name format a64
col size_mb format 9999999
select 'Data Files' as File_type ,
       file_name,tablespace_name,
       trunc(bytes/1024/1024) as Size_Mb,
       autoextensible
from   dba_data_files
union all
select 'Temp Files' as File_type ,
       file_name,tablespace_name,
       trunc(bytes/1024/1024) as Size_Mb,
       autoextensible
from   dba_temp_files;
```

FILE_TYPE	FILE_NAME	TABLESPACE_NAME	SIZE_MB	AUT
Data Files	+ARCH/CIBRAC/DATAFILE/system.261.1036097273	SYSTEM	700	YES
Data Files	+ARCH/CIBRAC/DATAFILE/sysaux.262.1036097285	SYSAUX	1500	YES
Data Files	+ARCH/CIBRAC/DATAFILE/undotbs1.263.1036097291	UNDOTBS1	270	YES
Data Files	+ARCH/CIBRAC/DATAFILE/undotbs2.265.1036097321	UNDOTBS2	200	YES
Data Files	+ARCH/CIBRAC/DATAFILE/users.266.1036097325	USERS	7	YES
Data Files	+ARCH/CIBRAC/DATAFILE/data_olap.418.1099433173	DATA_OLAP	100	YES
Data Files	+ARCH/CIBRAC/DATAFILE/ccdata.dbf	CCDATA	1741	YES
Temp Files	+ARCH/CIBRAC/TEMPFILE/temp.264.1036097295	TEMP	70	YES

8 rows selected.

Data Files 代表某一表空间的数据文件，Temp Files 代表某一临时表空间的临时文件。

2.2.3.1 扩容数据表空间文件

➤ resize 文件：

Alter database datafile '数据文件全路径名,即上面查出来的路径' resize 新的文件大小;

➤ 新建一个文件：

Alter tablespace 表空间名 add datafile '数据文件全路径名,路径即用上面查询出的' size 文件大小;

Alter tablespace 表空间名 add datafile '+DATA' size 30g;

2.2.3.2 扩容临时表空间文件

➤ resize 文件：

Alter database tempfile '临时表空间数据文件全路径名,即上面查出来的路径' resize 新的文件大小;

➤ 新建一个文件：

Alter tablespace 表空间名 tempfile '数据文件全路径名,路径即用上面查询出的' size 文件大小;

Alter tablespace 表空间名 add tempfile '+DATA' size 30g;

2.2.3 undo 表空间使用率高

功能：Undo 表空间为回滚表空间，主要用来事物回滚、数据库恢复、读一致性、闪回查询；

使用DBA用户登录查询：

```
CLEAR COLUMNS BREAKS COMPUTES
COLUMN tablespace    FORMAT a20          HEADING 'Tablespace Name'
COLUMN sizeb         FORMAT 999,999,999   HEADING 'Used Size'
COLUMN sizea         FORMAT 999,999,999   HEADING 'Tablespace Size'
COLUMN status        FORMAT a12          HEADING 'Status'
COLUMN pct           FORMAT a8           HEADING 'Used Pct'
CLEAR COMPUTES BREAKS
BREAK ON tablespace on report
COMPUTE sum LABEL "Total: " OF sizeb ON report
select a.tablespace_name tablespace
       ,b.status                status
       ,b.bytes                 sizeb
       ,a.bytes                 sizea
       ,round(100*(b.bytes/a.bytes),2)||'%' pct
from
  (select tablespace_name
       ,sum(bytes)/1024/1024 bytes
  from dba_data_files
  group by tablespace_name) a,
  (select tablespace_name
       ,status
       ,sum(bytes)/1024/1024 bytes
  from dba_undo_extents
  group by tablespace_name,status) b
where a.tablespace_name=b.tablespace_name
order by 1,2;
```

```
Tablespace Name    Status          Used Size  Tablespace Size  Used Pct
-----
UNDOTBS1           ACTIVE          1          270   .37%
                  EXPIRED        19          270   7.06%
                  UNEXPIRED      31          270  11.62%
UNDOTBS2           EXPIRED        16          200   8.19%
                  UNEXPIRED      17          200   8.5%
*****
Total:              85
5 rows selected.
SQL>
```

其中 active 即为活动的，unexpired 为未过期的，expired 为过期的，其中 expired 是可以重复利用的，unexpired 按照中心的配置如果没有特殊的如闪回等需求外，一般也是可以重复利用的，所以如果 active 占比不高的话 undo 表空间的告警可以忽略。如需应急可使用上述表空间扩容的方式进行扩容解决当前问题，而后再去分析原因。

2.2.4 临时表空间使用率高

功能：用来管理数据库排序操作以及用于存储临时表、中间排序结果等临时对象。

使用DBA用户登录查询哪些sql占用TEMP较高：

```
set linesize 1000
col username for a20
col sess for a20
select s.sid || ', ' || s.serial# as sess, s.username, o.blocks*8192/1024/1024
mb_used,
o.tablespace, h.sql_id, substr(h.sql_text, 1, 100) from v$tempseg_usage o, v$session
s, v$sqlarea h where o.session_addr=s.saddr and s.sql_id=h.sql_id order by MB_USED;
```

查看 temp 表空间的使用量

使用DBA用户登录查询TEMP表空间的使用率：

```
select tablespace_name, round(free_space/1024/1024/1024, 2) "free
gb", round(tablespace_size/1024/1024/1024, 2)
total, round(nvl(free_space, 0)*100/tablespace_size, 3) free_percent from
dba_temp_free_space;
```

或者使用如下方式查询

```
select
f.tablespace_name, d.file_name , round(((f.bytes_free+f.bytes_used)/1024/1024, 2)
total_mb,
round (((f.bytes_free+f.bytes_used)-nvl(p.bytes_used, 0))/1024/1024, 2) free_mb,
round (nvl(p.bytes_used, 0)/1024/1024, 2) used_mb
from sys.v$temp_space_header f, dba_temp_files d, sys.v$temp_extent_pool p
where f.tablespace_name(+) = d.tablespace_name and f.file_id(+) = d.file_id and
p.file_id(+) = d.file_id;
```

或者使用如下方式查询

```
SELECT A.tablespace_name tablespace, D.gb_total,
SUM (A.used_blocks * D.block_size) / 1024 / 1024 / 1024 gb_used,
D.gb_total - SUM (A.used_blocks * D.block_size) / 1024 / 1024 / 1024 gb_free
FROM v$sort_segment A,
(
SELECT B.name, C.block_size, SUM (C.bytes) / 1024 / 1024 / 1024 gb_total
FROM v$tablespace B, v$tempfile C
WHERE B.ts# = C.ts#
GROUP BY B.name, C.block_size
) D
WHERE A.tablespace_name = D.name
GROUP by A.tablespace_name, D.gb_total;
```

2.2.5 sysaux 表空间使用率高

SYSAUX 表空间作为 system 的辅助表空间，如果真的达到 100%并不会影响数据库的使用，但是会影响各种性能数据的收集，建议还是尽快扩容。

查看 sysaux 里面各个组成的使用量

```
使用DBA用户登录查询：
col schema for a20
col item for a30
select occupant_name "Item",space_usage_kbytes/1048576 "Space
Usaged(GB)",schema_name "Schema",move_procedure "Move Porcedure" from
v$sysaux_occupants order by 2 desc;
```

其中比较受关注的：

- SM/AWR 就是 awr 报告，默认保存 7 天

可通过如下语句查看保留时间：

```
select * from dba_hist_wr_control;
```

- SM/OPTSTAT 就是统计信息，默认保存 31 天

可通过如下语句查看保留时间

```
select dbms_stats.get_stats_history_retention from dual;
```

2.2.6 system 表空间使用率高

system 表空间作为数据库的系统表空间，如果达到 100%会引起数据库挂起，要求及时发现及时处理。如需应急可使用上述表空间扩容的方式进行扩容解决当前问题，而后再去分析原因。

查看前 10 个占用表空间的对象：

```
select * from (select segment_name,sum(bytes) from dba_segments where
tablespace_name='SYSTEM' group by segment_name order by 2 desc) where rownum<10;
```

如果审计开启并放在 DB 中，一般来说都是 AUD\$审计表过大

查看审计的模式

```
show parameter audit
```

如果是 OS 就是放在操作系统目录 show parameter audit_file_dest 上；如果是 DB 就是放在数据库表空间内。

2.3 归档日志管理

2.3.1 归档目录满

(1) root 登录主机

```
#su - oracle
$export ORACLE_SID=实例名
$rman target /
RMAN> crosscheck archivelog all;
```

➤ 下面表示删除 1 天前的归档日志

```
RMAN> delete noprompt archivelog until time 'sysdate-1';
```

如果在 DG 环境下上述方式删不掉可使用 force 参数进行删除:

```
RMAN> delete force noprompt archivelog until time 'sysdate-1';
```

➤ 下面表示删除 30 分钟前的归档日志，适用于文件系统大小不能满足一天的归档量的时候（具体时长可根据需要修改）

```
RMAN> delete noprompt archivelog until time sysdate-30/1440';
```

(2) 另如果是使用的操作系统 rm 命令删除的归档日志，建议再运行如下命令清除 catalog 目录下的记录:

root 登录主机

```
#su - oracle
$export ORACLE_SID=实例名
$rman target /
RMAN> crosscheck archivelog all;
RMAN> list expired archivelog all;
RMAN> delete noprompt expired archivelog all;
```

2.3.2 查看最近几天内归档日志量

使用DBA用户登录查询:

```
select trunc(completion_time) as
"Date",count(*) ,((sum(blocks*block_size))/1024/1024) MB from v$archived_log group
by trunc(completion_time) order by 1 desc;
```

3 性能诊断

3.1 等待事件

3.1.1 检查系统中存在的等待事件

使用 DBA 用户登录查询:

```
--查看当前是否有较为严重的等待事件
```

```
set linesize 200 pagesize 1000
col event for a80
select inst_id, event, count(*)
  from gv$session
 where event not like '%SQL%'
    and event not like '%Stream%'
    and event not like '%rdbms%'
    and event not like '%timer%'
    and event not like '%idle%'
 group by inst_id, event
 order by 1, 3;
```

功能:

- 初步判断或定位系统等待事件，如磁盘 IO 较高，查看是否有 rman 备份任务等等；

INST_ID	EVENT	COUNT(*)
1	Data Guard: Timer	1
1	VKRM Idle	1
1	Data Guard: Gap Manager	1
1	VKTM Logical Idle wait	1
1	wait for unread message on broadcast channel	1
1	heartbeat redo informer	1
1	PGA memory operation	2
1	watchdog main loop	2

8 rows selected.

3.1.2 检查系统正执行的 SQL 对应的等待事件

使用 DBA 用户登录查询：

```
--查看当前有哪些SQL存在等待事件
set linesize 200
col sql_text for a80
col sql_id for a30
col event for a40
select a.inst_id,b.sql_id,a.event,count(*) count,b.sql_text
  from gv$session a,gv$sql b
 where a.wait_class!='Idle' and a.sql_address=b.address and a.inst_id=b.inst_id
 group by a.inst_id,b.sql_text,b.sql_id,a.event
 order by count;
```

3.1.3 定位引起等待的源头 session

使用 DBA 用户登录查询：


```

SET LINES 200 PAGES 1000
col program for a30
col machine for a30
col username for a20
col event for a30
select
sid,serial#,program,machine,username,event,final_blocking_session,final_blocking_in
stance from v$session where blocking_session is not null;

```

SID	SERIAL#	PROGRAM	MACHINE	USERNAME	EVENT	FINAL_BLOCKING_SESSION	FINAL_BLOCKING_INSTANCE
3218	22411	sqlplus@rac19c2 (TNS V1-V3)	rac19c2	SYS	enq: TX - row lock contention	3226	2
3224	14114	sqlplus@rac19c2 (TNS V1-V3)	rac19c2	SYS	enq: TX - row lock contention	3226	2
3229	59600	sqlplus@rac19c2 (TNS V1-V3)	rac19c2	SYS	enq: TX - row lock contention	3226	2
3234	23138	sqlplus@rac19c2 (TNS V1-V3)	rac19c2	SYS	enq: TX - row lock contention	3226	2

SQL>

功能:

- 快速定位引起等待的源头 session，（final_blocking_session 为 11G 后引入的字段，记录引起等待的最终 session）

查看源头 session（holder）的详细信息

查看上面定位出的源头session的详细信息

```
SQL> select sid,serial#,program,machine,username,sql_id,event from v$session where
sid=&sid;
```

例如结果如下:

SID	SERIAL#	PROGRAM	MACHINE	USERNAME	SQL_ID	EVENT
3226	9898	sqlplus@rac19c2 (TNS V1-V3)	rac19c2	SYS		SQL*Net message from client

功能:

- 查看引起等待的源头 session 的详细信息，以便最终定位问题及是否可以通过 kill session 方式应急处理。

3.2 消耗资源的 session

3.2.1 查看消耗 CPU 最为严重的 session

使用DBA用户登录查询：

```
-- sessions with highest CPU consumption

SELECT s.sid, s.serial#, p.spid as "OS PID", s.username, s.module, st.value/100 as
"CPU sec"

FROM v$sesstat st, v$statname sn, v$session s, v$process p

WHERE sn.name = 'CPU used by this session' -- CPU

AND st.statistic# = sn.statistic#

AND st.sid = s.sid

AND s.paddr = p.addr

AND s.last_call_et < 1800 -- active within last 1/2 hour

AND s.logon_time > (SYSDATE - 240/1440) -- sessions logged on within 4 hours

ORDER BY st.value;
```

3.2.2 查看消耗 DB time 最高的 session

```
-- sessions with highest DB Time usage

SELECT s.sid, s.serial#, p.spid as "OS PID", s.username, s.module, st.value/100 as
"DB Time (sec)"

, stcpu.value/100 as "CPU Time (sec)", round(stcpu.value / st.value * 100, 2) as "%
CPU"

FROM v$sesstat st, v$statname sn, v$session s, v$sesstat stcpu, v$statname sncpu,
v$process p

WHERE sn.name = 'DB time' -- CPU

AND st.statistic# = sn.statistic#

AND st.sid = s.sid

AND sncpu.name = 'CPU used by this session' -- CPU

AND stcpu.statistic# = sncpu.statistic#

AND stcpu.sid = st.sid

AND s.paddr = p.addr

AND s.last_call_et < 1800 -- active within last 1/2 hour

AND s.logon_time > (SYSDATE - 240/1440) -- sessions logged on within 4 hours
```

```
AND st.value > 0
ORDER BY st.value;
```

3.2.3 查看等待最为严重的 Session

通过指定等待事件，查看等待较为严重的 session

```
-- sessions with the highest time for a certain wait
SELECT s.sid, s.serial#, p.spid as "OS PID", s.username, s.module, se.time_waited
FROM v$session_event se, v$session s, v$process p
WHERE se.event = '&event_name'
AND s.last_call_et < 1800 -- active within last 1/2 hour
AND s.logon_time > (SYSDATE - 240/1440) -- sessions logged on within 4 hours
AND se.sid = s.sid
AND s.paddr = p.addr
ORDER BY se.time_waited;
SQL> /
```

例如：Enter value for event_name: db file sequential read

3.3 锁或 latch

3.3.1 检查系统是否出现锁或者堵塞

使用 DBA 用户登录数据库查询：

```
alter session set nls_date_format='yyyymmdd hh24:mi:ss';
select sysdate from dual;
set lines 200 pages 1000
col blocking_instance format 99
col inst_id format 99
col sid format 99999999
col blocking_session format 999999
col event for a40
col file_name format a40
col sql_text format a60
col PROGRAM format a18
col module format a18
col machine format a18
```

```

COLUMN owner FORMAT A20
COLUMN username FORMAT A20
COLUMN object_owner FORMAT A20
COLUMN object_name FORMAT A30
COLUMN locked_mode FORMAT A15
prompt "blocking sessions"
select blocking_instance, blocking_session, count(*)
  from gv$session
 where blocking_session is not null
 group by blocking_instance, blocking_session
 order by 1, 3;
prompt
prompt
prompt "blocking detail"
select inst_id,
       sid,
       serial#,
       nvl(sql_id,prev_sql_id) sql_id,
       prev_sql_id,
       event,
       state,
       seconds_in_wait,
       p1,
       p2,
       blocking_instance,
       blocking_session
  from gv$session
 where blocking_session is not null
 order by blocking_instance,blocking_session,inst_id,seconds_in_wait ;
prompt "Holder and Waiter:"
select decode(request,0,'***holder:', '      waiter:')
holder,sid,id1,id2,lmode,request,type,ctime,block
from v$lock
where (id1,id2,type) in
(select id1,id2,type from v$lock where request>0)
order by id1,request;
prompt
prompt
prompt "Event enq: TX - row lock contention waiting object"
select se.inst_id,
       se.sid,
       se.serial#,
       obj.owner,
       obj.object_name,

```

```

        obj.object_type,
        se.ROW_WAIT_BLOCK#
    from gv$session se,
        dba_objects obj
    where se.event='enq: TX - row lock contention'
        and se.row_wait_obj# = obj.object_id;
prompt
prompt "Lock info"
select INST_ID, SID, TYPE, ID1, ID2, LMODE, REQUEST, CTIME, BLOCK from GV$lock where
block=1 or request<>0 ORDER BY 1,2;
prompt
prompt
prompt "Locked objects:"
SELECT b.session_id AS sid,
       NVL(b.oracle_username, ' (oracle)') AS username,
       a.owner AS object_owner,
       a.object_name,
       Decode(b.locked_mode, 0, 'None',
              1, 'Null (NULL)',
              2, 'Row-S (SS)',
              3, 'Row-X (SX)',
              4, 'Share (S)',
              5, 'S/Row-X (SSX)',
              6, 'Exclusive (X)',
              b.locked_mode) locked_mode,
       b.os_user_name
FROM   dba_objects a,
       v$locked_object b
WHERE  a.object_id = b.object_id
ORDER BY 1, 2, 3, 4;
prompt
prompt
prompt "blocking sql text"
select se.inst_id,
       se.sid,
       se.sql_id,
       sq.sql_text
from
    gv$session se, v$sql sq
where se.sql_id = sq.sql_id
      and (inst_id, sid) in
      (select inst_id,
             sid
      from

```

```

        (select distinct
            blocking_instance as inst_id,
            blocking_session as sid
        from gv$session
        where blocking_session is not null )
    );
prompt
prompt
prompt "blocked sql text"
select se.inst_id,
       se.sid,
       nvl(se.sql_id, se.prev_sql_id) sql_id,
       sq.sql_text
  from gv$session se, v$sql sq
 where blocking_session is not null
       and nvl(se.sql_id, prev_sql_id) = sq.sql_id;
prompt
prompt blocking session info
select se.inst_id,
       se.sid,
       nvl(se.sql_id, se.prev_sql_id) sql_id,
       logon_time,
       program,
       username,
       machine,
       module,
       last_call_et
  from
    gv$session se
 where (inst_id, sid) in
    (select inst_id,
         sid
      from
        (select distinct
            blocking_instance as inst_id,
            blocking_session as sid
          from gv$session
          where blocking_session is not null )
        );
prompt
prompt
prompt blocking session consume rollback segments
select se.inst_id,
       se.sid,

```

```

        se.serial#,
        nvl(se.sql_id, se.prev_sql_id) sql_id,
        tr.used_ublk,
        tr.used_urec
from
    gv$session se, gv$transaction tr
where (se.inst_id, se.sid) in
    (select inst_id,
        sid
    from
        (select distinct
            blocking_instance as inst_id,
            blocking_session as sid
        from gv$session
        where blocking_session is not null )
    )
and se.taddr = tr.addr;

```

功能:

检查系统是否出现堵塞, 如是否存在 block 的会话、是否有行锁、是否有对象锁、是否有 block 的会话;

3.3.2 目前锁等待的语句的 session 号以及语句

使用 DBA 用户登录数据库查询:

(1) 查看锁等待的session:

```

select s.sid, s.serial#, s.sql_id, s.username, s.lockwait, s.status, s.machine, s.program
from v$session s, v$locked_object l where s.sid=l.session_id;
sql_id即为该语句id

```

(2) 查看锁等待的对象:

```

col owner for a50
col object_name for a50
select b.owner, b.object_name, s.sid, s.serial#, s.sql_id, l.locked_mode from
v$locked_object l, dba_objects b, v$session s where b.object_id=l.object_id and
l.session_id=s.sid;

select 'alter system kill session ''' || b.sid || ',' || b.serial# || ''' immediate;' from
v$locked_object a, v$session b, dba_objects c
where a.session_id=b.sid and c.object_name='APM_PROJECT' and
a.object_id=c.object_id order by b.logon_time ;

```

(3) 查找是哪个会话正在blocking另一个会话

```

select blocking_session, sid, wait_class, seconds_in_wait from v$session where
blocking_session is not null;

```

说明: blocking_session为正在阻塞的sid, 产生了死锁, seconds_in_wait为等待该死锁的

秒数

(4) Kill具体的session

```
SQL> alter system kill session 'sid,serial#';
```

sid 和 serial#请根据具体会话信息填写。

如果再次查询发现该会话还在，并尝试重新 kill 会话的时候，报

ORA-00031--session marked for kill

表示：这个进程（会话）正在执行网络 IO、回滚一个事务或者执行数据的导入导出，这个进程（会话）是不能被杀掉的。

(5) 如果会话无法立即kill掉，可考虑kill操作系统进程

根据（2）和（3）查询的结果，查询（3）中的blocking_session对应的操作系统进程号

```
SQL> select spid, osuser, s.program from v$process p, v$session s where  
p.addr=s.paddr and s.sid='sid号';
```

3.3.3 检查行锁 (enq: TX - row lock contention)

使用 DBA 用户登录查询：

```
/****** to check which row causes lock contention *****/  
select program, machine,  
nvl(o.object_name, 'The LOCK is NOT on TABLE rows (likely on INDEX)') table_name,  
l.sql_text locking_sql,  
case substr(o.object_type, 1, 5) when 'TABLE' then  
'select * from ' || owner || '.' || o.object_name || ' where rowid=DBMS_ROWID.ROWID_CREATE  
(1, ' || 0.data_object_id || ', ' || ROW_WAIT_FILE# || ', ' || ROW_WAIT_BLOCK# || ', ' || ROW_WAIT_ROW# || ');'  
else  
nvl(o.object_type, 'Can NOT find lock records on non-Table objects')  
end find_lock_record  
from v$session s, v$sqlstats l, dba_objects o  
where event like 'enq: TX - row lock contention'  
and s.state='WAITING'  
and s.sql_id=l.sql_id(+)  
and s.ROW_WAIT_OBJ#=o.object_id(+);
```

3.3.4 查看被锁的对象信息

使用 DBA 用户登录查询：

```
set linesize 180  
col username for a15  
col lock_level for a15  
col owner for a10  
col object_name for a15  
col object_type for a15  
col machine for a15
```



```
col program for a20
col osuser for a15
col terminal for a10
SELECT s.username,
decode(l.type,'TM','TABLE LOCK',
'TX','ROW LOCK',
NULL) LOCK_LEVEL,
o.owner,o.object_name,o.object_type,
s.sid,s.serial#,s.terminal,s.machine,s.program,s.osuser
FROM v$session s,v$lock l,dba_objects o
WHERE l.sid = s.sid
AND l.id1 = o.object_id(+)
AND s.username is NOT NULL
and owner='&OWNER';
```

3.3.5 快速定位锁源头 session 和解锁

使用 DBA 用户登录数据库查询：
快速处理锁等待问题

1. 快速查看引起锁等待的源头session

```
SQL> col program for a30
SQL> col machine for a30
SQL> col username for a20
SQL> col event for a30
SQL> select
sid,serial#,program,machine,username,event,final_blocking_session,final_blocking_in
stance from v$session where blocking_session is not null;
```

SID	SERIAL#	PROGRAM	MACHINE	
USERNAME	EVENT		FINAL_BLOCKING_SESSION	
FINAL_BLOCKING_INSTANCE				
3218	22411	sqlplus@rac19c2 (TNS V1-V3)	rac19c2	
SYS	enq: TX - row lock contention			3226
2				
3224	14114	sqlplus@rac19c2 (TNS V1-V3)	rac19c2	
SYS	enq: TX - row lock contention			3226
2				
3229	59600	sqlplus@rac19c2 (TNS V1-V3)	rac19c2	
SYS	enq: TX - row lock contention			3226
2				

```

3234      23138 sqlplus@rac19c2 (TNS V1-V3)      rac19c2
SYS              enq: TX - row lock contention              3226
2

SQL>

```

2. 查看源头session的详细信息

```
SQL> select sid, serial#, program, machine, username, sql_id, event from v$session where
sid=3226;
```

SID	SERIAL#	PROGRAM	MACHINE
USERNAME	SQL_ID	EVENT	
3226	9898	sqlplus@rac19c2 (TNS V1-V3)	rac19c2
SYS		SQL*Net message from client	

```
SQL>
```

可以看到上面查出的信息中，sql_id为空，也就是说可能是该session之前执行过的一些DDL操作已执行完但未commit，这时，我们可以通过如下语句进行查看

查看源头session执行的SQL语句

```

set linesize 300
col sql_text for a80
col uname for a20
col schema for a20
select o.sql_id, o.sid, o.sql_text, o.user_name uname, s.schemaname schema,
to_char(to_date(t.start_time, 'MM/DD/RR HH24:MI:SS'), 'DD-Mon-YYYY HH24:MI:SS')
tran_start,
to_char(q.last_active_time, 'DD-Mon-YYYY HH24:MI:SS') last_active_time
from   v$open_cursor o, v$session s, v$transaction t, v$sqlarea q
where  o.saddr = s.saddr
and t.ses_addr = o.saddr
and q.sql_id = o.sql_id
and q.last_active_time >= to_date(t.start_time, 'MM/DD/RR HH24:MI:SS')
and    o.sid = s.sid
and    ( O.SID = &sid)
and (trim(upper(o.sql_text)) like '%INSERT%'
or trim(upper(o.sql_text)) like '%UPDATE%'
or trim(upper(o.sql_text)) like '%DELETE%'
or trim(upper(o.sql_text)) like '%LOCK%'
or trim(upper(o.sql_text)) like '%SELECT%FOR%UPDATE%')
order by last_active_time;

```

3. 评估是否可以kill掉占有锁的源头session

通过与业务人员沟通确定可以kill阻塞会话后，再进行kill操作

```
alter system kill session 'SID,SERIAL#';
```

3.4 语句执行慢

3.4.1 查看当前执行 SQL 的平均资源消耗

查看当前正在执行的 SQL，及其平均物理读，逻辑读，平均执行时间等

—通过总体消耗除以总执行次数（executions）来计算平均消耗。总体消耗和总执行次数均包含当前正在执行还未执行完的统计。所以计算出来的平均消耗会偏小一些，executions越小，误差会越大。如果executions=1 那么代表本次执行为实例启动后初次执行，平均消耗代表本SQL当前的实际消耗。

```
set linesize 200
col sql_text for a80
select s.sql_id,s.executions,round(s.rows_processed/s.executions,2)
rows_pec,round(s.disk_reads/s.executions) diskread_pec,
round(s.buffer_gets/s.executions)
bufferget_pec,round(s.cpu_time/1000000/s.executions,2) cputime_pec,
round(s.elapsed_time/1000000/s.executions,2) elapsedtime_pec,
round(s.sorts/s.executions) sort_pec,s.sql_text
from v$sqlstats s,v$session sess
where sess.sql_id=s.sql_id(+) and sess.username is not null and
sess.status='ACTIVE';
```

3.4.2 查看当前长时间运行的 SQL

查询 1:

--注：查看当前执行时间排在前 10 位的 SQL

```
alter session set nls_date_format='YYYY-MM-DD HH24:mi:ss';
set linesize 1000
col username for a20
col event for a40
col program for a30
select * from (select
sid,serial#,username,program,event,seconds_in_wait,sql_id,last_call_et as
elapsed_seconds from v$session where status='ACTIVE' and username is not null and
last_call_et<>0 and sql_id is not null order by last_call_et desc) where
rownum<=10;
```

查询 2:

--注：默认执行时间超过 **5 秒**（参数 `_sqlmon_threshold`）的才可能记入 `v$sql_monitor`

```
select
sid,status,program,username,sql_id,SQL_PLAN_HASH_VALUE,SQL_EXEC_START,round(ELAPSED
_TIME/1000000,0) as elapsed_seconds,sql_text from v$sql_monitor where
status='EXECUTING' ;
```

查询 3:

--注：默认操作时间（这些操作有可能是备份，恢复，收集统计信息，Hash Join,Sort ,Nested loop, Table Scan, Index Scan 等等）**超过 6 秒**的才可能记入 `v$session_longops` 。有些 SQL 可能总的执行时间很长，但每个操作都没有超过 6 秒，同样不会被捕获到。

```
select distinct a.sid,a.username,b.sql_id,b.sql_text
from v$session a,v$sql b,v$session_longops c
where
a.sid = c.sid and
a.sql_id = b.sql_id and
totalwork <> sofar;
```

3.4.3 查看耗费性能的 SQL

使用DBA用户登录查询：

```
SELECT C.SID, --会话序号 NUMBER
C.SERIAL#, ----会话序号 NUMBER
C.STATUS, --状态 VARCHAR2
A.USER_NAME, --数据用户
C.PROGRAM, --当前运行程序
C.TERMINAL, --终端名称
C.CLIENT_INFO AS IPADDRESS, -- IP地址,需要建logon on database触发器获取IP
B.DISK_READS, --读硬盘次数 NUMBER
B.EXECUTIONS, --执行次数 NUMBER
B.ROWS_PROCESSED, --通过语法分析的sql返回的数据总行数 NUMBER
ROUND(B.CPU_TIME / 1000000, 2) AS CPU_RUNTIME, --CPU_TIME[CPU时间，单位微秒
(百万分之一秒)NUMBER ]
ROUND(B.CPU_TIME / DECODE(B.EXECUTIONS, 0, 1, NULL, 1, B.EXECUTIONS) /
1000000,
3) AS EVERY_CPURTIME, --单次执行时间秒
ROUND(B.ROWS_PROCESSED /
DECODE(B.EXECUTIONS, 0, 1, NULL, 1, B.EXECUTIONS),
0) AS EVERY_ROWS, --每次返回数据行数
ROUND(B.DISK_READS /
DECODE(B.EXECUTIONS, 0, 1, NULL, 1, B.EXECUTIONS),
0) AS EVERY_DISKREADS, --每次读磁盘次数
```

```
ROUND((DECODE(B.BUFFER_GETS, NULL, 1, B.BUFFER_GETS) - B.DISK_READS) /
      DECODE(B.BUFFER_GETS, 0, 1, NULL, 1, B.BUFFER_GETS),
      2) HIT_RADIO, --命中率
B.SORTS, --排序次数 NUMBER
B.SQL_TEXT, --sql
B.SQL_FULLTEXT,
B.FIRST_LOAD_TIME --创建的时间 VARCHAR2
FROM V$OPEN_CURSOR A, V$SQLAREA B, V$SESSION C
WHERE A.ADDRESS = B.ADDRESS
      AND A.SID = C.SID
      -- AND UPPER(C.PROGRAM) <> 'PLSQLDEV.EXE'
ORDER BY A.USER_NAME;
```

3.4.4 查询目前正在运行的耗时最长的 10 个 sql 语句

使用 DBA 用户登录查询：

```
-- last_call_et可用于计算该SQL语句已经执行的时间（单位为秒）
alter session set nls_date_format='YYYY-MM-DD HH24:mi:ss';
set linesize 1000
col username for a20
col event for a40
col program for a30
select * from (select
sid,serial#,username,program,event,seconds_in_wait,sql_id,last_call_et from
v$session where status='ACTIVE' and username is not null and sql_id is not null
order by last_call_et desc) where rownum<=10;
```

3.4.5 查看未使用绑定变量的 sql 语句

使用 DBA 用户登录查询：

```
SQL> select plan_hash_value,count(*) from v$sqlarea group by plan_hash_value having
count(*)>200 order by 2 desc;
通过plan_hash_value定位到SQL文本
SQL>select sql_id,sql_text from v$sqlarea where plan_hash_value=&id and rownum<11;
```

3.4.6 通过查询 ash 报告中某一会话在某一段时间内的执行语句及执行时长

使用 DBA 用户登录查询：

```
select
SESSION_ID,SESSION_SERIAL#,TOP_LEVEL_SQL_ID,sql_id,sql_exec_start,count(*) from
v$active_session_history
where to_char(sample_time,'yyyymmdd hh24:mi:ss') between '20191018 09:00:00'
and '20191018 09:10:00' and session_id=2836 group by
```

```
SESSION_ID, SESSION_SERIAL#, TOP_LEVEL_SQL_ID, sql_id, sql_exec_start order by
sql_exec_start;
```

3.4.7 查看当前占用大量 TEMP 表空间的 SQL

—查看当前正在执行的SQL占用的temp表空间的大小

```
SET LINES 200 PAGES 1000
col username for a20
col osuser for a20
col tablespace for a20
col sql_text for a80
SELECT a.username, a.sid, a.serial#, a.osuser, b.tablespace,
b.blocks*8192/1024/1024 mb_used, c.sql_text
FROM v$session a, v$tempseg_usage b, v$sqlarea c
WHERE a.saddr = b.session_addr
AND c.address= a.sql_address
AND c.hash_value = a.sql_hash_value
ORDER BY b.tablespace, b.blocks;
```

查看当前使用temp表空间大于指定值的SQL, 例如, 查看使用temp表空间大于500M的SQL

```
col username for a30
col osuser for a30
col sql_text for a80
set linesize 250
SELECT sysdate, a.username, a.sid, a.serial#, a.osuser, b.blocks*8192/1024/1024 as
mb_used, c.sql_text
FROM v$session a, v$tempseg_usage b, v$sqlarea c
WHERE --b.tablespace = <temporary tablespace name to monitor> and
a.saddr = b.session_addr
AND c.address= a.sql_address
AND c.hash_value = a.sql_hash_value
AND b.blocks*8192/1024/1024 > 500
ORDER BY b.tablespace, b.blocks;
```

3.4.8 查看当前占用大量 UNDO 表空间的 SQL

大量使用 undo 表空间的 session 查询

如：当前使用超过500M的

```
select
s.username, s.sid, s.serial#, s.logon_time, s.sql_id, t.xidusn, t.ubafil, t.ubablk, round(t
.Used_ublk*8192/1024/1024, 1) as MB_USED, t.start_date, t.status from v$session
s, v$transaction t where s.saddr=t.ses_addr and t.Used_ublk>500*1024/8 order by
t.Used_ublk desc;
```

3.5 执行计划

3.5.1 查询某个 sql 语句最后一次执行的执行计划

```
select * from table(dbms_xplan.display_cursor('&sql_id', null, 'all last'));
```

说明：查询该语句最后一次执行的执行计划

```
SQL> select * from table(dbms_xplan.display_cursor('&sql_id', null, 'advanced last'));
Enter value for sql_id: fxbbw0d3arvat
old 1: select * from table(dbms_xplan.display_cursor('&sql_id', null, 'advanced last'))
new 1: select * from table(dbms_xplan.display_cursor('fxbbw0d3arvat', null, 'advanced last'))

PLAN_TABLE_OUTPUT
-----
SQL_ID fxbbw0d3arvat, child number 0
-----
select s.sid || ',' || s.serial# as
sess,s.username,o.blocks*8192/1024/1024 mb_used,
o.tablespace,h.sql_id,substr(h.sql_text,1,100) from v$tempseg_usage
```

3.5.2 查询某个 SQL 语句历史执行计划是否改变

使用 DBA 用户登录查询：

```
col BEGIN_INTERVAL_TIME for a30
col plan_hash_value for 99999999999999
set pagesize 2000
set linesize 400
set heading on
col BTIME for a20
col i_n for 9
select TO_CHAR(T.Begin_interval_time, 'MM-DD HH24:MI') BTIME, s.instance_number i_n,
s.snap_id,
s.sql_id, s.plan_hash_value,
s.EXECUTIONS_DELTA,
round(s.PX_SERVERS_EXECS_DELTA/s.EXECUTIONS_DELTA) P_PXSrv,
round(BUFFER_GETS_DELTA/executions_delta) P_gets ,
round(s.DISK_READS_DELTA/s.EXECUTIONS_DELTA) P_Reads,
round(CPU_TIME_DELTA/executions_delta/1000) P_cpu,
round(ROWS_PROCESSED_DELTA/executions_delta) P_rows ,
round(ccwait_delta/executions_delta/1000) P_ccwait,
round(s.ELAPSED_TIME_DELTA/s.EXECUTIONS_DELTA/1000/1000) P_elptime
from dba_hist_sqlstat s, dba_hist_snapshot t
```

```
where s.snap_id=t.snap_id and s.instance_number=t.instance_number
and s.EXECUTIONS_DELTA>0
and s.sql_id='&sqlid'
order by s.instance_number,s.sql_id,s.snap_id;
```

3.5.3 执行计划变更分析

首先,发现性能故障需要优先判断是属于 sql 性能还是数据库性能相关故障.一般来说,在 awr/ash 和 em 监控中发现对应 top event 和 latch/高消耗资源(cpu/io/mem)/争用有很有可能是 top sql 导致,需要查看对应的 top sql 信息;

其次,需要快速判断 top sql 是否有多个版本 sql plan

```
SELECT ROWNUM,
       v.plan_hash_value,
       TO_CHAR(v.avg_elapsed_time_secs, '9999999999999999D990'),
       TO_CHAR(v.avg_cpu_time_secs, '9999999999999999D990'),
       TO_CHAR(v.avg_user_io_wait_time_secs, '9999999999999999D990'),
       TO_CHAR(v.avg_concurrency_wait_time_secs, '9999999999999999D990'),
       TO_CHAR(v.avg_application_wait_time_secs, '9999999999999999D990'),
       TO_CHAR(v.avg_cluster_wait_time_secs, '9999999999999999D990'),
       TO_CHAR(v.avg_plsql_exec_time_secs, '9999999999999999D990'),
       TO_CHAR(v.avg_java_exec_time_secs, '9999999999999999D990'),
       v.avg_buffer_gets,
       v.avg_disk_reads,
       v.avg_direct_writes,
       v.avg_rows_processed,
       v.total_executions,
       v.total_fetches,
       v.total_loads,
       v.total_invalidations,
       v.total_parse_calls,
       v.child_cursors,
       v.min_optimizer_cost,
       DECODE(v.min_optimizer_cost, v.max_optimizer_cost, NULL, v.max_optimizer_cost),
       v.min_optimizer_env_hash_value,
       DECODE(v.min_optimizer_env_hash_value, v.max_optimizer_env_hash_value, NULL,
v.max_optimizer_env_hash_value),
       v.first_load_time,
       v.last_load_time,
       TO_CHAR(v.last_active_time, 'YYYY-MM-DD/HH24:MI:SS')
FROM
  (SELECT
    /*+ NO_MERGE */
    plan_hash_value,
    ROUND((SUM(elapsed_time) /SUM(GREATEST(executions, 1))) / 1e6, 3)
    avg_elapsed_time_secs,
```



```

        ROUND((SUM(cpu_time) /SUM(GREATEST(executions, 1))) / 1e6, 3)
avg_cpu_time_secs,
        ROUND((SUM(user_io_wait_time) /SUM(GREATEST(executions, 1))) / 1e6, 3)
avg_user_io_wait_time_secs,
        ROUND((SUM(concurrency_wait_time)/SUM(GREATEST(executions, 1))) / 1e6, 3)
avg_concurrency_wait_time_secs,
        ROUND((SUM(application_wait_time)/SUM(GREATEST(executions, 1))) / 1e6, 3)
avg_application_wait_time_secs,
        ROUND((SUM(cluster_wait_time) /SUM(GREATEST(executions, 1))) / 1e6, 3)
avg_cluster_wait_time_secs,
        ROUND((SUM(plsql_exec_time) /SUM(GREATEST(executions, 1))) / 1e6, 3)
avg_plsql_exec_time_secs,
        ROUND((SUM(java_exec_time) /SUM(GREATEST(executions, 1))) / 1e6, 3)
avg_java_exec_time_secs,
        ROUND(SUM(buffer_gets) /SUM(GREATEST(executions, 1)))
avg_buffer_gets,
        ROUND(SUM(disk_reads) /SUM(GREATEST(executions, 1))) avg_disk_reads,
        ROUND(SUM(direct_writes) /SUM(GREATEST(executions, 1)))
avg_direct_writes,
        ROUND(SUM(rows_processed) /SUM(GREATEST(executions, 1)))
avg_rows_processed,
        SUM(GREATEST(executions, 1)) total_executions,
        SUM(fetches) total_fetches,
        SUM(loads) total_loads,
        SUM(invalidations) total_invalidations,
        SUM(parse_calls) total_parse_calls,
        COUNT(*) child_cursors,
        MIN(optimizer_cost) min_optimizer_cost,
        MAX(optimizer_cost) max_optimizer_cost,
        MIN(optimizer_env_hash_value) min_optimizer_env_hash_value,
        MAX(optimizer_env_hash_value) max_optimizer_env_hash_value,
        MIN(first_load_time) first_load_time,
        MAX(last_load_time) last_load_time,
        MAX(last_active_time) last_active_time
FROM gv$sql
WHERE sql_id = '&v_sql_id'
GROUP BY plan_hash_value
ORDER BY 2
) v;

```

接着获取当前 sql plan 详细信息,判断执行计划中 estim rows 和 actual rows 差异,快速判断实际记录数和优化器估算记录数之间差异.

```
alter session set STATISTICS_LEVEL = ALL;
```

```
select * from table(dbms_xplan.display_cursor('&v_sql_id',null,'ADVANCED ALLSTATS
LAST PEEKED_BINDS'));
```

快速查看 sql 对应表统计信息是否准确,

```
SELECT ROWNUM,
       v.table_name,
       v.owner,
       CASE
         WHEN v.partitioned = 'YES'
         THEN LOWER(v.table_name
                   ||'_ '
                   ||v.owner)
         ||'=>'
         ||v.partitioned
         ELSE v.partitioned
       END
       || v.degree,
       v.temporary,
       v.num_rows,
       v.sample_size,
       v.sample_size_perc,
       v.last_analyzed,
       v.blocks,
       v.avg_row_len,
       v.global_stats,
       v.user_stats,
       v.stattype_locked,
       v.stale_stats,
       v.stale_stats_perc,
       LOWER(v.table_name
             ||'_ '
             ||v.owner)
       ||'">'
       ||v.columns
       || LOWER(v.table_name
               ||'_ '
               ||v.owner)
       ||'">'
       ||v.indexes
       || LOWER(v.table_name
               ||'_ '
               ||v.owner)
       ||'">'
       ||v.index_columns Versions
```

```

FROM
  ( WITH object AS
    (SELECT
      /*+ MATERIALIZE */
      object_owner owner,
      object_name name
    FROM gv$sql_plan
    WHERE inst_id IN
      (SELECT inst_id FROM gv$instance
      )
    AND sql_id      = '&v_sql_id'
    AND object_owner IS NOT NULL
    AND object_name IS NOT NULL
    UNION
    SELECT object_owner owner,
      object_name name
    FROM dba_hist_sql_plan
    WHERE sql_id      = '&v_sql_id'
    AND object_owner IS NOT NULL
    AND object_name IS NOT NULL
    ),
  plan_tables AS
  (SELECT
    /*+ MATERIALIZE */
    'TABLE' object_type,
    t.owner object_owner,
    t.table_name object_name
  FROM dba_tab_statistics t, -- include fixed objects
    object o
  WHERE t.owner      = o.owner
  AND t.table_name = o.name
  UNION
  SELECT 'TABLE' object_type,
    i.table_owner object_owner,
    i.table_name object_name
  FROM dba_indexes i,
    object o
  WHERE i.owner      = o.owner
  AND i.index_name = o.name
  )
SELECT
  /*+ NO_MERGE LEADING(pt s t m) */
  s.table_name,
  s.owner,

```

```

t.partitioned,
t.degree,
t.temporary,
s.num_rows,
s.sample_size,
CASE
  WHEN s.num_rows > 0
  THEN TO_CHAR(ROUND(s.sample_size * 100 / s.num_rows, 1), '99999990D0')
END sample_size_perc,
TO_CHAR(s.last_analyzed, 'YYYY-MM-DD/HH24:MI:SS') last_analyzed,
s.blocks,
s.avg_row_len,
s.global_stats,
s.user_stats,
s.stattype_locked,
s.stale_stats,
CASE
  WHEN s.num_rows > 0
  THEN TO_CHAR(ROUND((m.inserts + m.updates + m.deletes) * 100 / s.num_rows, 1),
'99999990D0')
END stale_stats_perc,
(SELECT COUNT(*)
FROM dba_tab_cols c
WHERE c.owner      = s.owner
AND c.table_name = s.table_name
) columns,
(SELECT COUNT(*)
FROM dba_indexes i
WHERE i.table_owner = s.owner
AND i.table_name    = s.table_name
) indexes,
(SELECT COUNT(*)
FROM dba_ind_columns ic
WHERE ic.table_owner = s.owner
AND ic.table_name    = s.table_name
) index_columns
FROM plan_tables pt,
dba_tab_statistics s,
dba_tables t,
sys.dba_tab_modifications m -- requires sys on 10g
WHERE pt.object_type = 'TABLE'
AND pt.object_owner  = s.owner
AND pt.object_name    = s.table_name
AND pt.object_type    = s.object_type

```

```
AND s.owner          = t.owner
AND s.table_name     = t.table_name
AND t.owner          = m.table_owner(+)
AND t.table_name     = m.table_name(+)
AND m.partition_name IS NULL
ORDER BY s.table_name,
        s.owner
) v;
```

通过 sql_id,查看对应表结构和相关统计信息;

如果统计信息不准,可以快速手工收集当前表统计信息(如果表收集统计信息,需要 cascade,同步收集对应索引统计信息);

```
1. exec dbms_stats.delete_table_stats(ownname=>'&owner',
    tabname=>'&table_name',cascade_indexes=>true);

2. exec dbms_stats.gather_table_stats(ownname=>'&owner',
    tabname=>'&table_name',
    estimate_percent => DBMS_STATS.AUTO_SAMPLE_SIZE,
    cascade=>true,
    degree=> auto_degree,
    method_opt=>'for all columns size 1');
```

根据实际情况选择合理参数和方式(比如表/索引/分区表或者索引等).

详细命令解说

```
DBMS_STATS.GATHER_TABLE_STATS (
    ownname          VARCHAR2,  --要分析表的拥有者
    tabname          VARCHAR2,  --要分析的表名
    partname         VARCHAR2 DEFAULT NULL,  --分区的名字,只对分区表或分区索引有用
    estimate_percent NUMBER    DEFAULT to_estimate_percent_type
                                (get_param('ESTIMATE_PERCENT')), --
    --采样行的百分比
    block_sample     BOOLEAN   DEFAULT FALSE,  ---是否用块采样代替行采样.
    method_opt       VARCHAR2 DEFAULT get_param('METHOD_OPT'),
    -- (直方图histogram选项)

    method_opt的取值如下(默认值为FOR ALL COLUMNS SIZE AUTO):
    for all columns: 统计该表所有列的直方图.
    for all indexed columns: 统计该表上定义索引列的直方图.
    for all hidden columns:统计你看不到列的直方图
    for columns <list> SIZE <N> | REPEAT | AUTO | SKEWONLY:
    N: 直方图桶数,取值范围[1,254],1相当于不收集直方图
    REPEAT - 只收集原本有直方图信息的列
    AUTO - 由Oracle自行决定N的大小
    SKEWONLY- 只收集非均匀分布列的直方图,系统自动决定桶数(bucket )
```

推荐使用AUTO或SKEWONLY

```

degree          NUMBER    DEFAULT to_degree_type(get_param('DEGREE')),
--- 决定并行度
granularity      VARCHAR2  DEFAULT GET_PARAM('GRANULARITY'),
-- (统计信息收集的粒度, 针对分区表)
global -> 全局统计信息
partition -> 分区统计信息
sub-partition -> 子分区统计信息
cascade         BOOLEAN   DEFAULT to_cascade_type(get_param('CASCADE')),
--- 是否同时收集索引的统计信息
stattab         VARCHAR2  DEFAULT NULL,
statid          VARCHAR2  DEFAULT NULL,
statown         VARCHAR2  DEFAULT NULL,
no_invalidate   BOOLEAN   DEFAULT to_no_invalidate_type (
                                get_param('NO_INVALIDATE')),
--- cursor依赖的统计信息不会立刻生效, 设置为FALSE才会即刻生效
stattype        VARCHAR2  DEFAULT 'DATA',
force           BOOLEAN   DEFAULT FALSE); ---即使表锁住了也收集统计信息.
```

同时根据实际情况,判断 **where** 条件中是否有合理索引,是否需要通过创建索引或者组合索引来调整执行计划,达到优化 sql;

在排除统计信息问题,索引不存在等问题后.在紧急情况下,作为优先恢复业务要求,可以优先用 **profile** 绑定之前执行计划,以达到不选择最差的执行计划目的.

需要强调,这是一种临时方案.如果要分析原因并真正优化,需要进一步收集 **sqlhc** 数据,并人工判断和分析,建议提交 **ssc** 进行分析.

3.5.4 绑定执行计划

在排除统计信息问题,索引不存在等问题后.在紧急情况下,作为优先恢复业务要求,可以优先用 **profile** 绑定之前执行计划,以达到不选择最差的执行计划目的.

需要强调,这是一种临时方案.如果要分析原因并真正优化,需要进一步收集 **sqlhc** 数据,并人工判断和分析,建议提交 **ssc** 进行分析.

1.绑定执行计划

绑定方法 1: 使用 [coe_load_sql_profile.sql](#) 脚本绑定

存在历史执行计划, 通过sql profile固定执行计划, 一步搞定
coe_load_sql_profile.sql

Sql_id Sql_id(两个sql_id是一样的)
会列出多个plan_hash_value, 选AVG_ET_SECS时间最短的那个

如果没有好的可用计划, 需要

先在sql加hint并执行, 获取正确执行计划, 记录sql_id:

```
leading(v t w u) USE_HASH(t) USE_HASH(w) USE_HASH(u) swap_join_inputs(w)
no_swap_join_inputs(u)
```

调用coe_load_sql_profile.sql

Sql_id(原sql_id)

Sql_id(加hint优化后的sql_id)

Plan_hash_value (加hint优化后的sql_id的plan_hash_value, 一般只会有一个)

绑定方法 2: 使用 coe_xfr_sql_profile.sql 脚本绑定

示例使用coe_xfr_sql_profile.sql脚本进行绑定

1. 生成绑定脚本

```
SQL> @coe_xfr_sql_profile.sql
```

Parameter 1:

SQL_ID (required)

Enter value for 1: 4tz8b5kxbgkd2

Parameter 2:

PLAN_HASH_VALUE (required)

Enter value for 2: 2213930528

Values passed to coe_xfr_sql_profile:

~~~~~

SQL\_ID : "4tz8b5kxbgkd2"

PLAN\_HASH\_VALUE: "2213930528"

```
SQL>BEGIN
```

```
2 IF :sql_text IS NULL THEN
```

```
3 RAISE_APPLICATION_ERROR(-20100, 'SQL_TEXT for SQL_ID &&sql_id. was not
found in memory (gv$sqltext_with_newlines) or AWR (dba_hist_sqltext).');
```

```
4 END IF;
```

```
5 END;
```

```
6 /
```

```
SQL>SET TERM OFF;
SQL>BEGIN
  2   IF :other_xml IS NULL THEN
  3       RAISE_APPLICATION_ERROR(-20101, 'PLAN for SQL_ID &&sql_id. and PHV
&&plan_hash_value. was not found in memory (gv$sql_plan) or AWR
(dba_hist_sql_plan).');
  4   END IF;
  5   END;
  6   /
SQL>SET TERM OFF;
```

Execute coe\_xfr\_sql\_profile\_4tz8b5kxbgkd2\_2213930528.sql  
on TARGET system in order to create a custom SQL Profile  
with plan 2213930528 linked to adjusted sql\_text.

COE\_XFR\_SQL\_PROFILE completed.  
SQL>

2. 将 force\_match=FALSE 改为force\_match=TRUE  
vi coe\_xfr\_sql\_profile\_4tz8b5kxbgkd2\_2213930528.sql

3. 执行绑定

```
SQL> @coe_xfr_sql_profile_4tz8b5kxbgkd2_2213930528.sql
```

4. 查看执行计划，绑定成功：

```
SQL> select * from table(dbms_xplan.display_cursor('4tz8b5kxbgkd2'));
```

PLAN\_TABLE\_OUTPUT

```
-----
SQL_ID 4tz8b5kxbgkd2, child number 0
-----
```

```
select count(2) from for_test WHERE object_name='testtable' AND
owner='test' AND namespace ='313227600018' AND DATA_OBJECT_ID =1 AND
object_type='table'
```

Plan hash value: 2213930528



| Id | Operation | Name | Rows | Bytes | Cost (%CPU) | Time |
|----|-----------|------|------|-------|-------------|------|
|----|-----------|------|------|-------|-------------|------|

PLAN\_TABLE\_OUTPUT

|     |                   |          |   |     |          |          |
|-----|-------------------|----------|---|-----|----------|----------|
| 0   | SELECT STATEMENT  |          |   |     | 51 (100) |          |
| 1   | SORT AGGREGATE    |          | 1 | 120 |          |          |
| * 2 | TABLE ACCESS FULL | FOR_TEST | 1 | 120 | 51 (0)   | 00:00:01 |

Predicate Information (identified by operation id):

```
2 - filter(("OBJECT_NAME"=' testtable' AND "OWNER"=' test' AND
          "NAMESPACE"=313227600018 AND "DATA_OBJECT_ID"=1 AND
          "OBJECT_TYPE"=' table'))
```

PLAN\_TABLE\_OUTPUT

Note

- SQL profile coe\_4tz8b5kxbgkd2\_2213930528 used for this statement

27 rows selected.

SQL>

## 2.查看绑定是否生效

通过 profile 绑定执行计划后观察新执行计划是否起效,如果没有起效,需要 purge cursor.

举例:

```
SQL> select address, hash_value, executions, loads, version_count, invalidations,
parse_calls from v$sqlarea where sql_text = 'select ename from emp where
empno=7900';
```

| ADDRESS | HASH_VALUE | EXECUTIONS | LOADS | VERSION_COUNT | INVALIDATIONS |
|---------|------------|------------|-------|---------------|---------------|
|---------|------------|------------|-------|---------------|---------------|

| PARSE_CALLS |
|-------------|
|-------------|

```
-----
0000000382E80750 1052545619 1 1 1 0
1
```

```
SQL> exec dbms_shared_pool.purge ('0000000382E80750,1052545619','C');
```

```
PL/SQL procedure successfully completed.
```

```
SQL> select address, hash_value, executions, loads, version_count, invalidations,
parse_calls from v$sqlarea where sql_text = 'select ename from emp where
empno=7900';
```

```
no rows selected
```

完成以后,观察数据库性能和之前对应sql性能

### 3.5.5 使用 sqlhc 查看执行计划情况

与执行计划相关,比较简单的办法是直接收取这个 sql 的 sqlhc,这样与执行计划相关的很多信息,都会展示在相应的 html 文件中

Sqlhc 的收集方法参见 6.3

#### Current Plans Summary (GV\$SQL)

Execution Plans performance metrics for gk2x7wjs52jf while still in memory. Plans ordered by average elapsed time.

| # | Plan HV    | Avg Elapsed Time (secs) | Avg CPU Time (secs) | Avg IO Time (secs) | Avg Conc Time (secs) | Avg Appl Time (secs) | Avg Clus Time (secs) | Avg PLSQL Time (secs) | Avg Java Time (secs) | Avg Buffer Gets | Avg Disk Reads | Avg Direct Writes | Avg Rows Proc | Total Execs | Total Fetch | Total Loads | Total Inval | Total Parse Calls | Total Child Cursors | Min Cost | Max Cost  | Min Opt Env HV | Max Opt Env HV | First Load          | Last Load           | Last Active         |
|---|------------|-------------------------|---------------------|--------------------|----------------------|----------------------|----------------------|-----------------------|----------------------|-----------------|----------------|-------------------|---------------|-------------|-------------|-------------|-------------|-------------------|---------------------|----------|-----------|----------------|----------------|---------------------|---------------------|---------------------|
| 1 | 3143864413 | 1.244                   | 1.228               | 0.008              | 0.005                | 0.000                | 0.002                | 0.000                 | 0.000                | 109410          | 9              | 0                 | 1             | 211         | 211         | 4           | 1           | 2                 | 1                   | 1970     | 824968226 |                |                | 2021-02-23/20:47:12 | 2021-03-25/10:57:30 | 2021-03-25/11:20:08 |
| # | Plan HV    | Avg Elapsed Time (secs) | Avg CPU Time (secs) | Avg IO Time (secs) | Avg Conc Time (secs) | Avg Appl Time (secs) | Avg Clus Time (secs) | Avg PLSQL Time (secs) | Avg Java Time (secs) | Avg Buffer Gets | Avg Disk Reads | Avg Direct Writes | Avg Rows Proc | Total Execs | Total Fetch | Total Loads | Total Inval | Total Parse Calls | Total Child Cursors | Min Cost | Max Cost  | Min Opt Env HV | Max Opt Env HV | First Load          | Last Load           | Last Active         |

- 执行计划的效率情况

### Current Execution Plans (last execution)

Captured while still in memory. Metrics below are for the last execution of each child cursor.  
If STATISTICS\_LEVEL was set to ALL at the time of the hard-parse then A-Rows column is populated.

Inst: 2 Child: 3  
Plan hash value: 3143864413

| Id   | Operation                           | Name                          | E-Rows | E-Bytes | Cost (%CPU) | E-Time   | OMem | lMem | Used-Mem |
|------|-------------------------------------|-------------------------------|--------|---------|-------------|----------|------|------|----------|
| 0    | SELECT STATEMENT                    |                               |        |         | 1970 (100)  |          |      |      |          |
| 1    | SORT AGGREGATE                      |                               | 1      |         |             |          |      |      |          |
| 2    | VIEW                                |                               | 7      |         | 1970 (1)    | 00:00:01 |      |      |          |
| 3    | UNION-ALL                           |                               |        |         |             |          |      |      |          |
| 4    | NESTED LOOPS                        |                               | 1      | 117     | 9 (0)       | 00:00:01 |      |      |          |
| 5    | NESTED LOOPS                        |                               | 1      | 117     | 9 (0)       | 00:00:01 |      |      |          |
| 6    | NESTED LOOPS                        |                               | 1      | 87      | 6 (0)       | 00:00:01 |      |      |          |
| 7    | NESTED LOOPS                        |                               | 1      | 77      | 5 (0)       | 00:00:01 |      |      |          |
| * 8  | TABLE ACCESS BY INDEX ROWID BATCHED | ZJJG_TRANS_DEP                | 1      | 67      | 4 (0)       | 00:00:01 |      |      |          |
| * 9  | INDEX RANGE SCAN                    | IDX_WD_ZJJG_QUERY_FLOW        | 1      |         | 2 (0)       | 00:00:01 |      |      |          |
| * 10 | INDEX UNIQUE SCAN                   | PK_ZJJG_TRD_BID               | 1      | 10      | 1 (0)       | 00:00:01 |      |      |          |
| * 11 | INDEX UNIQUE SCAN                   | PK_ZJJG_TRD_SUB_ACCT          | 1      | 10      | 1 (0)       | 00:00:01 |      |      |          |
| * 12 | INDEX RANGE SCAN                    | IDX_ZJJG_ACCT_DETAIL_SER_DATE | 1      |         | 2 (0)       | 00:00:01 |      |      |          |
| * 13 | TABLE ACCESS BY INDEX ROWID         | ZJJG_ACCT_DETAIL              | 1      | 30      | 3 (0)       | 00:00:01 |      |      |          |

- 执行计划的路径信息（还可以查询历史执行计划的情况）

## 3.6 统计信息

### 3.6.1 查询表的统计信息的最新时间和行数

```
col table_name for a20
col owner for a20
select owner, table_name, to_char(LAST_ANALYZED, 'MM/DD/YYYY HH24:MI'), num_rows from
dba_tables where OWNER=upper('&username') and table_name =upper('&tablename');
```

说明：输入表的用户名和表名。

### 3.6.2 手动收集统计信息

```
BEGIN
dbms_stats.gather_table_stats(ownname => '&username',
tabname => '&tablename',
estimate_percent => dbms_stats.auto_sample_size,
method_opt => 'for all indexed columns size repeat',
no_invalidate => FALSE,
degree => 4,
cascade => TRUE);
END;
/
```

说明：手动对某个用户的某张表收集统计信息，并将与该表相关的索引也进行更新。

**estimate\_percent => dbms\_stats.auto\_sample\_size** ## 根据 Best Practices for Gathering Optimizer Statistics 白皮书中的说明，建议使用采用默认值，即

dbms\_stats.auto\_sample\_size, 参见如下说明:

Oracle Database 11g introduced a new hash-based sampling algorithm that provides deterministic statistics, addressing the two key aspects of accuracy and speed<sup>2</sup>. It has accuracy close to a 100% sample (“reading all data”) but with the cost of, at most, a 10% sample. This new algorithm is only used when ESTIMATE\_PERCENT is set to AUTO\_SAMPLE\_SIZE (the default) in any of the DBMS\_STATS.GATHER\_\*\_STATS procedures.

It is highly recommended from Oracle Database 11g onward you let ESTIMATE\_PERCENT default. If you manually set the ESTIMATE\_PERCENT parameter, even if it is set to 100%, you will get the old statistics gathering algorithm.

采用新的算法后, 采样比例接近 100%, 采样 cost 只有接近原来 10%采样的消耗, 所以优势比较明显。

**method\_opt => for all indexed columns size repeat** ##表示当前有哪些列收集了直方图, 现在就对哪些列收集直方图, 对一个稳定运行的系统, 应该采用 repeat 方式  
**no\_invalidate => FALSE** ##共享池中涉及到该表的游标是否立即失效  
**degree => 4** ##根据表大小, CPU 资源和负载设置  
**cascade => TRUE** ##表示收集表的统计信息的时候, 是否级联收集索引的统计信息, 一般设置为 TRUE

### 3.6.3 给予普通用户授予统计信息更新的权限

```
grant execute on dbms_stats to &username;
```

Username 就是普通用户的用户名。

### 3.6.4 自动收集统计信息任务

#### 3.6.3.1 查看自动收集统计信息是否开启:

```
select client_name,status from DBA_AUTOTASK_CLIENT where client_name='auto  
optimizer stats collection';
```

ENABLED 就表示开启的。

#### 3.6.3.2 查询数据库自动收集统计信息的时间:

```
set linesize 1000  
col window_name for a20  
col REPEAT_INTERVAL for a80  
col DURATION for a20  
select w.window_name,w.repeat_interval,w.duration,w.enabled from  
dba_autotask_window_clients c,dba_scheduler_windows w where c.window_name =  
w.window_name and c.optimizer_stats = 'ENABLED';
```

```
set linesize 1000
col window_name for a20
col REPEAT_INTERVAL for a80
col DURATION for a20
SQL> select w.window_name,w.repeat_interval,w.duration,w.enabled from dba_autotask_window_clients c,dba_scheduler_windows w where
ts = 'ENABLED';
```

| WINDOW_NAME      | REPEAT_INTERVAL                                       | DURATION      | ENABL |
|------------------|-------------------------------------------------------|---------------|-------|
| THURSDAY_WINDOW  | freq=daily;byday=THU;byhour=22;byminute=0; bysecond=0 | +000 04:00:00 | TRUE  |
| TUESDAY_WINDOW   | freq=daily;byday=TUE;byhour=22;byminute=0; bysecond=0 | +000 04:00:00 | TRUE  |
| FRIDAY_WINDOW    | freq=daily;byday=FRI;byhour=22;byminute=0; bysecond=0 | +000 04:00:00 | TRUE  |
| WEDNESDAY_WINDOW | freq=daily;byday=WED;byhour=22;byminute=0; bysecond=0 | +000 04:00:00 | TRUE  |
| SUNDAY_WINDOW    | freq=daily;byday=SUN;byhour=6;byminute=0; bysecond=0  | +000 20:00:00 | TRUE  |
| MONDAY_WINDOW    | freq=daily;byday=THU;byhour=21;byminute=0; bysecond=0 | +000 05:00:00 | TRUE  |
| SATURDAY_WINDOW  | freq=daily;byday=SAT;byhour=6;byminute=0; bysecond=0  | +000 05:00:00 | TRUE  |

7 rows selected.

### (1) 修改统计信息收集时间

如:

### disable 某一天的统计信息任务

```
BEGIN
  DBMS_SCHEDULER.DISABLE(
    name => '"SYS"."MONDAY_WINDOW"',
    force => TRUE);
END;
/
```

### 修改周一从 21 点开始收集

```
BEGIN
  DBMS_SCHEDULER.SET_ATTRIBUTE(
    name => '"SYS"."MONDAY_WINDOW"',
    attribute => 'REPEAT_INTERVAL',
    value => 'freq=daily;byday=THU;byhour=21;byminute=0; bysecond=0 ');
END;
/
```

### 启用某一天的统计信息任务

```
BEGIN
  DBMS_SCHEDULER.ENABLE(
    name => '"SYS"."MONDAY_WINDOW"' );
END;
/
```

###修改某一天统计信息收集的持续时间

```
BEGIN
  DBMS_SCHEDULER.SET_ATTRIBUTE(
    name => '"SYS"."MONDAY_WINDOW"',
```

```
attribute => 'DURATION',
value => numtodsinterval(300,'minute'));
END;
/
```

### 3.7 SGA/PGA 使用

#### 3.7.1 查看 SGA 各组件大小

使用DBA用户登录查询：

查看当前SGA总大小

```
select round(sum(value)/1024/1024,0) SGA_MB from v$sga;
```

查看当前SGA各组件大小

```
select component, round(current_size/1024/1024,0) as MB_CURRENT,
round(min_size/1024/1024,0) as MB_MIN, round(max_size/1024/1024,0) as MB_MAX from
v$sga_dynamic_components;
```

#### 3.7.2 查看 shared pool 中占用内存较多的组件

查看shared pool中TOP 10的组件

```
select * from (select pool,name,bytes from v$sgastat where pool='shared pool' order
by bytes desc) where rownum<=10;
```

#### 3.7.3 查看 shared pool 中组件大小的历史信息

--查看shared pool中组件大小的历史信息

```
col BEGIN_INTERVAL_TIME for a40
```

```
col SNAP_ID for 999999999
```

```
col name for a30
```

```
col pool for a30
```

```
SELECT A.begin_interval_time,B.* FROM DBA_HIST_SGASTAT B,DBA_HIST_SNAPSHOT A
WHERE A.snap_id=B.SNAP_ID AND A.instance_number=B.INSTANCE_NUMBER AND
b.pool='shared pool' ORDER BY A.SNAP_ID,B.bytes;
```

#### 3.7.4 查看 SGA 各组件大小变动记录

查看 SGA 中各组件大小变动记录

```
set linesize 200
col start_time for a40
col end_time for a40
col component for a30
col name for a30
select start_time,
component,
oper_type,
oper_mode,
initial_size/1024/124,
final_size/1024/124,
end_time
from v$sga_resize_ops
order by start_time, component;
```

### 3.7.5 查看 shared pool 中各子池各组件大小

查看 shared pool 中各子池中各组件大小

```
SELECT
    subpool
    , name
    , SUM(bytes)
    , ROUND(SUM(bytes)/1048576, 2) MB
FROM (
    SELECT
        'shared pool (' || DECODE(TO_CHAR(ksmdsid), '0', '0 - Unused', ksmdsid) || '):'
    subpool
        , ksmsnam      name
        , ksmsslens    bytes
    FROM
        x$ksmss
    WHERE
        ksmsslens > 0
```

```
)
GROUP BY
    subpool
    , name
ORDER BY
    subpool    ASC
    , SUM(bytes) DESC;
```

### 3.7.6 查看当前 PGA 设定最大值

查看 PGA 设定的最大限制， `pga_aggregate_limit` 为实例可以使用的 PGA 的最大值，到达此值会报错 ORA-4030 错误

```
SQL> show parameter pga;

NAME                                TYPE                                VALUE
-----                                -
pga_aggregate_limit                 big integer 15000M
pga_aggregate_target                 big integer 379M
SQL>
```

### 3.7.7 查看实例当前使用的 PGA 总大小

查看实例当前使用的 PGA 总大小

```
--其中total为pga_aggregate_target设定的大小值。该值可以突破，真正限定使用上限的为pga_aggregate_limit

select name,total,round(used,2)used,round(total-used,2)
free,round(used/total*100,2)pctused
from (select 'PGA' name,(select value/1024/1024 total from v$pgastat where
name='aggregate PGA target parameter')total,
(select value/1024/1024 used from v$pgastat where name='total PGA allocated')used
from dual);
```

### 3.7.8 查看当前占用 PGA 较大的 session

查看当前占用 PGA 较大的 session

```
set long 20000 longchunk 20000
break on sql_fulltext
```



```
col sql_fulltext for a100
set pagesize 10000
set linesize 200

select s.sid,s.serial#,s.sql_id,round(pga_used_mem/1024/1024,2)
pga_used_MB,round(PGA_ALLOC_MEM/1024/1024,2)
PGA_ALLOC_MB,round(PGA_MAX_MEM/1024/1024,2) PGA_MAX_MB,sql_fulltext from

  v$session s, v$process p, v$sqlarea q

  where p.addr = s.paddr

  and s.sql_id=q.sql_id(+)

  order by PGA_ALLOC_MEM;
```

或者使用如下方式查看

```
set pages500 lines110 trims on
clear col
col name format a30
col username format a20
break on username nodup skip 1

select vses.username||': '||vsst.sid||','||vses.serial# username, vstt.name,
round(max(vsst.value)/1024/1024,2) value_MB
from v$sesstat vsst, v$statname vstt, v$session vses
where vstt.statistic# = vsst.statistic# and vsst.sid = vses.sid and vstt.name in
('session pga memory')
--,'session pga memory max','session uga memory','session uga memory max','session
cursor cache count','session cursor cache hits','session stored procedure
space','opened cursors current','opened cursors cumulative') --and vses.username is
not null

group by vses.username, vsst.sid, vses.serial#, vstt.name order by
value_MB,vses.username, vsst.sid, vses.serial#, vstt.name;
```

## 4 常见等待事件分析

### 4.1 Log file sync

#### 4.1.1 Log file sync

##### 【故障现象】

log file sync是非常常见的等待事件，有log file sync并不一定出现性能问题，但是如果log file sync的平均等待时间过高（超过5ms），或者log file sync等待事件过多（这种也分情况），那么可能就会出现性能问题

##### 【可能故障原因】

- redo磁盘性能问题
- COMMIT/ROLLBACK过多
- IO 争用等

##### 【应急分析】

一：是否是IO异常导致lgwr进程写入缓慢

1. 可以查看lgwr的trace文件

more \*lgwr\*.trc 看是否有类似如下 lgwr 写入缓慢的 warning 信息，如果有写入十分缓慢的信息，则大多说明 IO 不佳，可能与 IO 子系统，包括 IO 链路和存储相关。

```
*** 2022-06-17 11:36:54.421
Warning: log write elapsed time 783ms, size 4KB
*** 2022-06-17 11:36:55.436
Warning: log write elapsed time 1014ms, size 387KB
*** 2022-06-17 11:37:01.939
Warning: log write elapsed time 670ms, size 5KB
*** 2022-06-17 11:37:05.291
Warning: log write elapsed time 645ms, size 0KB
*** 2022-06-17 11:37:09.550
Warning: log write elapsed time 658ms, size 4KB
*** 2022-06-17 11:37:48.166
Warning: log write elapsed time 501ms, size 4KB
*** 2022-06-17 11:37:49.969
Warning: log write elapsed time 534ms, size 53KB
*** 2022-06-17 11:37:50.705
Warning: log write elapsed time 693ms, size 6KB
*** 2022-06-17 11:37:54.342
Warning: log write elapsed time 514ms, size 9KB
*** 2022-06-17 11:37:56.541
Warning: log write elapsed time 609ms, size 13KB
```

2. 比较 log file sync 和 log file parallel write 的平均等待时间

## 比较'log file sync'和'log file parallel write'的平均等待时间。

等待事件'log file parallel write'表示 LGWR 正在等待写 redo 操作。该事件的持续时间就是等待 IO 操作部分的时间。

结合事件“log file sync”看同步操作消耗在 IO 的时间，由此推断，有多少处理时间消耗在 CPU 上。

| <b>Top 5 Timed Foreground Events</b> |         |         |               |           |            |  |
|--------------------------------------|---------|---------|---------------|-----------|------------|--|
| Event                                | Waits   | Time(s) | Avg wait (ms) | % DB time | Wait Class |  |
| log file sync                        | 868,667 | 62,544  | 72            | 58.29     | Commit     |  |

| <b>Foreground Wait Events</b>                                                                                                                                                                                                                                                                                                  |         |             |                     |               |            |           |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|-------------|---------------------|---------------|------------|-----------|
| <ul style="list-style-type: none"> <li>s - second, ms - millisecond - 1000th of a second</li> <li>Only events with Total Wait Time (s) &gt;= .001 are shown</li> <li>ordered by wait time desc, waits desc (idle events last)</li> <li>%Timeouts: value of 0 indicates value was &lt; .5%. Value of null is truly 0</li> </ul> |         |             |                     |               |            |           |
| Event                                                                                                                                                                                                                                                                                                                          | Waits   | %Time -outs | Total Wait Time (s) | Avg wait (ms) | Waits /txn | % DB time |
| log file sync                                                                                                                                                                                                                                                                                                                  | 868,667 | 0           | 62,544              | 72            | 1.01       | 58.29     |

| <b>Background Wait Events</b>                                                                                                                                                                                                                                       |        |             |                     |               |            |           |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|-------------|---------------------|---------------|------------|-----------|
| <ul style="list-style-type: none"> <li>ordered by wait time desc, waits desc (idle events last)</li> <li>Only events with Total Wait Time (s) &gt;= .001 are shown</li> <li>%Timeouts: value of 0 indicates value was &lt; .5%. Value of null is truly 0</li> </ul> |        |             |                     |               |            |           |
| Event                                                                                                                                                                                                                                                               | Waits  | %Time -outs | Total Wait Time (s) | Avg wait (ms) | Waits /txn | % bg time |
| log file parallel write                                                                                                                                                                                                                                             | 93,144 | 0           | 5,495               | 59            | 0.11       | 40.31     |

上面的例子显示了'log file sync' 和 'log file parallel write' 都有很高的等待时间

注意:

如果'log file sync'的时间消耗在'log file parallel write'上的比例高，那么大部分的等待时间是由于 IO（等待 redo 写入）。应该检查 LGWR 在 IO 方面的性能。作为一个经验法则，'log file parallel write'平均时间超过 20 毫秒，意味着 IO 子系统有问题。（当然这个时间对于拥有更多 cache 的现代的存储系统以及 SSD，NVRAM 来说会更小。因为硬件等诸多条件的影响，具体的值可参考正常时间段，可接受范围内的平均等待时间。）

LGWR 倾向于做很多小的 IO 操作，而不是大块的 IO 操作。大部分的磁盘配置并不能在这种场景下很好的工作，可能会发生间歇性物理 IO 缓慢。但是从平均等待时间来看，IO 等待的时间并不长（它们仅仅是善于处理这种小而多的 IO 负载），磁盘设备提供商据此断定没有磁盘问题。因为系统里还有其它的 IO 操作，这些正常的 IO 操作的等待时间很短，所有这些 IO 操作平均起来的等待时间并不长，这就掩盖了间歇性物理 IO 缓慢（IO 异常值）的问题。但是间歇性物理 IO 缓慢的问题会造成很高的'log file sync'，虽然平均的'log file parallel write'是处于正常性能的范围。

如果你发现系统的'log file sync'很高，但是'log file parallel write'是处于正常的范围，那么这可能是由于间歇性物理 IO 缓慢导致的。可以检查 AWR 的部分，也可以使用一些像 OSWatcher 一样的工具(参照 [Document 301137.1](#)) 来确定是否存在间歇性物理 IO 缓慢。如果可以确定存在间歇性物理 IO 缓慢，那么你需要与磁盘提供商一起来解决这个问题。

### 建议

- 与系统管理员一起检查重做日志所在的文件系统以及本地卷的位置，以提高 IO 性能。
  - 不要把重做日志放在需要额外计算的较老的或者复杂的 RAID 上，比如 RAID-5 或者 RAID-6 或者只有很少 cache 部件的存储上
  - 不要把重做日志放在上一代或者较老的 Solid State Disk (SSD)磁盘上
- 虽然通常情况下，SSD 写入性能好于平均水平，他们可能会遇到写峰值,从而导致大量的严重'log file sync'等待并引发数据库性能不稳定或者 hung 住(关于这一点您需要详尽的测试，因为我们也碰到一些即使 SSD 的写性能不稳定但数据库性能仍然可

以接受的系统)

(Engineered Systems (Exadata, SuperCluster 和 Oracle Database Appliance) 除外, 因为在这些系统上已经为使用 SSD 来存放重做日志而做了额外的优化)

- 监控其他可能需要写到相同路径的进程, 确保该磁盘具有足够的带宽, 足以应付所要求的容量。如果不能满足, 增加硬盘或者更快的磁盘设备或者把这些文件分散到不同的磁盘设备。
- 确保 LOG\_BUFFER 不要太大, 一个非常大的 log\_buffer 的不利影响就是刷新需要更长的等待时间。当缓冲区满了的时候, 它必须将所有数据写入到重做日志文件。LGWR 将一直等待, 直到最后的 I/O 完成。

## 二：是否为应用程序提交过多

在这种情况下, 要回答的问题是“**是否应用程序 commit 过于频繁?**”。

如果是, 那么过多的 commit 活动可能会导致性能问题, 因为把 redo 从日志缓冲区刷新到重做日志可能会导致等待'log file sync'。

如果'log file sync'的平均等待时间比'log file parallel write'高很多, 这意味着大部分时间等待可能不是由于等待 redo 的写入, 因而问题的原因可能不是 IO 慢导致。剩余时间是 CPU 活动, 而且是过多的 commit 导致的最常见的竞争。

此外, 如果'log file sync'的平均等待时间低, 但等待次数高, 那么应用程序可能 commit 过于频繁。

### 比较 user commit/rollback 同 user calls 比值的平均值

在 AWR 或 Statspack 报告中, 如果每次 commit/rollback 的平均 user calls ( $\text{user calls}/(\text{user commits}+\text{user rollbacks})$ ) 小于 30, 表明 commit 过于频繁

| Instance Activity Stats     |           |            |           |
|-----------------------------|-----------|------------|-----------|
| • Ordered by statistic name |           |            |           |
| Statistic                   | Total     | per Second | per Trans |
| user calls                  | 4,962,433 | 2,755.00   | 5.76      |
| user commits                | 861,662   | 478.37     | 1.00      |

在上面的例子中, 我们看到, 平均每 5.76 次 user calls 就会有一次 commit, 大约高出建议值 5 倍。基于经验, 我们期望 user calls/user commit 至少是 25。当然, 这取决于应用程序设计。

### 建议

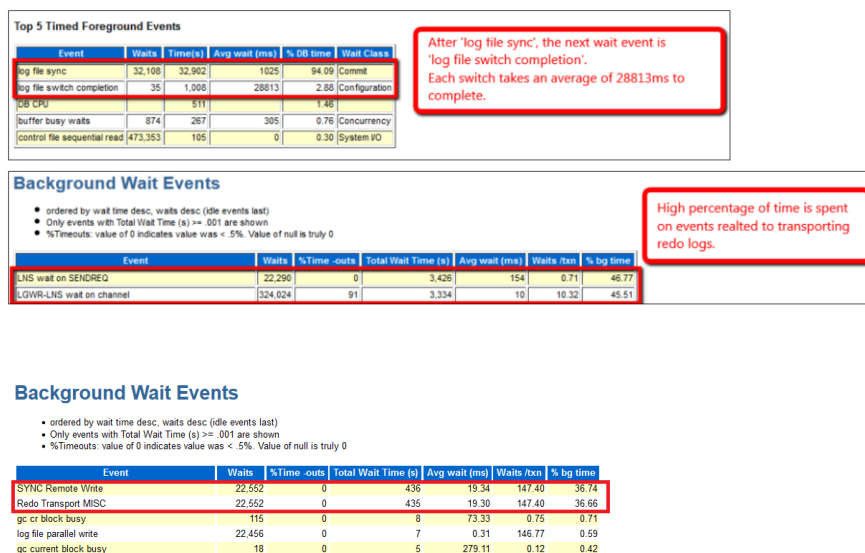
- 如果有很多短事务, 看是否可能把这些事务组合在一起, 从而减少 commit 操作。因为每一个 commit 都必须收到相关 REDO 已写到磁盘上的确认, 额外的 commit 会显著的增加开销。虽然 Oracle 可以将某些 commit 组合在一起, 通过事务的批处理来减少 commit 的总体数量还是可以带来非常有益的效果。

- 看看是否有操作可以使用 COMMIT NOWAIT 选项（务必在使用前应明白语义）。
- 看看是否有操作可以安全地使用 NOLOGGING/ UNRECOVERABLE 选项完成。

### 三：其他相关联的等待事件引起（如 ADG 的远程日志传输）

检查 AWR 报告，看是否有跟 LGWR 相关的，显示占用了显著数量时间的其他事件，因为这可能会给出导致这个问题的一个线索。前台和后台事件都应该进行检查。

例如下面的 AWR 显示某些其他前台和后台等待事件等待高，意味着 ADG 传输重做日志到远程位置的问题，如网络异常，带宽不足，短时间内产生大量日志等，这可能会导致 foreground 进程等待“log file sync”。



### 【信息收集】

Script to Collect Log File Sync Diagnostic Information (lfsdiag.sql) (Doc ID 1064487.1)



lfsdiag.sql

执行 MOS 文档中的脚本，收集相关。

收集 alert file 及相关 trace file，每个节点

生成问题正常时间段/有问题时间段的 AWR/ASH 报告，每个节点

如果 log file parallel write 等待时间超过 10ms（正常 1ms），建议做 dd 检查 IO 链路（包括：OS，共享文件系统，光纤卡，存储，磁盘等），并做正常和异常时间段的对比。以及相关主机上是否有告警灯亮；

OSW 数据，参见 7.6 章节

如果怀疑是 bug，需要上传 opatch 详细列表(clusterware 和 db software)

opatch lsinventory -detail

## 4.2 Log buffer space

### 4.2.1 Log buffer space

#### 【故障现象】

log buffer space等待产生的本质原因是redo log的产生速度大于LGWR写出redo log到online logfile的速度。因此，如果有较为严重的log buffer space等待，会引起需要写入redo log的DML操作变得缓慢。一些select等操作可能不会受到影响。

Top 10 Foreground Events by Total Wait Time

| Event                    | Waits   | Total Wait Time (sec) | Avg Wait  | % DB time | Wait Class    |
|--------------------------|---------|-----------------------|-----------|-----------|---------------|
| buffer busy waits        | 219,406 | 196.4K                | 895.08ms  | 27.7      | Concurrency   |
| db file sequential read  | 779,411 | 139.9K                | 179.55ms  | 19.7      | User I/O      |
| log buffer space         | 19,309  | 55.6K                 | 2880.66ms | 7.8       | Configuration |
| read by other session    | 57,411  | 30.2K                 | 525.48ms  | 4.3       | User I/O      |
| DB CPU                   |         | 26.3K                 |           | 3.7       |               |
| log file sync            | 14,526  | 26.1K                 | 1799.74ms | 3.7       | Commit        |
| db file parallel read    | 20,322  | 16.8K                 | 827.85ms  | 2.4       | User I/O      |
| enq: SQ - contention     | 27,502  | 15.8K                 | 575.53ms  | 2.2       | Configuration |
| Disk file operations I/O | 16,194  | 13.9K                 | 857.59ms  | 2.0       | User I/O      |
| library cache lock       | 316     | 8811                  | 27.88 s   | 1.2       | Concurrency   |

#### 【可能故障原因】

- 有产生大量redo log的操作
- Redo日志所在磁盘IO异常或者IO耗尽
- Log\_buffer太小，不能很好缓冲突然大量的redo log的写入

#### 【应急分析】

一：是否是IO异常导致lgwr进程写入缓慢

1. 可以查看lgwr的trace文件

more \*lgwr\*.trc 看是否有类似如下 lgwr 写入缓慢的 warning 信息，如果有写入十分缓慢的信息，则大多说明 IO 不佳，可能与 IO 子系统，包括 IO 链路和存储相关。

```
*** 2022-02-27T01:36:25.454517+08:00
Warning: log write elapsed time 2453ms, size 3601KB

*** 2022-02-27T01:36:27.258443+08:00
Warning: log write elapsed time 1803ms, size 6021KB

*** 2022-02-27T01:36:30.289372+08:00
Warning: log write elapsed time 2645ms, size 2200KB

*** 2022-02-27T01:36:33.333831+08:00
Warning: log write elapsed time 2503ms, size 3829KB

*** 2022-02-27T01:36:34.539608+08:00
Warning: log write elapsed time 921ms, size 3942KB

*** 2022-02-27T01:36:36.590738+08:00
Warning: log write elapsed time 743ms, size 1987KB

*** 2022-02-27T01:36:40.987084+08:00
Warning: log write elapsed time 4138ms, size 2901KB

*** 2022-02-27T01:36:45.266830+08:00
Warning: log write elapsed time 1055ms, size 196KB

*** 2022-02-27T01:38:15.270852+08:00
Warning: log write elapsed time 1216ms, size 4130KB

*** 2022-02-27T01:38:16.247385+08:00
Warning: log write elapsed time 574ms, size 3610KB

*** 2022-02-27T01:38:28.409786+08:00
Warning: log write elapsed time 12162ms, size 1781KB
```

#### 建议:

- 与系统管理员一起检查重做日志所在的文件系统以及本地卷的位置,以提高 IO 性能。
  - 重做日志应该放在单独的物理磁盘,避免与其他数据文件等放在相同的磁盘。当 redo 文件存在多个副本时,每个副本也应该单独存放。
  - 不要把重做日志放在需要额外计算的较老的或者复杂的 RAID 上,比如 RAID-5 或者 RAID-6 或者只有很少 cache 部件的存储上
  - 不要把重做日志放在上一代或者较老的 Solid State Disk (SSD)磁盘上
- 虽然通常情况下,SSD 写入性能好于平均水平,他们可能会遇到写峰值,从而导致大量的严重'log file sync'等待并引发数据库

## 二: 查看是否存在产生大量 redo log 的 DML 操作

#### 使用 DBA 用户登录查询:

```
alter session set nls_date_format='YYYY-MM-DD HH24:mi:ss';
set linesize 1000
col username for a20
col event for a40
col program for a30
select * from (select
sid,serial#,username,event,sql_id,last_call_et,program,seconds_in_wait from
v$session where status='ACTIVE' and username is not null order by last_call_et
desc) where rownum<=10;
```

**建议:**

- 若存在可能产生大量的 redo 日志，并且不经常执行的 SQL 操作，则需要与应用确认并优化相关语句。

### 三：确认 log\_buffer 参数是否设置太小

Show parameter log\_buffer;

最佳实践的建议值为 log\_buffer 设置为 100M.

注：log\_buffer 参数为静态参数，需要重启实例。

## 4.3 Buffer busy waits

### 4.3.1 Buffer busy waits

#### 【故障现象】

当一个session需要访问一个数据块（database block）时候，此数据块正在被其他session所访问。就会出现buffer busy waits等待。主要的情况有如下两种：

- 一个session请求修改data block时，发现其它session正以不兼容的模式占有这个data block（即可能正被其他session修改）
- 一个session读取data block时，发现该data block正在被其它session从数据文件读取到buffer cache（这种情况已单独计算为read by other session等待事件）

#### 【可能原因】

- 应用发起大量连接/并发突增
- 大量session同时修改少量的block

#### 【应急分析】

1. 可以通过如下语句来定位等待buffer busy waits最多的sql语句

```
select event ,sql_id,count(*) ct from v$session
where STATUS='ACTIVE'
group by event,sql_id
order by ct;
```

```
select sql_id,sql_text from v$sqlarea where sql_id='&sql_id';
```



## 2.查看等待 buffer busy waits 涉及的对象

可通过如下语句来统计等待 buffer busy waits 最为严重的对象

```
select 'Segment Header' class,
       a.segment_type,
       a.segment_name,
       a.partition_name
  from dba_segments a, VSESSION b
 where a.header_file = b.p1
       and a.header_block = b.p2
       and b.event = 'buffer busy waits'
union ALL
select 'Freelist Groups' class,
       a.segment_type,
       a.segment_name,
       a.partition_name
  from dba_segments a, VSESSION b
 where a.header_file = b.p1
       and b.p2 between a.header_block + 1 and
         (a.header_block + a.freelist_groups)
       and a.freelist_groups > 1
       and b.event = 'buffer busy waits'
union ALL
select a.segment_type || ' block' class,
       a.segment_type,
       a.segment_name,
       a.partition_name
  from Db_Extents a, VSESSION b
 where a.file_id = b.p1
       and b.p2 between a.block_id and a.block_id + a.blocks - 1
       and b.event = 'buffer busy waits'
       and not exists (select 1
                       from dba_segments
                       where header_file = b.p1
                         and header_block = b.p2);
```

建议的处理方式:

- 如果问题是由于大量相同SQL批量执行导致，联系相关应用及时处理并发过高的异常。
- 尽量使得同时修改的数据落在不同的block，即将数据打散，如常用的方式为创建反转索引，哈希分区等

## 4.4 Row cache lock

### 4.4.1 Row cache lock

#### 【故障现象】

这个等待表示的是等待data dictionary cache中的锁。用P1表示等待的是哪种类型的dictionary cache

#### 【可能原因】

常见的类型及可能原因，（如下仅为一般性指导，不同类型需要具体分析）

##### DC\_TABLESPACES

最可能的原因是新 extent 的分配。如果 extent 大小设置过小，那么应用程序可能会不断地要求新的 extent，这可能导致争用。你有很小的 extent 尺寸，正在迅速增长的对象吗？（通过查找具有大量 extents 的对象可以定位它们）。检查 insert/update 活动的 trace，查找那些就有很多 extents 的对象。

##### DC\_SEQUENCES

检查应用程序用到的 sequence 的 cache 的大小

##### DC\_USERS

一个会话正在对一个用户执行 GRANT，与此同时此用户正在登录到数据库中，此时可能会发生死锁或导致“WAITED TOO LONG FOR A ROW CACHE ENQUEUE LOCK!”。

##### DC\_OBJECT\_IDS

[Document 11693365.8](#) Bug 11693365 - Concurrent Drop table and Select on Reference constraint table hangs(deadlock) - fixed in 12.1

##### DC\_SEGMENTS

这很可能是 segment 的分配导致的。确定持有锁的用户正在做什么并使用 errorstacks 进行诊断。

## DC\_ROLLBACK\_SEGMENTS

这可能是由于 rollback 段的分配导致的。正如 dc\_segments，确定谁持有锁并收集 errorstack 来进行诊断。请记住，在多节点系统（RAC）上，持有者可能在另一节点上，因此需要所有节点的 systemstate。

## DC\_TABLE\_SCNS

[Document 5756769.8](#) Bug 5756769 - Deadlock between Create MVIEW and DML - fixed in 10.2.0.5 ,11.1.07 and 11.2.0.1

## DC\_AWR\_CONTROL

此 enqueue 关系到 AWR（Automatic Workload Repository）的控制权。任何操纵 AWR 资料库的操作将持有它。要分析这个问题，需要查找是那些进程阻塞了它们。

## 【应急分析】

1. 查看当前等待row cache lock的P1字段，其代表dictionary cache 类型

```
SQL> select p1text, p1, p2text, p2, p3text, p3 from v$session where event='row cache lock';
```

| P1TEXT     | P1     | P2TEXT    | P2 | P3TEXT | P3 |
|------------|--------|-----------|----|--------|----|
| cache id 8 | mode 0 | request 3 |    |        |    |

2. 确认等待的 dictionary cache 类型

```
SQL> select parameter, count, gets, getmisses, modifications from v$rowcache where cache#=8;
```

| PARAMETER    | COUNT | GETS  | GETMISSES | MODIFICATIONS |
|--------------|-------|-------|-----------|---------------|
| DC_SEQUENCES | 869   | 76843 | 508432    | 4500          |

需要注意的是：

确认 dictionary cache 类型一般会从 v\$rowcache 中查询

```
SELECT DISTINCT cache#, type, parameter
FROM v$rowcache
WHERE cache# = &P1
```

在 11G 中，v\$rowcache 中可能不包含所有有效的 cache#，如果上面的查询没有返回结果，也可以使用如下的方式查询：

```
SELECT DISTINCT cache#, cache_name
```

```
FROM v$rowcache_parent
WHERE cache# = &P1;
```

或者:

```
set pages 1000
column cache# format 99999
column name format a33
column latch# format 999999
select distinct s.kqrstcln latch#,r.cache#,r.parameter name,r.type,r.subordinate#
from v$rowcache r,x$kqrst s
where r.cache#!=s.kqrstcid
order by 1,4,5;
```

### 3.具体分析

需要根据 data dictionary 的类型具体分析，每种类型可能对应着一些常见的原因。参考上面的【可能原因】章节，就一般性的常见原因进行分析和排除，如不能确定问题原因，可以收集相关信息进行深入分析

### 4.相关信息收集

#### 1) Systemstate

当问题发生时，可能会自动产生一个 systemstate dump 文件。如:

Wed Sep 21 13:39:19 2011

System State dumped to trace file /oracle/diag/rdbms/..../.trc

如未产生，则可以考虑手动收集（必须是在问题发生时收集）。

具体方法可参考 4.2.4 收集 systemstate 信息

#### 2) AWR, ADDM 和 ASH 报告

收集两份 AWR 报告，一份有问题时间段的，另一个是没有问题时间段的，因为这些可以帮助我们理解问题发生时数据库的状况。AWR, ADDM, ASH 报告，可以相互取长补短，从而更完整地理解整个问题。

取决于 AWR 快照生成的时间间隔，收集最小时间间隔的报告。默认的快照是一个小时的时间间隔。

```
SQL>@$ORACLE_HOME/rdbms/admin/awrrpt.sql
```

```
SQL>@$ORACLE_HOME/rdbms/admin/addmrpt.sql
```

```
SQL>@$ORACLE_HOME/rdbms/admin/ashrpt.sql
```

鉴于分析 systemstate 是一件很复杂的事情，您可以创建一个服务请求，并上传 alert.log, systemstate dump, 以及问题发生前和问题发生时的 AWR 报告请 Oracle 技术支持来分析。

## 4.5 Db file scattered read

### 4.5.1 Db file scattered read

#### 【故障现象】

当一个session在等待multiblock IO（多块读）时就会显示这个等待事件。

多发生在如下两种情况下：

- FULL TABLE SCANS
- INDEX FAST FULL SCANS

Oracle会从数据文件一次读取DB\_FILE\_MULTIBLOCK\_READ\_COUNT参数设置的block，并分散到buffer cache中。目的是提高full scan的执行效率。

#### 【可能原因】

一般的原因数据库上存在较多FULL TABLE SCANS 或者INDEX FAST FULL SCANS。全表扫或者索引快速全扫可能意味着性能不佳。需要找出等待db file scattered read的SQL语句看是否有优化空间。

#### 【应急分析】

##### 1. 查看等待db file scattered read的sql\_id

```
SQL> select sid,serial#,sql_id,event,program,machine from v$session where event=' db file scattered read' ;
```

或者通过如下的方式查看 1 分钟内等待 db file scattered read 最多的 SQL

```
SQL> select sql_id,event,count(1) ct from v$active_session_history where event=' db file scattered read ' and sample_time>sysdate-1/24/60 group by sql_id,event order by ct;
```

另外，可以辅助查看 AWR 报告的"Top SQL by Disk Reads"，查看引起大量 IO 操作的 sql 语句

##### 2. 查看相关 SQL 的执行计划：

确认执行计划是否存在 FULL TABLE SCANS 或者 INDEX FAST FULL SCANS

通过上面确认的 SQL\_ID 来查看执行计划：

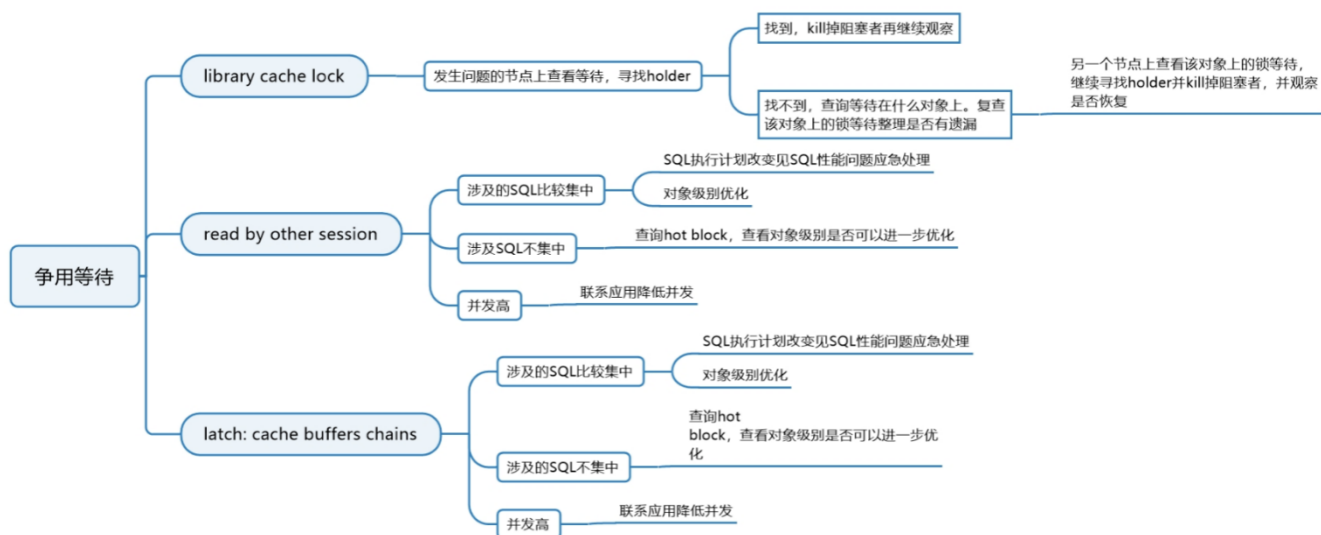
```
SQL> select * from table(dbms_xplan.display_cursor('&sql_id',null,'allstats last'));
```

### 3.具体分析

FULL TABLE SCANS 或者 INDEX FAST FULL SCANS 本身的性能可能不佳，另外可能会导致大量的 IO 消耗，从而引发整个 DB 的性能问题。

分析具体影响以及这些 SQL 语句是否有优化空间。如，是否可以通过建立合适的索引等进行优化。

## 4.6 争用等待



### 4.6.1 library cache lock/pin

#### 【故障现象】

某些 session 执行操作被堵塞，检查 event 发现 'library cache lock/pin' 等待；

#### 【可能故障原因】

library cache lock/pin 发生在多个 session 对相同 library cache 对象进行争用发生，一般来说在存储过程编译过程中发生并堵塞编译。

#### 【应急措施】

在 RAC 环境中，Library cache lock/pin 与 enqueue 不同，他不是全局等待。首先我们在其中一个发生 library cache lock/pin 等待的节点上查看等待事件：

```
set linesize 160
```

```
set pagesize 999
column username format a16
column event format a40
column p1 format 99999999999999999999
column p2 format 9999999999999999
column p3 format 9999999999
column sql_child_number format 9999 heading 'SQL Child|Number'
Select sid,
       seq#,
       username,
       event,
       sql_id,
       sql_child_number,
       plraw,
       p2,
       last_call_et
From v$session
Where event Not In ('SQL*Net message from client',
                   'rdbms ipc message',
                   'rdbms ipc message',
                   'SQL*Net message to client',
                   'pmon timer',
                   'smon timer',
                   'wakeup time manager',
                   'gcs remote message')
And event Not Like 'Streams AQ:%'
Order By event, sql_id, p1;
```

根据 plraw 查找 library cache lock/pin 的阻塞对象:

```
--library cache lock
set linesize 200 pagesize 999
column username format a10
```

```

column program format a30
column machine format a15
column sql_id format a14
column event format a30
column owner format a12
column object_name format a25
select s.sid, s.username, s.program, s.machine, s.sql_id, s.event,
       s.status, l.kgllktype as type, l.kgllkmod as lmode,
       o.kglnaown as owner, o.kglnaobj as object_name
from dba_kgllock l, x$kglob o, v$session s
where l.kgllkhd1 = '&plraw'
      and l.kgllkreq = 0
      and l.kgllkmod > 1
      and l.kgllkhd1 = o.kglhdadr
      and s.SADDR = l.kgllkuse;

--library cache pin
set linesize 160
set linesize 200
column username format a10
column sql_id format a14
column event format a30
col ospid format 999999
column owner format a12
column object_name format a25
Select /*+rule*/
s.sid,
s.username,
s.sql_id,
s.event,
s.status,
p.kglpnmod As lmode,

```



```
p.kglpnreq As req,
o.spid      As ospid,
k.kglnaown As owner,
k.kglnaobj As object_name

From v$session_wait w, x$kglnp p, v$session s, v$process o, x$kglob k
Where w.event Like '%library cache pin%'

And w.plraw = '&plraw'

And w.plraw = p.kglpnhdl

And p.kglpnreq = 0

And p.kglpnmod > 0

And p.kglpnuse = s.saddr

And s.paddr = o.addr

And p.kglpnhdl = k.kglhdadr;
```

等待较多的情况下，通过多次执行上面对应的两个脚本梳理出阻塞关系，最终查询 holder 在做什么，然后 kill 处理。RAC 中大多数情况会查到最后发现没有阻塞对象，这时候需要根据梳理到最后的 plraw 查看保护的物体是什么。

```
set lines 160 pages 999

col kglnaown for a12
col kglnaobj for a25

select kglnaown, kglnaobj from x$kglob where kglhdadr = '&plraw';
```

根据这个对象再在本节点复查一遍锁情况

```
set lines 160 pages 999

col owner for a12
col name for a25

select session_id sid, owner, name, type, mode_held held, mode_requested request
from dba_ddl_locks
where name like '%PIN_TEST%';
```

上面的 name 替换成实际值，如果查到的会话动作都是之前已经梳理过的阻塞关系，那么说明阻塞者可能再另外的节点上。

检查另一个节点

```
set lines 160 pages 999
col owner for a12
col name for a25
select session_id sid, owner, name, type, mode_held held, mode_requested request
  from dba_ddl_locks
 where name like '%PIN_TEST%';
```

上面的 name 替换成实际值，记录下锁情况，再查看这个节点上的等待：

```
set linesize 160
set pagesize 999
column username format a16
column event format a40
column p1 format 99999999999999999999
column p2 format 9999999999999999
column p3 format 9999999999
column sql_child_number format 9999 heading 'SQL Child/Number'
Select sid,
       seq#,
       username,
       event,
       sql_id,
       sql_child_number,
       plrow,
       p2,
       last_call_et
  From v$sqlsession
 Where event Not In ('SQL*Net message from client',
                    'rdbms ipc message',
                    'rdbms ipc message',
                    'SQL*Net message to client',
                    'pmon timer',
                    'smon timer',
                    'wakeup time manager',
```

```

                                ,gcs remote message')
And event Not Like 'Streams AQ:%'
Order By event, sql_id, pl;

```

复杂的环境下多个节点都会有 library cache lock/pin 等待，根据前面查找 pin/lock 阻塞者的方法继续梳理，直到找到其他等待的 holder。

另外一种查法，适合阻塞不多，通过对象查找的 kglhdadr 不多从而很容易确认 session 的情况

```
set lines 160 pages 999

col kglnaown for a12

col kglnaobj for a25

select kglhdadr,kglnaown, kglnaobj from x$kglob where kglnaobj = 'PIN_TEST';

--代入查到的kglhdadr

set linesize 160

set pagesize 999

column username format a16

column event format a40

column p1 format 99999999999999999999

column p2 format 9999999999999999

column p3 format 9999999999

column sql_child_number format 9999 heading 'SQL Child/Number'

Select sid,

      seq#,

      username,

      event,

      sql_id,

      sql_child_number,

      plraw,

      p2,

      last_call_et

From v$session s

Where s.saddr In
```

```
(Select kgllkuse From dba_kgllock Where kgllkhd1 = '00000000A8411CF8');
```

根据实际情况严重程度进行如下紧急处理：

对所有导致library cache lock/pin的session进行kill，解决堵塞情况。

注意在 os 级别 kill 之前,先用 ps 命令查看一下该进程,如果是 DB 进程,不可随意 kill,否则会导致系统 crash

#### 【后续分析】

此类问题主要是由于并发执行对象编译导致的，解决思路就是将编译动作串行执行，减少并发争用。同时，后续需要查询此类操作为什么发起，在业务高峰期应当避免。

### 4.6.2 read by other session

#### 【故障现象】

某些 SQL 语句执行缓慢，检查 event 发现'read by other session'等待；

#### 【可能故障原因】

read by other session 说明多个 session 正在相同的 block 上进行读的争用，说明当前存在读的 hot block，一般同 db file sequential read 或 db file scattered read 同时出现。

#### 【应急措施】

在 read by other session 发生的实例上执行查询：

```
set linesize 180
set pagesize 999
column username format a16
column event format a40
column p1 format 99999999999999999999
column p2 format 9999999999999999
column p3 format 99999999999
column sql_child_number format 9999 heading 'SQL Child|Number'
Select sid,
       username,
       event,
       sql_id,
       sql_child_number,
       p1,
       p2,
```

```

        p3,
        row_wait_obj#,
        last_call_et
    From v$session
    Where event event In ('db file sequential read',
                        'read by other session',
                        'db file scattered read')
    Order By event, sql_id, p1;

```

找到在 hot block 上读争用的 SQL，对 SQL 语句的执行计划进行检查。

根据实际情况严重程度进行如下紧急处理：

如果SQL语句比较分散，对集中的P1和P2的对象（可以根据ROW\_WAIT\_OBJ#去查对象）进行检查，是否存在统计信息过期和索引失效导致热点问题，以及对象级别上是否可以采用分区等方式分散IO。

```

--根据 row_wait_obj#查对象，速度较快，但有时候结果不一定准确
set lines 160 pages 999
col owner for a20
col object_name for a20
col subobject_name for a20
col object_type for a20
col lastddltime for a30

select owner,
       object_name,
       subobject_name,
       object_type,
       status,
       to_char(last_ddl_time, 'yyyy-mm-dd hh24:mi:ss') lastddltime
from dba_objects
where object_id=&obj;

--根据 p1 和 p2 查对象，速度慢，结果会更准确
set lines 160
set pages 999

```

```
col owner format a30
col segment_name format a40
col segment_type format a30
col partition_name format a40

Select owner, segment_name, segment_type, partition_name
  From dba_extents
 Where file_id = &file_id
       And &block_id Between block_id And block_id + blocks - 1;
```

如果SQL语句比较集中，则直接检查SQL，是否存在执行计划错误。处理详见3.5  
如果问题是由于大量相同SQL批量执行导致，联系相关应用及时处理并发过高的异常。  
如果是新上的业务SQL，并且性能本身就很差，联系相关应用及时处理。

#### 【后续分析】

此类问题主要是由于业务并发高导致或 SQL 本身执行计划异常或者 SQL 本身性能问题造成。

### 4.6.3 latch: cache buffers chains

#### 【故障现象】

某些 SQL 语句执行缓慢，检查 event 发现'latch: cache buffers chains'等待；

#### 【可能故障原因】

latch: cache buffers chains 表明当前 SQL 执行过程中产生热块；

#### 【应急措施】

在 latch: cache buffers chains 发生的实例上执行查询：

```
set linesize 180
set pagesize 999
column username format a16
column event format a40
column p1 format 99999999999999999999
column p2 format 9999999999999999
column p3 format 9999999999
column sql_child_number format 9999 heading 'SQL Child|Number'
Select sid,
       seq#,
       username,
```

```

event,
sql_id,
sql_child_number,
p1raw,
p2,
row_wait_obj#,
last_call_et

From v$session
Where event Not In ('latch: cache buffers chains')
Order By event, sql_id, p1;

```

找到读争用的 SQL，对 SQL 语句的执行计划进行检查。

根据实际情况严重程度进行如下紧急处理：

如果SQL语句比较分散，对集中的P1RAW的对象（可以根据ROW\_WAIT\_OBJ#去查对象）进行检查，是否存在统计信息过期和索引失效导致热点问题，以及对象级别上是否可以采用分区等方式分散IO。如存在执行计划错误，对相关SQL执行计划进行purge并重新解析；

—根据 row\_wait\_obj#查对象，速度较快，但有时候结果不一定准确

```

set lines 160 pages 999
col owner for a20
col object_name for a20
col subobject_name for a20
col object_type for a20
col lastddltime for a30

select owner,
       object_name,
       subobject_name,
       object_type,
       status,
       to_char(last_ddl_time, 'yyyy-mm-dd hh24:mi:ss') lastddltime
from dba_objects
where object_id=&obj;

```

--根据 plraw 查对象，速度慢，结果会更准确

```
set lines 160 pages 999
col object_name for a30
Select a.hladdr,
       a.file#,
       a.dbablk,
       a.tch,
       a.obj,
       b.owner || '.' || b.object_name object_name
  From x$bh a, dba_objects b
 Where (a.obj = b.object_id Or a.obj = b.data_object_id)
       And a.hladdr In ('&plraw')
Union
Select hladdr, file#, dbablk, tch, obj, Null
  From x$bh
 Where obj In (Select obj
               From x$bh
               Where hladdr In ('&plraw')
               Minus
               Select object_id
                 From dba_objects
               Minus
               Select data_object_id
                 From dba_objects)
       And hladdr In ('&plraw')
Order By 4 desc;
```

如果 SQL 语句比较集中，则直接检查 SQL，是否存在执行计划错误。处理详见 SQL 性能问题应急处理

如果问题是由于大量相同 SQL 批量执行导致，联系相关应用及时处理并发过高的异常。

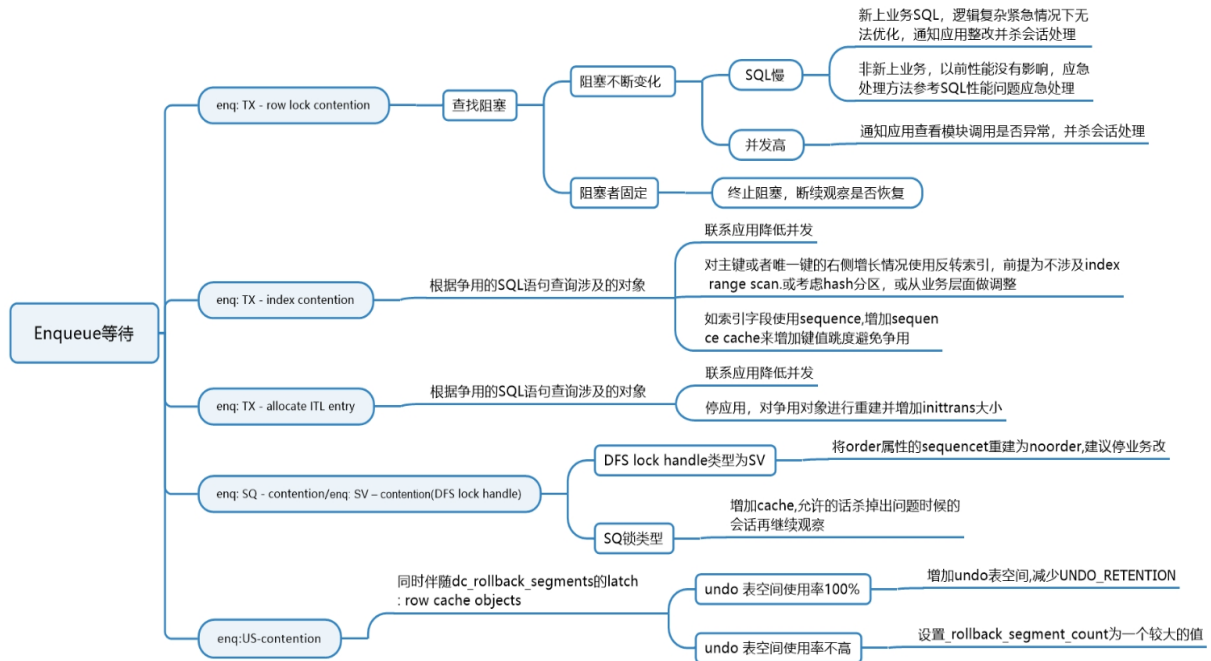
如果是新上的业务 SQL，并且性能本身就很差，联系相关应用及时处理，

### 【后续分析】

此类问题主要是由于业务并发高导致或 SQL 本身执行计划异常或者 SQL 本身性能问题造成。



## 4.7 enqueue 等待



### 4.7.1 enq: TX - row lock contention

#### 【故障现象】

应用反应堵塞, 检查数据库等待事件出现'enq: TX - row lock contention'

#### 【可能故障原因】

- 业务更新或者删除同一行记录
- 对创建位图索引的列值更新
- 对主键或唯一键插入相同记录

#### 【应急措施】

在 enq: TX - row lock contention 发生的实例上执行查询:

```

set linesize 180
set pagesize 999
column username format a16
column event format a40
column sql_child_number format 9999 heading 'SQL Child|Number'
column blocking_instance format 9999 heading 'Blocker|Instance'
column blocking_session format 99999 heading 'Blocker|Session'
column FINAL_BLOCKING_INSTANCE format 9999 heading 'F_Blocker|Instance'
  
```

```
column FINAL_BLOCKING_SESSION format 99999 heading 'F_Blocker|Session'
column "Lock" format a6
column "Mode" format a6
Select sid,
       seq#,
       username,
       event,
       sql_id,
       sql_child_number,
       chr(to_char(bitand(pl, -16777216)) / 16777215) ||
       chr(to_char(bitand(pl, 16711680)) / 65535) "Lock",
       to_char(bitand(pl, 65535)) "Mode",
       blocking_instance,
       blocking_session,
       final_blocking_instance,
       final_blocking_session,
       last_call_et
From v$session
Where event = 'enq: TX - row lock contention';
```

关注 **blocking\_instance/blocking\_session** 和 **final\_blocking\_instance/final\_blocking\_session**(适用于 11g)

如果 **final\_blocking\_instance/final\_blocking\_session** 没有变化, 说明可能是由于某个进程没有提交事务导致的 **enq: TX = row lock contention**

到 holder 所在的实例执行以下查询:

```
set lines 180 pages 999
column username format a15
column program format a35
column machine format a20
column event format a30
column sql_id format a13
column status format a10
Select s.sid,
```

```
s.username,
s.event,
s.program,
s.machine,
s.sql_id,
s.sql_child_number,
s.status,
s.last_call_et

From v$session s
Where s.sid in (&sid);
```

确认不是 DB 进程，对堵塞者生成 kill 语句：

```
set lines 200 pages 999
col username for a15
col machine for a20
col program for a35
col spid for 9999999999
col dbkill for a40
col oskill for a20
col service_name for a20
select a.username,
       a.program,
       b.addr,
       b.spid,
       'alter system kill session ''' || a.sid || ',' || a.serial# || ''';' "dbkill",
       'kill -9 ' || b.spid "oskill"
from v$session a, v$process b
where a.paddr = b.addr(+)
      and a.sid = &sid
      and a.serial#=&serial
order by a.username, a.program;
```

执行生成的 dbkill 命令后继续观察 session 状态，如果长时间没有消失，在 OS 级别执行生成的 oskill 命令(如果是 DB 进程，不可随意 kill，否则会导致系统 crash):

```
ps -ef|grep &spid
```

```
kill -9 &spid
```

如果 blocking\_instance/blocking\_session 和 final\_blocking\_instance/final\_blocking\_session 均在不停变化，说明可能是由于业务 SQL 执行慢或者是并发高于正常值导致。

确认应用模块，执行如下查询：

```
set lines 180 pages 999
column username format a15
column program format a35
column machine format a20
column event format a30
column sql_id format a13
column status format a10
Select s.sid,
       s.username,
       s.event,
       s.program,
       s.machine,
       s.sql_id,
       s.sql_child_number,
       s.status,
       s.last_call_et
  From v$session s
 Where event = 'enq: TX - row lock contention'
 Order By event, sql_id;
```

查看 SQL 执行计划，进一步分析是否是 SQL 性能问题，如下查询生成查看 SQL 执行计划的语句：

```
set lines 160 pages 999
set long 99999
col PLAN_TABLE_OUTPUT for a160
select sql_fulltext,
       parsing_schema_name,
```

```
'select * from table(dbms_xplan.display_cursor('' || sql_id ||
'', ' || child_number || ', 'all iostats last +PEEKED_BINDS'')));' xplan
from v$sql
where sql_id = '&sql_id';
```

包括 SQL 本身性能问题或者并发突发性增长，这类情况下 kill 阻塞者并不能解决问题，通知应用处理，允许的条件下通过临时 kill 所有等待的会话释放：

```
set lines 200 pages 999
col username for a15
col machine for a20
col program for a35
col spid for 999999999999
col dbkill for a40
col oskill for a20
Select a.username,
       a.program,
       a.machine,
       b.addr,
       b.spid,
       'alter system kill session ''' || a.sid || ', ' || a.serial# || ''';' "dbkill",
       'kill -9 ' || b.spid "oskill"
From v$session a, v$process b
Where a.paddr = b.addr(+)
      And a.event = 'enq: TX - row lock contention'
      And a.type = 'USER'
Order By a.username, a.program;
```

### 【后续分析】

如果是固定阻塞者，根据 blocker session 信息，联系相关应用确认该事务未正常提交的原因，避免下次问题发生。

如果是 SQL 性能变差导致执行慢进而产生等待，进一步分析处理 SQL 性能问题。

如果 SQL 执行效率没有问题，那么有可能是并发异常导致的等待，根据应用模块及执行 SQL 的信息，联系相关应用及时处理并发过高的异常。

## 4.7.2 enq: TX - index contention

### 【故障现象】

某些 DML 语句执行缓慢，查询等待事件出现 “enq: TX - index contention” 等待。

### 【可能故障原因】

- 应用并发高，插入时索引块分裂
- Update ... set 更新的键值被包含在索引中，并发高也会引起

### 【应急措施】

在 enq: TX - index contention 发生的实例上在查找是哪些 SQL 语句导致大量的争用：

```
set linesize 180
set pagesize 999
column username format a16
column event format a40
column sql_child_number format 9999 heading 'SQL Child|Number'
column blocking_instance format 9999 heading 'Blocker|Instance'
column blocking_session format 99999 heading 'Blocker|Session'
column FINAL_BLOCKING_INSTANCE format 9999 heading 'F_Blocker|Instance'
column FINAL_BLOCKING_SESSION format 99999 heading 'F_Blocker|Session'
column "Lock" format a6
column "Mode" format a6
Select sid,
       seq#,
       username,
       event,
       sql_id,
       sql_child_number,
       chr(to_char(bitand(p1, -16777216)) / 16777215) ||
       chr(to_char(bitand(p1, 16711680)) / 65535) "Lock",
       to_char(bitand(p1, 65535)) "Mode",
       blocking_instance,
       blocking_session,
       final_blocking_instance,
       final_blocking_session,
```

```
last_call_et

From v$session

Where event = ' enq: TX - index contention' ;
```

查看执行计划，检查这些 SQL 执行中涉及的对象，对象上涉及的索引对象又是哪些，生成查看执行计划的语句如下：

```
set lines 160 pages 999
set long 99999
col PLAN_TABLE_OUTPUT for a160
select sql_fulltext,
       parsing_schema_name,
       'select * from table(dbms_xplan.display_cursor('' ' || sql_id ||
       ''', ' || child_number || ', ''all iostats last +PEEKED_BINDS'')));' xplan
from v$sql
where sql_id = '&sql_id';
```

根据语句 `explain plan for` 生成到 `plan_table` 中，并获取涉及的索引：

```
explain plan for statement

set lines 200 pages 999
col tab format a40
col ind format a50
col column_name format a25
col column_position format 9
break on tab
with t1 as
  (select /*+ materialize */ distinct object_owner from plan_table where object_type =
  'TABLE'),
t2 as
  (select /*+ materialize */ distinct object_name from plan_table where object_type =
  'TABLE')
select table_owner || '.' || table_name tab,
       index_owner || '.' || index_name ind,
       column_name,
       column_position
```

```
from dba_ind_columns
where table_owner in (select * from t1)
    and table_name in (select * from t2)
order by 1, 2, 3, 4;
```

或者采取更简便的方法，收集问题时段的 AWR 报告，检查 Segments by Row Lock Waits 部分指标：

## Segments by Row Lock Waits

- % of Capture shows % of row lock waits for each top segment compared
- with total row lock waits for all segments captured by the Snapshot

| Owner | Tablespace Name | Object Name              | Subobject Name | Obj. Type | Row Lock Waits | % of Capture |
|-------|-----------------|--------------------------|----------------|-----------|----------------|--------------|
| RPT   | RPT_INX         | IDX_RP_BUSI_RECORD_1     |                | INDEX     | 14             | 14.29        |
| RPT   | RPT_INX         | PK_CHK_ORD_INTERFACE_572 |                | INDEX     | 9              | 9.18         |
| RPT   | RPT_INX         | PK_CHK_ORD_INTERFACE_579 |                | INDEX     | 9              | 9.18         |
| RPT   | RPT_INX         | PK_RP_BUSI_RECORD        |                | INDEX     | 9              | 9.18         |
| RPT   | RPT_INX         | IDX_RP_PAY_RECORD_4_570  |                | INDEX     | 6              | 6.12         |

手工产生一个 AWR 快照的方法如下：

```
exec DBMS_WORKLOAD_REPOSITORY.create_snapshot();
```

根据实际情况严重程度进行如下紧急处理：

- 降低应用并发度，降低 index 争用；
- 使用反向索引重建，或使用 hash 分区重建索引；
- 如索引字段使用 sequence，可考虑增加 sequence cache 来增加键值跳度避免争用。

### 【后续分析】

该等待一般由于业务繁忙导致，通过降低并发或调整索引可以避免，反向索引由于对范围查询有一定影响，需要进行评估，hash 索引分区也可以一定程度避免争用。

## 4.7.3 enq: TX - allocate ITL entry

### 【故障现象】

某些批量 DML 语句执行缓慢，查询等待事件发现'enq: TX - allocate ITL entry'；

### 【可能故障原因】

‘enq: TX - allocate ITL entry’为分配 ITL 事务槽的等待，可能是因为 PCTFREE 不足，BLOCK 中没有足够空间分配 ITL，ORACLE 只能重用 ITL，但是这个时候由于没有 COMMIT，无法重用 ITL，所以会出现 allocate ITL 等待事件。

### 【应急措施】

在 enq: TX - allocate ITL entry 发生的实例上在查找是哪些 SQL 语句导致大量的争用：

```
set linesize 180
set pagesize 999
```



```
column username format a16
column event format a40
column sql_child_number format 9999 heading 'SQL Child|Number'
column blocking_instance format 9999 heading 'Blocker|Instance'
column blocking_session format 99999 heading 'Blocker|Session'
column FINAL_BLOCKING_INSTANCE format 9999 heading 'F_Blocker|Instance'
column FINAL_BLOCKING_SESSION format 99999 heading 'F_Blocker|Session'
column "Lock" format a6
column "Mode" format a6
Select sid,
       seq#,
       username,
       event,
       sql_id,
       sql_child_number,
       chr(to_char(bitand(p1, -16777216)) / 16777215) ||
       chr(to_char(bitand(p1, 16711680)) / 65535) "Lock",
       to_char(bitand(p1, 65535)) "Mode",
       blocking_instance,
       blocking_session,
       final_blocking_instance,
       final_blocking_session,
       last_call_et
From v$session
Where event = 'enq: TX - allocate ITL entry';
```

查看执行计划, 检查这些 SQL 执行中涉及的对象, 以及对象上涉及的索引对象又是哪些, 生成查看执行计划的语句如下:

```
set lines 160 pages 999
set long 99999
col PLAN_TABLE_OUTPUT for a160
select sql_fulltext,
       parsing_schema_name,
```

```
'select * from table(dbms_xplan.display_cursor('' || sql_id ||
'', ' || child_number || ', 'all iostats last +PEEKED_BINDS'')));' xplan
from v$sql
where sql_id = '&sql_id';
```

根据语句 **explain plan for** 生成到 **plan\_table** 中，并获取涉及的对象和索引：

```
explain plan for statement
--执行计划涉及的表和索引
set lines 200 pages 999
col owner format a30
col segment_name format a30
col segment_type format a30
col degree for a5
with t1 as
(select /*+ materialize */ object_owner,object_type from plan_table),
t2 as
(select /*+ materialize */ object_name,object_type from plan_table)
select a.owner, a.segment_name, a.segment_type, sum(a.bytes / 1024 / 1024) "Size(Mb)",
trim(b.degree) degree, b.num_rows
  from dba_segments a
 left join dba_tables b on a.owner = b.owner and a.segment_name = b.table_name
where a.owner in (select object_owner from t1 where object_type = 'TABLE')
  and a.segment_name in (select object_name from t2 where object_type = 'TABLE')
group by a.owner,a.segment_type, a.segment_name, trim(b.degree), b.num_rows
union all
select a.owner, a.segment_name, a.segment_type, sum(a.bytes / 1024 / 1024) "Size(Mb)",
trim(b.degree) degree, b.num_rows
  from dba_segments a
 left join dba_indexes b on a.owner = b.owner and a.segment_name = b.index_name
where a.owner in (select object_owner from t1 where object_type like 'INDEX%')
  and a.segment_name in (select object_name from t2 where object_type like 'INDEX%')
group by a.owner,a.segment_type, a.segment_name, trim(b.degree), b.num_rows
union all
----table in the index
```

```
select a.owner, '*' || a.segment_name , a.segment_type, sum(a.bytes / 1024 / 1024)
"Size(Mb)", trim(b.degree) degree, b.num_rows

from dba_segments a

left join dba_tables b on a.owner = b.owner and a.segment_name = b.table_name

where a.owner in (select table_owner

from dba_indexes

where owner in (select object_owner from t1)

and index_name in (select object_name from t2))

and a.segment_name in (select table_name

from dba_indexes

where owner in (select object_owner from t1)

and index_name in (select object_name from t2))

group by a.owner, a.segment_type, a.segment_name, trim(b.degree), b.num_rows

order by 3,4;
```

--涉及的表上的所有索引

```
col tab format a40
col ind format a50
col column_name format a25
col column_position format 9
break on tab

with t1 as

(select /*+ materialize */ distinct object_owner from plan_table where object_type =
'TABLE'),

t2 as

(select /*+ materialize */ distinct object_name from plan_table where object_type =
'TABLE')

select table_owner || '.' || table_name tab,

index_owner || '.' || index_name ind,

column_name,

column_position

from dba_ind_columns

where table_owner in (select * from t1)

and table_name in (select * from t2)
```

```
order by 1, 2, 3, 4;
```

或者采取更简便的方法，收集问题时段的 AWR 报告，检查 Segments by ITL Waits 部分指标：

## Segments by ITL Waits

- % of Capture shows % of ITL waits for each top segment compared
- with total ITL waits for all segments captured by the Snapshot

| Owner | Tablespace Name | Object Name          | Subobject Name | Obj. Type | ITL Waits | % of Capture |
|-------|-----------------|----------------------|----------------|-----------|-----------|--------------|
| RPT   | RPT_INX         | IDX_RP_BUSI_RECORD_1 |                | INDEX     | 2         | 50.00        |
| RPT   | RPT_INX         | PK_RP_BUSI_RECORD    |                | INDEX     | 2         | 50.00        |

手工产生一个 AWR 快照的方法如下：

```
exec DBMS_WORKLOAD_REPOSITORY.create_snapshot();
```

根据实际情况严重程度进行如下紧急处理：

- 降低应用并发度，避免 ITL 争用；
- 停止应用，对争用对象进行重建并增加 initrans 大小。

### 【后续分析】

解决此类问题，可以考虑增加 PCTFREE 和 initrans 大小，需要注意该修改只能对于新 block 生效，已经存放数据的 block 不会发生改变，因此建议对整个对象进行重建；另外可以考虑修改业务逻辑，减少频繁访问。

## 4.7.4 enq: SQ - contention/enq: SV – contention(DFS lock handle)

### 【故障现象】

- 现象一：应用反应堵塞，检查数据库等待事件出现' DFS lock handle'，并且检查出是 SV 锁。（11g 开始直接可以观察到 enq: SV – contention）
- 现象二：应用反应堵塞，检查数据库等待事件出现'enq: SQ - contention'。参考文档：853652.1

### 【可能故障原因】

- 需要根据 p1 判断类型来确认，这里处理的例子是 SV 锁
- SV 锁是 RAC 上赋予了 cache+order 的 Sequence 上发生
- SQ 锁是赋予了 cache+noorder 属性的 Sequence 上发生

### 【应急措施】

在 DFS lock handle 发生的实例上执行查询：

```

set linesize 160
set pagesize 999
column username format a16
column event format a40
column p1 format 99999999999999999999
column p2 format 9999999999999999
column p3 format 9999999999
column sql_child_number format 9999 heading 'SQL Child|Number'
Select sid,
        username,
        event,
        sql_id,
        sql_child_number,
        p1,
        p2,
        p3 ,
        last_call_et
  From v$session
 where event = 'DFS lock handle'
 order by event, sql_id, p1;

```

11g 以上直接查询对应的 enq: SV – contention 即可

根据 p1 查类型

```

select chr(bitand(&p1,-16777216)/16777215) ||
chr(bitand(&p1,16711680)/65535) type,
mod(&p1, 16) md
from dual;

```

下面的问题以查出来的是 SV 锁为例

```

set lines 160 pages 999
col owner for a20
col object_name for a20

```

```
col subobject_name for a20
col object_type for a20
col lastddltime for a30

select owner,
       object_name,
       subobject_name,
       object_type,
       status,
       to_char(last_ddl_time, 'yyyy-mm-dd hh24:mi:ss') lastddltime
from dba_objects
where object_id=&p2;

--代入上一步查到的 sequence_name
select * from dba_sequences where sequence_name = '&sequence_name';
```

enq: SQ – contention 同样可以根据 p2 找到 sequence\_name，并查看 sequence 属性。过程没有太大差别。

根据实际情况严重程度进行如下紧急处理：

- 增加 cache
- 允许的条件下临时把 DFS lock handle 或者 enq: SQ – contention 等待的会话杀掉，观察是否还有等待。

```
set lines 200 pages 999
col username for a15
col machine for a20
col program for a35
col spid for 99999999999
col dbkill for a40
col oskill for a20
Select a.username,
       a.program,
       a.machine,
       b.addr,
```

```

        b.spid,
        'alter system kill session ''' || a.sid || ',' || a.serial# || ''';' "dbkill",
        'kill -9 ' || b.spid "oskill"

From v$session a, v$process b
Where a.paddr = b.addr(+)
      And a.event = 'DFS lock handle'
      And a.type = 'USER'
Order By a.username, a.program;
```

### 【后续分析】

对于 SV 锁，在 RAC 环境尽量避免 order，sequence 的作用是保证全局唯一，如果需要同时实现全局有序，应该在应用侧实现。联系相关应用将 order 属性的 sequence 重建为 noorder，建议停业务改。

## 4. 7. 5 enq: US – contention

### 【故障现象】

应用处理速度下降，查询等待事件发现大量的 'enq: US – contention'，同时伴随 dc\_rollback\_segments 的 latch: row cache objects 等待事件；

### 【可能故障原因】

- 业务并发量大，online undo segment 的个数不足；
- 业务并发量大，undo 表空间不够大；

### 【应急措施】

在 enq: US – contention 发生的实例上执行查询：

```

column UNXPSTEALCNT heading "# Unexpired|Stolen"
column EXPSTEALCNT heading "# Expired|Reused"
column SSOLDERRCNT heading "ORA-1555|Error"
column NOSPACERRCNT heading "Out-Of-space|Error"
column MAXQUERYLEN heading "Max Query|Length"

select inst_id, to_char(begin_time,'MM/DD/YYYY HH24:MI') begin_time,
UNXPSTEALCNT, EXPSTEALCNT , SSOLDERRCNT, NOSPACERRCNT, MAXQUERYLEN, TUNED_UNDORETENTION
from v$undostat
order by inst_id, begin_time;
```

--关注 online segment 的数量

```
select tablespace_name , status , count(*) number_of_rollback_segments from
dba_rollback_segs group by tablespace_name , status;
```

--检查 undo tablespace 的使用情况

```
SELECT /* + RULE */
      df.tablespace_name "Tablespace",
      df.bytes / (1024 * 1024) "Size (MB)",
      SUM (fs.bytes) / (1024 * 1024) "Free (MB)",
      NVL (ROUND (SUM (fs.bytes) * 100 / df.bytes), 1) "% Free",
      ROUND ( (df.bytes - SUM (fs.bytes)) * 100 / df.bytes) "% Used"
FROM dba_free_space fs,
      ( SELECT tablespace_name, SUM (bytes) bytes
        FROM dba_data_files
        GROUP BY tablespace_name) df
WHERE fs.tablespace_name(+) = df.tablespace_name
GROUP BY df.tablespace_name, df.bytes
UNION ALL
SELECT /* + RULE */
      df.tablespace_name tspace,
      fs.bytes / (1024 * 1024),
      SUM (df.bytes_free) / (1024 * 1024),
      NVL (ROUND ( (SUM (fs.bytes) - df.bytes_used) * 100 / fs.bytes), 1),
      ROUND ( (SUM (fs.bytes) - df.bytes_free) * 100 / fs.bytes)
FROM dba_temp_files fs,
      ( SELECT tablespace_name, bytes_free, bytes_used
        FROM v$temp_space_header
        GROUP BY tablespace_name, bytes_free, bytes_used) df
WHERE fs.tablespace_name(+) = df.tablespace_name
GROUP BY df.tablespace_name,
      fs.bytes,
      df.bytes_free,
```



```
df.bytes_used
ORDER BY 4 DESC;
```

1) 如果TUNED\_UNDORETENTION的值异常大, online 状态的undo segment 不是很多, 且会慢慢增加, 同时undo 表空间使用率不高, 根据实际情况严重程度进行如下紧急处理:

a) 设置 UNDO TABLESPACE 里的所有 DATAFILE 为 AUTOEXTEND ON, 同时 MAX SIZE 设置为当前 UNDO TABLESPACE 大小

语法如下:

```
ALTER DATABASE DATAFILE '<datafile_filename>' AUTOEXTEND ON MAXSIZE <current_size>;
```

b) 根据实例情况设置 \_rollback\_segment\_count 为比较大的值, 语法如下:

```
alter system set "_rollback_segment_count"=2000;
```

2) 如果TUNED\_UNDORETENTION的值异常大, online 状态的undo segment基本保持不变, 且undo 表空间使用率达到100%, 根据实际情况严重程度进行如下紧急处理:

增加 undo 表空间的大小:

```
alter tablespace UNDOTBS1XXX ADD DATAFILE 'XXXXX' size XXXXM;
```

### 【后续分析】

此类问题主要是业务并发量大, undo 表空间不足或 online segment 不足引起的, 解决思路是增加 undo 表空间, 或增加 online segment 的个数以及降低 TUNED\_UNDORETENTION。

## 5 故障处理

### 5.1 常见故障处理及信息收集

#### 5.1.1 ORA-04031

04031 实际上是一个进程申请内存时, shared pool 因为内存不足而找不到满足大小并且连续的内存空间而产生的错误信息, 严重时可能导致整个实例 hang。可能的故障原因多为物理内存耗尽, shared pool 配置不合理, 内存碎片化严重, 应用未使用绑定变量等。如果条件允许, 建议先收集信息, 然后进行应急处理, 尝试是否能解决问题。

## 【应急处理】

应急处理建议先收集必要的信息（必须在问题期间收集的信息），再进行 flush shared pool 或者重启 DB 的操作。

### 1. 建议收集 heapdump 和 library cache dump:

收集 heapdump 和 library cache dump.（对诊断最直接，但会消耗一定的资源，且**必须再问题发生期间收集**）

\*\*\*\*\*

#### 1) 收取 Heapdump 的信息（**必须在问题发生期间收集**）：命令如下：

```
sqlplus /nolog
connect / as sysdba
oradebug setmypid
oradebug unlimit
oradebug dump heapdump 536871426
oradebug tracefile_name
oradebug close_trace
```

影响分析：从经验来看，一般 Heapdump 执行不会对系统造成太多的影响，影响基本可以忽略。如果产生影响，也是系统产生 4031 后影响结果，非本身该操作影响。

#### 2) 收取 library cache dump（必须在问题发生期间收集）。命令如下：

```
ALTER SYSTEM SET EVENTS 'IMMEDIATE TRACE NAME LIBRARY_CACHE LEVEL 16';
```

影响分析：从经验来看，library cache dump 可能会对系统有些少许的影响(依据系统负载不同而影响不同)，如果故障期间在非业务高峰期，建议可以使用。

备注：执行 Heapdump 以及 library cache dump 有必要可以联系 oracle 支持进行指导。

\*\*\*\*\*

### 2. 以下应急方案在可行的情况下可以逐一尝试：

1. 调大问题池的 size，例如调大 shared pool 的大小，可暂时缓解 ORA-04031，争取分析的时间，然后在空闲时段进行数据库实例重启。

```
alter system set shared_pool_size=10G ;
```

**需要判断是否可以调大 shared\_pool\_size**，一般可通过如下方式

```
SQL> select NAME, VALUE/1024/1024 from v$parameter where name
in('shared_pool_size','db_cache_size','sga_target','sga_max_size')
2 union
3 select component, current_size/1024/1024 from v$sga_dynamic_components where
```

```
component in ('shared pool','DEFAULT buffer cache','large pool');
```

| NAME                 | VALUE/1024/1024 |
|----------------------|-----------------|
| -----                | -----           |
| DEFAULT buffer cache | 400             |
| db_cache_size        | 304             |
| large pool           | 16              |
| sga_max_size         | 3072            |
| sga_target           | 2048            |
| shared pool          | 1504            |
| shared_pool_size     | 1008            |

7 rows selected.

SQL>

如上示例中，红色部分为 SGA 中占用内存最大的两块 shared\_pool\_size 和 db\_cache\_size 参数设定值。

绿色部分为 shared pool 和 buffer cache 的当前实际分配内存大小。

调整 shared\_pool\_size 大于当前 shared pool 值

满足如下条件才可调整：

如果 sga\_max\_size 比 sga\_target 大很多，那么我们可以通过调大 sga\_target 然后再调大 shared\_pool\_size

如：

```
SQL> alter system set sga_target=3000M;
```

```
SQL> alter system set shared_pool_size=2000M;
```

如果 sga\_target 比 (shared\_pool\_size + db\_cache\_size) 大很多，那么我们也可以尝试调大 shared\_pool\_size。但这种情况应该很少，因为一般也不建议这样配置

如：

```
SQL> alter system set shared_pool_size=1600M;
```

2. 不繁忙的系统，可以尝试 flush shared pool （发生问题的是 shared pool 的前提下，但在很多情况下可能不会奏效，会很快又出现 ORA-04031 的问题）

```
alter system flush shared_pool ;
```

3. 重启故障数据库实例

### 【后续分析】

补充收集 Alert log/trace file、AWR/ASH、OSW。进一步分析；必要时需要寻求 oracle support 的支持。

## 5.1.2 ORA-600

### 【故障现象】

Alert 日志中出现 ora-600 类型的错误。

### 【可能故障原因】

- oracle 内部错误

### 【信息收集】

1) 收集 alert log 及相关 trace file

参考如下脚本：可以一次性搜集所需要的信息。（根据时间要求，需要适当修改）

```
grep -E '2022-07-09 21|2022-07-09 22' *trc| awk -F: '{print $1}' | uniq |xargs tar cvf dump.tar
```

2) 或者使用 ADRCI 方法打包收集，参考 7.9 章节

## 5.1.3 ORA-7445

### 【故障现象】

Alert 日志中出现 ora-7445 类型的错误。

### 【可能故障原因】

- oracle 内部错误

### 【信息收集】

1) 收集 alert file 及相关 trace file

参考如下脚本：可以一次性搜集所需要的信息。（根据时间要求，需要适当修改）

```
grep -E '2022-07-09 21|2022-07-09 22' *trc| awk -F: '{print $1}' | uniq |xargs tar cvf dump.tar
```

2) 或者使用 ADRCI 方法打包收集：

如下为 1 个示例：

```
oracle@slesn1:~> adrci
adrci> show home
```

```

ADR Homes:
diag/rdbms/rac19c/rac19c1
diag/rdbms/db19c/rac19c1
diag/rdbms/db19c/db19c1
adrci> set homepath diag/rdbms/rac19c/rac19c1      <<<<<选择正确的 oracle home
adrci> show problem

ADR Home = /home/oracle/oracle/diag/rdbms/rac19c/rac19c1:
*****

PROBLEM_ID                                PROBLEM_KEY
LAST_INCIDENT        LASTINC_TIME
-----
1                                ORA  7445  [kclollnumiter]
264241                2022-05-06 16:31:12.329000 +08:00
1 row fetched

adrci> show incident -p "problem_key='ORA 7445 [kclollnumiter]'"

ADR Home = /home/oracle/oracle/diag/rdbms/rac19c/rac19c1:
*****

INCIDENT_ID                                PROBLEM_KEY
CREATE_TIME
-----
216241                ORA 7445 [kclollnumiter]                2022-
05-06 16:19:32.743000 +08:00
264241                ORA 7445 [kclollnumiter]                2022-
05-06 16:31:12.329000 +08:00
2 rows fetched

adrci> ips pack incident 264241 in /tmp      <<<<<<<<<< 收集 incident id=264241 期间产生的
日志

```

```
Generated package 1 in file /tmp/ORA7445kc_20220714120903_COM_1.zip, mode complete

adrci> ips pack problem 1 in /tmp <<<<<<收集 problem id=1 的错误即 ORA 7445
[kc1ollnumiter]期间的日志

Generated package 2 in file /tmp/ORA7445kc_20220714121312_COM_1.zip, mode complete
adrci>

adrci> ips pack problemkey 'ORA 7445 [kc1ollnumiter]' in /tmp <<<<<<收集 ORA 7445
[kc1ollnumiter]期间的日志

Generated package 3 in file /tmp/ORA7445kc_20220714121343_COM_1.zip, mode complete
adrci>

adrci> ips pack time '2022-07-14 12:00:00' to '2022-07-14 13:00:00' in /tmp <<<<<<
收集某时间段的日志

Generated package 4 in file /tmp/IPSPKG_20220714134007_COM_1.zip, mode complete
adrci>
```

#### 5.1.4 性能问题

##### 【故障现象】

业务反馈某些功能点执行效率低，或者登录数据库发现数据库有大量等待事件出现。

##### 【可能故障原因】

- 物理资源不足（内存不足、CPU 利用率高等）
- 数据库发生大规模阻塞
- 异常等待事件过多

##### 【应急处理】

1) 查看系统当前存在哪些较为严重的等待事件

使用 DBA 用户登录查询：

—查看当前是否有较为严重的等待事件

```
set linesize 200 pagesize 1000
col event for a80
select inst_id, event, count(*)
  from gv$session
 where event not like '%SQL%'
       and event not like '%Stream%'
       and event not like '%rdbms%'
       and event not like '%timer%'
```

```
and event not like '%idle%'
group by inst_id, event
order by 1, 3;
```

2) 根据具体的等待事件，参考对应章节的分析方法进一步分析

### 【信息收集】

ASH 数据(Active Session History ) Reports. 参见 8.2 节

AWR Reports. 参见 8.1 节.

收集 OS 性能数据.参见 8.7 节.

搜集 Hanganalyze and System State 数据 参见 8.9 节和 8.10 节

单个语句执行计划慢 需要 SQLHC 信息. 见 7.2.1 章节.

## 5.1.5 实例进程数满 ORA-00020 错误

### 【故障现象】

当 session 连接数量大于数据库设定值时，会报错 ora-00020 MAXIMUM NUMBER OF PROCESSES (%S) EXCEEDED，新的数据库连接将无法完成。

### 【可能故障原因】

数据库 session 过多，可能是 session 参数设置不合理，也可能是由于数据库进程堵塞导致的应用响应超时并不断发起新的连接。

### 【应急措施】

1. 由于连接数已满，因此 DBA 一般无法登陆数据库，首先需要查找 LOCAL=NO 的数据库连接并进行 kill -9,然后尝试抢占空闲 session 登陆数据库。

```
ps -ef | grep -v grep |grep "LOCAL=NO"
```

2. 查询当前数据库实例状态，是否存在异常等待事件，这些等待是可能导致应用 SQL 超时并引起应用不断发起新连接的原因：

```
set linesize 160
set pagesize 999
column username format a16
column event format a40
column p1 format 99999999999999999999
column p2 format 9999999999999999
column p3 format 9999999999
column sql_child_number format 9999 heading 'SQL Child|Number'
```

```
Select sid,
       seq#,
       username,
       event,
       sql_id,
       sql_child_number,
       plrow,
       p2,
       last_call_et
From v$session
Where event Not In ('SQL*Net message from client',
                   'rdbms ipc message',
                   'rdbms ipc message',
                   'SQL*Net message to client',
                   'pmon timer',
                   'smmon timer',
                   'wakeup time manager',
                   'gcs remote message')
And event Not Like 'Streams AQ:%' And WAIT_CLASS<>'Idle'
Order By event, sql_id, pl;
```

如果发现存在异常的争用或堵塞存在，尝试根据实际情况解决问题；

3. 如果当前数据库并没有异常等待，查询当前 session 信息，确认 INACTIVE session 数量并按照 program 排序：

```
set lines 180 pages 999
col username for a15
col machine for a20
col program for a35
col module for a40
Select Count(*), sql_id, status, username, machine, program, module
From v$session
Where Type = 'USER'
```



```
Group By sql_id, status, username, machine, program, module
Order By 1 Desc;
```

根据查询结果，对占用大量 session（尤其是 INACTIVE）的 program 进行检查，要求应用方面对程序进行检查，同时对大量 INACTIVE session 进行 kill 终止。

```
set pagesize 5000
Select 'alter system kill session '||''''||sid||','||serial#||''''||';'
  From v$session
Where Type = 'USER' and status='INACTIVE';
```

### 【后续分析】

检查 listener.log 日志看是否有连接风暴，如果有，需要调查产生的原因；  
检查 ASH 视图，检查当时是否有 lock 或 deadlock；

#### 5.1.6 长事务回滚问题

### 【故障现象】

对一张大表进行长事务 DML 操作时进程异常中断后需读取 UNDO 进行回滚，伴随有 latch:undo 或 db file sequential read 相关等待系统资源消耗上升，该表相关业务无法正常执行。

### 【可能故障原因】

- 长事务操作写入 UNDO,当需要回滚时数据量大且对操作资源消耗非常高。

### 【应急措施】

查回滚事务：

```
set lines 160 pages 999
alter session set nls_date_format='YYYY-MM-DD HH24:MI:SS';
Select a.usn,
       a.slt,
       a.seq,
       a.state,
       a.undoblocktotal "Total",
       a.undoblocksdone "Done",
       a.undoblocktotal - a.undoblocksdone "ToDo",
       decode(cputime,
```

```
0,
'unknown',
Sysdate + (((a.undoblockstotal - a.undoblocksdone) /
(a.undoblocksdone / a.cputime)) / 86400)) "Estimated time to complete"
From v$fast_start_transactions a;
```

上面的查询可以查出是否启用了并行回滚，及并行的进程数，当然查并行数也可以使用如下视图来查看

```
Select pid,xid from V$FAST_START_SERVERS;
```

有时候可能查不到，使用下面的脚本，主要关注 ktuxesiz 有没有在变化

```
Select addr, ktuxeusn, ktuxeslt, ktuxesqn, ktuxesiz, ktuxesta
From x$ktuxe
Where ktuxecfl Like '%DEAD%'
And ktuxesta != 'INACTIVE';
```

a) 如果应用许可，建立同结构空表，然后换名后再回插数据;(如果访问频繁无法 rename 时 kill 访问的会话再执行 rename，允许的话，停监听临时操作)

b) 如果不能使用空表，则需等待回滚完成，可设置参数开启并发回滚加快回滚速度;

i. 启用并行:

CPU 资源较空闲时可

```
alter system set fast_start_parallel_rollback=high;
```

注: low 是 2 倍 cpu 数, high 是 4 倍 CPU 数, 当并发子进程之间需要的资源冲突时(Wait for a undo 相关等待), 并发的回滚速度不如串行的回滚速度, 可将 fast\_start\_parallel\_rollback 改成 false, 以串行的方式进行回滚操作。这个参数可在线修改, 因此改完后再执行查回滚事务的脚本, 看速度有没有增加, 以选择最适合的模式。具体修改方法可参考文档 **How to Disable Parallel Transaction Recovery When Parallel Txn Recovery is Active (Doc ID 238507.1)**

需要注意的是, 除非 cancel/kill 的 session 是 parallel DML, 否则不会采用并行回滚。参考文档解释:

```
FAST_START_PARALLEL_ROLLBACK specifies the degree of parallelism used when recovering
terminated transactions. Terminated transactions are transactions that are active before
a system failure. If a system fails when there are uncommitted parallel DML or DDL
transactions, then you can speed up transaction recovery during startup by using this
parameter.
```

ii. 加快速度:

```
alter system set "_cleanup_rollback_entries"=400 scope=spfile;
```

该参数含义:

The number of undo records processed at one time when rolling back a transaction.  
Prevents long transactions from freezing out shorter transactions that also need to be rolled back.

This means that PMON cleans up 20 blocks every 3 minutes.

The number 20 is the default value of the parameter CLEANUP\_ROLLBACK\_ENTRIES.

So PMON is making progress, but it is slow because every 3 minutes PMON wakes up, does 20 blocks and sleeps again.

HOW TO SPEED UP THE CLEANUP OPERATION

-----  
To speed up cleanup of killed sessions you can increase the value of the CLEANUP\_ROLLBACK\_ENTRIES init.ora parameter. Its value is 20 by default.

20 是早期版本的默认值，修改该参数需重启数据库。

iii. 暂时禁用 smon 的恢复:

```
oradebug setorapid 'SMON's Oracle PID';
oradebug event 10513 trace name context forever, level 2
```

事后恢复使用:

```
oradebug setorapid 'SMON's Oracle PID';
oradebug event 10513 trace name context off
```

## 【后续分析】

OLTP 环境避免运行长事务，改为分批提交;

### 5.1.7 监听器 hang

## 【故障现象】

数据库监听器 hang 住，lsnrctl 命令无法响应，应用无法通过监听器创建新连接。

## 【可能故障原因】

- OS 层异常;
- 系统压力大;
- 数据库 BUG;

## 【应急措施】

先收集一小段时间的监听器进程 **truss** 信息，然后对监听器进行重启：

Sun Solaris truss, e.g.:

```
$ truss -Dlaefo <output file> -p <executable>
```

HP/UX tusc, eg :

```
$ tusc -afpo <output file> -T "%H:%M:%S" -p <executable>
```

IBM AIX truss, e.g.:

```
$ truss -Dlaefo <output file> -p <executable>
```

Linux strace, eg :

```
$ strace -fo <output file> <executable>
```

Kill 当前监听器进程，并重新启动

```
ps -ef|grep tns
```

```
kill -9 PID
```

```
lsnrctl start <listener_name>
```

```
SQL>alter system register;
```

## 【后续分析】

收集 OSW，监听日志，truss 信息，提交 MOS 后台进行分析。

### 5.1.8 数据文件有坏块

故障查询：

```
//查询
```

```
SQL>select * from v$database_block_corruption;
```

| File# | BLOCK#  | BLOCKS | CORRUPTION_CHANGE# | corruption |
|-------|---------|--------|--------------------|------------|
| 121   | 189808  | 1      | 0                  | CORRUPT    |
| 123   | 223957  | 1      | 0                  | CORRUPT    |
| 125   | 4068425 | 1      | 0                  | CORRUPT    |

```
127          3948630          1          0          CORRUPT
```

但是这个视图是不准确的，还需使用 oracle 用户的 dbv 命令查询这其他数据文件是否有坏块

```
//使用 oracle 用户登录
```

```
$ dbv file=数据文件全路径名称
```

### 故障处理：

- 1, 如果为索引表空间，查询那些坏块属于哪些索引，然后删除索引重建即可
- 2, 如果为数据表空间，查询那些坏块属于哪些对象，然后尝试从备份中 exp 导入对象

依次分别查询上面 dbv 查出的这些坏块属于哪些对象：

```
SQL>select      owner,segment_name,partition_name,segment_type,tablespace_name      from
dba_extents where file_id=&id and &block_id between block_id and block_id+blocks-1;
输入 file_id 和块号码
```

经查询，本次的这些坏块数据都没有对象占用，即都为空，因此使用 MOS 中的《RMAN Reports an ORA-19566: Exceeded Limit Of 0 Corrupt Blocks From Backup And No Segment Associated (文档 ID 457422.1)》解决方法 1：进行数据文件的缩小。

（1）但是本次数据文件缩小前，我们首先保证有一份完整的数据库备份才进行的数据文件的缩小，即上篇 mos 中的方法 2。

（2）然后进行数据文件的缩小

至于可缩多少空间，可以参照这边 MOS 文档《How to SHRINK or Reduce the datafile size by finding the HWM (文档 ID 1600774.1)》中的 SQL 语句查出：

```
SQL>
set verify off
column file_name format a50 word_wrapped
column smallest format 999,990 heading "Smallest|Size|Poss."
column currsz format 999,990 heading "Current|Size"
column savings format 999,990 heading "Poss. |Savings"
break on report
compute sum of savings on report
column value new_val blksize
select value from v$parameter where name = 'db_block_size';
/
```

```
select file_name,
       ceil( (nvl(hwm,1)*&blksize)/1024/1024 ) smallest,
       ceil( blocks*&blksize/1024/1024) currsz,
       ceil( blocks*&blksize/1024/1024) -
       ceil( (nvl(hwm,1)*&blksize)/1024/1024 ) savings
from dba_data_files a,
     ( select file_id, max(block_id+blocks-1) hwm
       from dba_extents
       group by file_id ) b
where a.file_id = b.file_id(+) and a.file_id=&file_id order by savings desc
/
```

|                                   | Smallest |         |         |
|-----------------------------------|----------|---------|---------|
|                                   | Size     | Current | Poss.   |
| FILE_NAME                         | Poss.    | Size    | Savings |
| -----                             | -----    | -----   | -----   |
| /oradata/lsxd/TB_IDX001_01_35.dbf | 258      | 30,720  | 30,462  |
|                                   |          |         | -----   |
| sum                               |          |         | 30,462  |

上述输出意味着，该数据文件总大小为 30720M，可以缩小 30462M，因此可缩小到 1G。

### 5.1.9 服务器 CPU 使用率高

(1) 根据操作系统命令定位消耗 CPU 严重的系统进程号

```
top
```

(2) 根据系统进程号得到正在执行的 sql 语句的 sql id 号

```
select sid,event,sql_id from v$session where paddr in (select addr from v$process where
spid = &1);
```

说明：&1 代表系统进程号即系统的 pid

或者，某几个固定的 Oracle 进程 CPU 使用率一直很高

```
SQL>set linesize 1000
```

```
select sid,event,sql_id from v$session where paddr in (select addr from v$process where
```

```
spid in (将进程号输入其中，以逗号隔开));
```

- (3) 根据 sql id 找到对应的 sql 语句  
查询 sql id 对应的 sql 语句

```
select sql_text from v$sqltext where sql_id='&l';
```

- (4) 进一步分析 sql 语句可能存在的性能问题  
查询 sql id 对应执行计划

```
select * from table(dbms_xplan.display_cursor('&sql_id',null,'all last'));
```

### 5.1.10 服务器内存使用率高

具体现象：

由于 ORACLE 数据库故障或应用程序效率问题，占用大量数据库服务器内存资源，导致 ORACLE 数据库性能下降，对外服务响应缓慢，严重时导致系统几乎不可用。

处理方案：

1. 以oracle操作系统用户登录数据库系统主机，查看系统内存和换页空间情况：

- 1) AIX系统

```
vmstat -w 1 10  
lsps -a
```

- 2) Linux系统

```
vmstat -w 1 10  
free -m
```

2. 检查占用内存和Paging高的进程

- 1) AIX系统：使用oracle/grid用户执行svmon命令

```
查看实际内存使用最高的 10 个进程  
svmon -uP -t10 | awk '/Pid/,/^$/ {print $0}'  
查看 pagespace 使用最高的 10 个进程  
svmon -gP -t10 | awk '/Pid/,/^$/ {print $0}'
```

2) 在Linux系统中，使用top命令后，选择M选项，列表会根据物理内存（不包括swap）使用情况对进程进行排序。

如果 top process 不是 oracle 数据库服务进程，联系系统管理员确认是否可以中止 top process。

如果 top process 是 oracle 数据库服务相关进程，则进行以下步骤。

3. 根据步骤2获得的PID，获得消耗内存多的OS里面的oracle进程正在执行的SQL\_ID, SQL CHIND NUMBERDE等信息

```
select sid,event,sql_id from v$session where paddr in (select addr from v$process
where spid = &l);
```

说明：&l代表系统进程号即系统的pid

或者，多个进程同时查询

SQL>set linesize 1000

```
select sid,event,sql_id from v$session where paddr in (select addr from v$process
where spid in (将进程号输入其中，以逗号隔开));
```

4. 根据sql id找到对应的sql语句

查询 sql id 对应的 sql 语句

```
select sql_text from v$sqltext where sql_id='&l';
```

5. 进一步分析sql语句可能存在的性能问题

查询sql id对应执行计划

```
select * from table(dbms_xplan.display_cursor('&sql_id',null,'all last'));
```

### 5.1.11 IO 性能缓慢

#### 【故障现象】

数据库由于 IO 性能缓慢，会导致数据库查询速度下降，继而引起 cache buffer chain, buffer busy wait 等相关等待事件，同时可能因为写 redo 等相关操作缓慢，引起 log file sync, log file switch(checkpoint incomplete)，导致数据库进程大量积压，最终阻塞前端应用

#### 【可能原因】

操作系统 IO 链路异常

存储端硬件故障或者由于 IO 资源被其他主机耗尽

数据库 IO 负载增大（业务压力增大或有异常 SQL 占用大量 IO）



### 【应急措施】

针对操作系统链路异常，如有配置多路径，建议 disable 掉故障链路，避免影响整体 IO 速度，并联系 OS 厂家检查

针对存储端硬件故障，建议联系厂家及时更换损坏硬件尤其是需要定期巡检相关设备，避免出现部件时快时慢的现象。

监控存储端 IO 资源消耗负载，做好 IO 负载隔离以及合理调度

针对数据库 IO 负载增大，需要检查业务量是否为合理增长。并及时查出相关 top SQL 进行优化

### 【后续分析】

AWR 报告分析

➤ 检查AWR top event是否为IO 等相关等待事件，并比对IO等待事件的等待时间是否有明显差异

| Event                   | Waits     | Total Wait Time (sec) | Wait Avg(ms) | % DB time | Wait Class  |
|-------------------------|-----------|-----------------------|--------------|-----------|-------------|
| DB CPU                  |           | 4497.6                |              | 23.5      |             |
| db file sequential read | 4,170,109 | 4092.8                | 1            | 21.4      | User I/O    |
| library cache lock      | 14,718    | 1987.5                | 135          | 10.4      | Concurrency |
| local write wait        | 350,749   | 1098.1                | 3            | 5.7       | User I/O    |
| latch free              | 1,283     | 816.3                 | 636          | 4.3       | Other       |
| cursor: pin S wait on X | 1,520     | 750.5                 | 494          | 3.9       | Concurrency |
| gc cr grant 2-way       | 2,051,140 | 665.5                 | 0            | 3.5       | Cluster     |
| log file sync           | 333,693   | 650.3                 | 2            | 3.4       | Commit      |
| db file scattered read  | 183,219   | 316.3                 | 2            | 1.7       | User I/O    |
| row cache lock          | 53,694    | 254.2                 | 5            | 1.3       | Concurrency |

➤ 对比故障时间段与正常时间段的IO资源消耗，如load profile, io profile, 以及IO stats章节

Load Profile

|              | Per Second | Per Transaction | Per Exec | Per Call |
|--------------|------------|-----------------|----------|----------|
| DB Time(s) : | 10.7       | 0.1             | 0.00     | 0.00     |
| DB CPU(s) :  | 2.5        | 0.0             | 0.00     | 0.00     |

|                               |             |          |  |  |
|-------------------------------|-------------|----------|--|--|
| Redo size (bytes):            | 4,140,878.6 | 21,260.1 |  |  |
| Logical read (blocks):        | 358,309.0   | 1,839.6  |  |  |
| Block changes:                | 16,387.7    | 84.1     |  |  |
| Physical read (blocks):       | 7,590.9     | 39.0     |  |  |
| Physical write (blocks):      | 1,333.5     | 6.9      |  |  |
| Read IO requests:             | 3,651.7     | 18.8     |  |  |
| Write IO requests:            | 582.4       | 3.0      |  |  |
| Read IO (MB):                 | 59.3        | 0.3      |  |  |
| Write IO (MB):                | 10.4        | 0.1      |  |  |
| Global Cache blocks received: | 125.5       | 0.6      |  |  |
| Global Cache blocks served:   | 4.7         | 0.0      |  |  |
| User calls:                   | 35,174.4    | 180.6    |  |  |
| Parses (SQL):                 | 17,042.0    | 87.5     |  |  |
| Hard parses (SQL):            | 24.3        | 0.1      |  |  |
| SQL Work Area (MB):           | 10.1        | 0.1      |  |  |
| Logons:                       | 3.7         | 0.0      |  |  |
| Executes (SQL):               | 18,216.2    | 93.5     |  |  |
| Rollbacks:                    | 4.4         | 0.0      |  |  |
| Transactions:                 | 194.8       |          |  |  |

## IO profile

|                       | Read+Write Per Second | Read per Second | Write Per Second |
|-----------------------|-----------------------|-----------------|------------------|
| Total Requests:       | 4,736.6               | 3,670.0         | 1,066.6          |
| Database Requests:    | 4,234.1               | 3,651.7         | 582.4            |
| Optimized Requests:   | 0.0                   | 0.0             | 0.0              |
| Redo Requests:        | 473.6                 | 5.1             | 468.5            |
| Total (MB):           | 84.4                  | 64.6            | 19.8             |
| Database (MB):        | 69.7                  | 59.3            | 10.4             |
| Optimized Total (MB): | 0.0                   | 0.0             | 0.0              |
| Redo (MB):            | 9.3                   | 5.1             | 4.2              |
| Database (blocks):    | 8,924.4               | 7,590.9         | 1,333.5          |
| Via Buffer Cache      | 6,116.8               | 4,850.7         | 1,266.0          |

(blocks):

|                  |         |         |      |
|------------------|---------|---------|------|
| Direct (blocks): | 2,807.6 | 2,740.1 | 67.5 |
|------------------|---------|---------|------|

对比上述指标是否有明显差异

检查 IO stat，查看 IO 资源主要消耗在哪里

| Function Name      | Reads: Data | Reqs per sec | Data per sec | Writes: Data | Reqs per sec | Data per sec | Waits: Count | Tm (m) |
|--------------------|-------------|--------------|--------------|--------------|--------------|--------------|--------------|--------|
| Buffer Cache Reads | 65.8G       | 3623.45      | 37.849M      | 0M           | 0.00         | 0M           | 4386.3K      | 0      |
| Direct Reads       | 37.1G       | 24.23        | 21.337M      | 0M           | 0.01         | 0M           | 0            | 0      |
| Others             | 9.1G        | 16.40        | 5.247M       | 9.1G         | 14.16        | 5.209M       | 25.6K        | 0      |
| DBWR               | 0M          | 0.00         | 0M           | 17.2G        | 579.63       | 9.885M       | 0            | 0      |
| LGWR               | 52M         | 1.85         | .029M        | 7.3G         | 469.99       | 4.215M       | 641.9K       | 0      |
| Direct Writes      | 124M        | 0.07         | .07M         | 938M         | 2.28         | .527M        | 0            | 0      |
| Streams AQ         | 0M          | 0.00         | 0M           | 0M           | 0.00         | 0M           | 3            | 0      |
| TOTAL:             | 112.2G      | 3666.01      | 64.532M      | 34.5G        | 1066.08      | 19.835M      | 5053.8K      | 0      |

## 1. ASH 分析

由于 AWR 采集间隔通常为半小时一次，没有办法很准确地反应瞬时 IO 问题的真实状态，因此我们往往需要 ASH dump 来分析发生时间很短的 IO 问题。

针对 IO 问题，我们主要关注的是故障时间段内，IO 相关的等待事件例如 db file scattered read、db file sequential read、controlfile sequential read、log file parallel write 的等待时间是否有明显增加的迹象

## 2. OSW 分析

OSW 主要是采集系统状态信息的工具，因此我们可以通过 OSW 工具来查看当时的 IO 情况

➤ 通常我们首先会查看 vmstat，确认是否有很高的 run queue，block queue，和 io wait，还需要对比正常时间段的值来确认是否正常

➤ 其次可以通过 iostat 查看每块盘的具体状态，是否长时间使用率为 100%，或者队列比较高

➤ 通过查看oswps我们可以确认数据库核心进程的状态是否为D，D状态通常为等待在IO，如果短时间的为D状态，为正常现象，但是如果长时间为D状态，并且不可终止，就极有可能hang在IO上

## 5.2 数据库 HANG 故障诊断

### 5.2.1 HANG 分析方法

- 查看 ALERT LOG 中是否有报错
- 查看相关会话信息
- 通过 V\$SESSION\_WAIT 查看等待事件
- 做 ASH 报告
- 做 AWR 报告
- 检查长时间执行的 SQL
- 通过 HANGANALYZE 分析查看是否有 HANG 住现象
- 查看操作系统资源

### 5.2.2 注意事项

3.1.3 至 3.1.5 步最好保存全程操作记录(匹配时间),其日志信息将会占用大量的物理磁盘(跟收集级别和进程数量有关)

### 5.2.3 收集 HangAnalyze 信息

```
$ date
$ sqlplus -prelim / as sysdba
SQL> oradebug setmypid;
SQL> oradebug unlimit;
SQL> oradebug -g all dump hanganalyze 10;
wait 30 secs..
SQL> oradebug -g all dump hanganalyze 10;
SQL> oradebug tracefile_name;
SQL> oradebug close_trace;
```

### 5.2.4 收集 SystemState 信息（在 Oracle support 建议下收集）

场景：当数据库发生整体性的性能问题（异常等待过多，hang、大规模阻塞等）

空间评估方法： 参照经验数据：

某省级移动公司 BOSS 系统大型数据库有 4000-5000 个会话 SGA 64G

节点 1 dump 时间大概 50 分钟左右/次.dump 大小 1.5G/次，2 次大概 3G 左右。

节点 2 dump 时间要少一些, 30 分钟左右/次,dump 出来的大小约 1G/次.

但需要说明的: 此值仅为参考值, 因为客户实际的环境和负载不同, 具体时间范围可能有所变化。

到底使用什么级别进行收集?

如果有 oracle support 指导, 那么按照指导设置级别。

如果没有 oracle support 指导, 那么建议设置级别为 10

请预先腾出可用空间。

```
$ date
$ sqlplus -prelim / as sysdba
SQL> oradebug setmypid;
SQL> oradebug unlimit;
SQL> oradebug -g all dump systemstat 10;
wait 30 secs..
SQL> oradebug -g all dump systemstat 10;
SQL> oradebug tracefile_name;
SQL> oradebug close_trace;
```

#### 5.2.5 收集 ErrorStack 信息（在 Oracle support 建议下收集）

场景: 针对特定进程进行 errorstack 信息收集的时候使用, 在 oracle support 建议下使用

```
$ date
$ sqlplus -prelim / as sysdba
SQL> oradebug setmypid;
SQL> oradebug unlimit;
SQL> oradebug -g all errorstack 3;
wait 30 secs..
SQL> oradebug -g all errorstack 3;
SQL> oradebug tracefile_name;
SQL> oradebug close_trace;
```

#### 5.2.6 收集相关日志信息

收集章节 2.8 的所有日志

## 6 高可用集群

### 6.1 RAC 集群

#### 6.1.1 数据库实例和集群启停

##### 6.1.3.1 停止数据库实例

直接关闭 GI 也会将数据库实例以 `abort` 方式一起关闭，因此关闭 GI 前需要手工关闭数据库实例。

使用 `grid` 用户登录 RAC 集群数据库服务器节点 1 进行关闭所有监听器。

```
//停止数据库监听器
$srvctl stop listener -l <listener_name> -n <node_name>    #<listener_name>和
node_name为具体的，
//检查是否还有遗留数据库实例连接，和业务人员确认后终止
$ps -ef |grep LOCAL=NO|grep -v ASM
$ps -ef |grep LOCAL=NO|grep -v ASM | grep -v grep | awk `{print $2}` | xargs kill -
9
```

使用 `oracle` 用户登录 RAC 集群数据库服务器所有节点。

```
//进行关闭前实例状态确认
SQL>Select group#,status,archived from v$log where status in ('ACTIVE','CURRENT');

SQL>select RECOVERY_ESTIMATED_IOS,ACTUAL_REDO_BLKs,TARGET_REDO_BLKs,
LOG_FILE_SIZE_REDO_BLKs,TARGET_MTtr,ESTIMATED_MTtr from v$instance_recovery;
```

当前 `ACTIVE` 状态的 `REDO LOG` 的数目不大于 1 个，观察 `v$instance_recovery` 视图中 `estimated_mttr` 值是可以接受的数据库启动时间。如果上述条件满足，可以立刻关闭数据库，否则最好延迟数据库的关闭,等待 I/O 量下降，否则异常关闭数据库，会导致数据库启动的时间达到 `estimated_mttr` 的时间。

使用 `grid` 用户登录 RAC 集群数据库服务器节点 1。

```
//依次停止RAC集群数据库所有实例
$srvctl stop instance -d <db_name> -i <instance_name>    #默认使用immediate方式
//确认实例状态
$ps -ef|grep pmon
$srvctl status database -d <db_name>
```

如长时间无法正常关闭实例，可考虑使用 `shutdown abort` 方式终止数据库实例。

数据库实例作为一种资源，可单独进行启动。使用 `grid` 用户登录 RAC 数据库服务器所需启动节点进行启动。

```
//启动数据库实例
$srvctl start instance -d <db_name> -i <instance_name>
<db_name>和<instance_name>分别为数据库名和实例名。
```

```
//检查数据库实例状态
$srvctl status database -d <dbname>
$ps -ef|grep pmon
```

使用 **oracle** 用户登录数据库服务器查看实例状态。

```
SQL>select status from v$instance;
SQL>alter system switch logfile;
SQL>alter system checkpoint;
```

**grid** 用户执行

```
//启动数据库监听器
$srvctl start listener -l <listener_name> -n <node_name>
//检查服务注册状态
lsnrctl status <listener_name>
service状态应当为READY
若实例及服务没有完成注册，使用下面命令手工注册
```

**oracle** 用户执行

```
SQL>alter system register;
```

### 6.1.3.2 启动数据库实例

数据库实例作为一种资源，可单独进行启动。使用 **grid** 用户登录 RAC 数据库服务器所需启动节点进行启动。

```
//启动数据库实例
$srvctl start instance -d <db_name> -i <instance_name>
<db_name>和<instance_name>分别为数据库名和实例名。
//检查数据库实例状态
$srvctl status database -d <dbname>
$ps -ef|grep pmon
```

使用 **oracle** 用户登录数据库服务器查看实例状态。

```
SQL>select status from v$instance;
SQL>alter system switch logfile;
SQL>alter system checkpoint;
```

**grid** 用户执行

```
//启动数据库监听器
$srvctl start listener -l <listener_name> -n <node_name>
//检查服务注册状态
lsnrctl status <listener_name>
service状态应当为READY
```

若实例及服务没有完成注册，使用下面命令手工注册

oracle 用户执行

```
SQL>alter system register;
```

### 6.1.3.3 启动 cluster 和 ohasd

//启动 cluster 和 ohasd

root 用户执行

```
#<GI_HOME>/bin/crsctl start crs
```

该命令会同时启动 cluster 资源和 ohasd 资源。

//检查 ohasd 资源是否启动成功

grid 用户执行

```
$crsctl stat res -t -init
```

该命令检查 ohasd 资源启动情况，ohasd 资源先于 cluster 资源启动，ohasd 资源启动完成后才进行 cluster 资源启动。

//检查 cluster 资源是否启动成功

grid 用户执行

```
$crsctl stat res -t
```

检查方法和 10gR2 及 11gR1 类似，除了 gsd 资源，其他资源应当都处于 online 状态。

### 6.1.3.4 停止 cluster 和 ohasd

//确认数据库实例关闭

grid 用户执行

```
$ps -ef|grep pmon  
$crsctl stat res -t
```

直接关闭 GI 也会将数据库实例以 abort 方式一起关闭，因此关闭 GI 前需要手工关闭数据库实例，方法见前章节<数据库关闭>。

//停止 cluster 和 ohasd

root 用户执行

```
#<GI_HOME>/bin/crsctl stop crs
```

11gR2 上该命令会同时关闭 cluster 资源和 ohasd 资源，需要单独关闭 cluster 可以使用 crsctl stop cluster 命令。

//检查关闭是否完成

grid 用户执行

```
$ps -ef|grep crsd
```



```
$ps -ef|grep ohasd
```

正常关闭情况下，**cluster** 资源关闭完成，**crsd** 进程会结束，**ohasd** 资源关闭完成，**ohasd** 进程结束。

#### 6.1.3.5 重启 **cluster** 和 **ohasd**

//当在启动过程中遇到问题，解决后想要重新启动集群，可以使用如下的方法：

**grid** 用户执行

```
$<GI_HOME>/bin/crsctl check crs
$<GI_HOME>/bin/crsctl stat res -t -init
$<GI_HOME>/bin/crsctl stat res -t
$ps -ef | egrep 'init|d.bin'
```

检查 **GI** 当前的运行状态。

//强制停止已经启动的 **GI** 相关组件

**root** 用户执行

```
#<GI_HOME>/bin/crsctl stop crs -f
```

**-f** 代表强制关闭 **GI** 相关组件。

//检查关闭是否完成

**grid** 用户执行

```
$ps -ef|grep crsd
$ps -ef|grep ohasd
```

正常情况下，**cluster** 资源关闭完成，**crsd** 进程会结束，**ohasd** 资源关闭完成，**ohasd** 进程结束。

//启动 **cluster** 和 **ohasd**

**root** 用户执行

```
#<GI_HOME>/bin/crsctl start crs
```

该命令会同时启动 **cluster** 资源和 **ohasd** 资源。

//检查 **ohasd** 和 **cluster** 资源是否启动成功

**grid** 用户执行

```
$<GI_HOME>/bin/crsctl check crs
$<GI_HOME>/bin/crsctl stat res -t -init
$<GI_HOME>/bin/crsctl stat res -t
```

该命令检查 **ohasd** 资源和 **cluster** 资源启动情况，**ohasd** 资源先于 **cluster** 资源启动，**ohasd** 资源启动完成后才进行 **cluster** 资源启动。

## 6.1.2 RAC 集群资源状态检测

### 6.1.3.1 cluster 资源状态监测

grid 用户执行

```
$crsctl check crs
$crsctl stat res -t
```

### 6.1.3.2 ohasd 资源状态监测

grid 用户执行

```
$crsctl stat res -t -init
```

该命令检查 ohasd 资源启动情况，ohasd 资源先于 cluster 资源启动，ohasd 资源启动完成后才进行 cluster 资源启动。

### 6.1.3.3 监听资源状态监测

grid 用户执行

```
srvctl status listener
srvctl status scan
```

### 6.1.3.4 其他资源状态监测

grid 用户执行

```
Usage: srvctl status database -d <db_unique_name> [-f] [-v]
Usage: srvctl status instance -d <db_unique_name> {-n <node_name> | -i
<inst_name_list>} [-f] [-v]
Usage: srvctl status service -d <db_unique_name> [-s "<service_name_list>"] [-f] [-v]
Usage: srvctl status nodeapps
Usage: srvctl status vip {-n <node_name> | -i <vip_name>} [-v]
Usage: srvctl status listener [-l <lsnr_name>] [-n <node_name>] [-v]
Usage: srvctl status asm [-n <node_name>] [-a] [-v]
Usage: srvctl status scan [-i <ordinal_number>] [-v]
Usage: srvctl status scan_listener [-i <ordinal_number>] [-v]
Usage: srvctl status srvpool [-g <pool_name>] [-a]
Usage: srvctl status server -n "<server_list>" [-a]
Usage: srvctl status oc4j [-n <node_name>] [-v]
Usage: srvctl status home -o <oracle_home> -s <state_file> -n <node_name>
Usage: srvctl status filesystem -d <volume_device> [-v]
Usage: srvctl status diskgroup -g <dg_name> [-n "<node_list>"] [-a] [-v]
Usage: srvctl status cvu [-n <node_name>]
Usage: srvctl status gns [-n <node_name>] [-v]
For detailed help on each command and object and its options use:
  srvctl <command> <object> -h
```

### 6.1.3 RAC 集群资源状态检测

#### 6.1.3.1 OCR/OLR 状态监测

root 用户执行

```
$GRID_HOME/bin/ocrcheck
$GRID_HOME/bin/ocrcheck -local
```

ocrcheck 输出示例如下:

```
[root@rac1 ~]# /oracle/app/12.2.0/grid/bin/ocrcheck
Status of Oracle Cluster Registry is as follows :
    Version                        :          4
    Total space (kbytes)           :    409568
    Used space (kbytes)            :     2244
    Available space (kbytes)       :    407324
    ID                             : 1685090119
    Device/File Name               :      +CRS
                                   Device/File integrity check succeeded
                                   Device/File not configured
                                   Device/File not configured
                                   Device/File not configured
                                   Device/File not configured
    Cluster registry integrity check succeeded
    Logical corruption check succeeded
```

ocrcheck -local 输出示例如下

```
[root@rac1 ~]# /oracle/app/12.2.0/grid/bin/ocrcheck -local
Status of Oracle Local Registry is as follows :
    Version                        :          4
    Total space (kbytes)           :    409568
    Used space (kbytes)            :     1152
    Available space (kbytes)       :    408416
    ID                             : 213592667
    Device/File Name               : /oracle/app/12.2.0/grid/cdata/rac1.olr
                                   Device/File integrity check succeeded
    Local registry integrity check succeeded
    Logical corruption check succeeded
```

#### 6.1.3.2 OCR/OLR 备份情况监测

grid 用户执行

```
ocrconfig -showbackup
ocrconfig -local -showbackup
```

ocrconfig -showbackup 输出示例如下：

```
-bash-3.2$ ocrconfig -showbackup
rac1      2022/05/20 00:46:53      +DATA:/rac/OCRBKUP/backup00.ocr.296.1105145209
476343201
rac1      2022/05/19 20:46:46      +DATA:/rac/OCRBKUP/backup01.ocr.295.1105130801
476343201
rac1      2022/05/15 21:50:42      +DATA:/rac/OCRBKUP/backup02.ocr.326.1104789037
476343201
rac1      2022/05/19 20:46:46      +DATA:/rac/OCRBKUP/day.ocr.285.1105130807
476343201
rac1      2022/05/19 20:46:46      +DATA:/rac/OCRBKUP/week.ocr.281.1105130807
476343201
PROT-25: Manual backups for the Oracle Cluster Registry are not available
```

OCR 每 4 个小时自动备份一次，保存最近的 3 份，以及每天和每周的备份各一个。

ocrconfig -local -showbackup 输出示例如下：

```
-bash-3.2$ ocrconfig -local -showbackup
rac1      2022/04/08 23:11:02
/oracle/app/12.2.0/grid/cdata/rac1/autobackup_20220408_231102.olr      476343201
rac1      2022/04/06 14:25:18
/oracle/app/12.2.0/grid/cdata/rac1/autobackup_20220406_142518.olr      476343201
rac1      2020/03/25 00:36:51
/oracle/app/12.2.0/grid/cdata/rac1/backup_20200325_003651.olr      476343201
```

OLR 在初始化时自动备份一次。

#### 6.1.4 集群服务状态检查

使用 grid 用户登录

```
$crsctl stat res -t
```

#### 6.1.5 集群 alert 日志检查

12C及以上：

```
su - grid
```

```
more $ORACLE_BASE/diag/crs/`hostname`/crs/trace/alert.log
```

11G:

```
su - grid
```

```
more $ORACLE_HOME/log/`hostname`|tr [A-Z] [a-z]/alert`hostname`|tr [A-Z] [a-z]`.log
```

### 6.1.6 集群 crsd 日志检查

```
12C及以上:
su - grid
more $ORACLE_BASE/diag/crs/`hostname`/crs/trace/crsd.trc
11G:
su - grid
more $ORACLE_HOME/log/`hostname`|tr [A-Z] [a-z]`/crsd/crsd.log
```

### 6.1.7 集群 cssd 日志检查

```
12C及以上:
su - grid
more $ORACLE_BASE/diag/crs/`hostname`/crs/trace/ocssd.trc
11G:
su - grid
more $ORACLE_HOME/log/`hostname`|tr [A-Z] [a-z]`/cssd/ocssd.log
```

### 6.1.8 ASM 日志检查

```
su - grid
more $ORACLE_BASE/diag/asm/+asm/$ORACLE_SID/trace/alert_$ORACLE_SID.log
```

### 6.1.9 数据库 alert 日志检查

```
su - oracle
more $ORACLE_BASE/diag/rdbms/`echo $ORACLE_SID`|tr -d [0-9]| tr [A-Z] [a-z]`/$ORACLE_SID/trace/alert_$ORACLE_SID.log
```

### 6.1.10 Clusterware 集群日志总结

#### 6.1.3.1 11G 集群日志

除了 SCAN 监听器日志和用于存储 OCR 和表决磁盘的 ASM 实例日志符合 ADR 结构，其他 Clusterware 的进程和服务都不符合 ADR 的结构，默认存放在 \$GRID\_HOME/log 目录下。

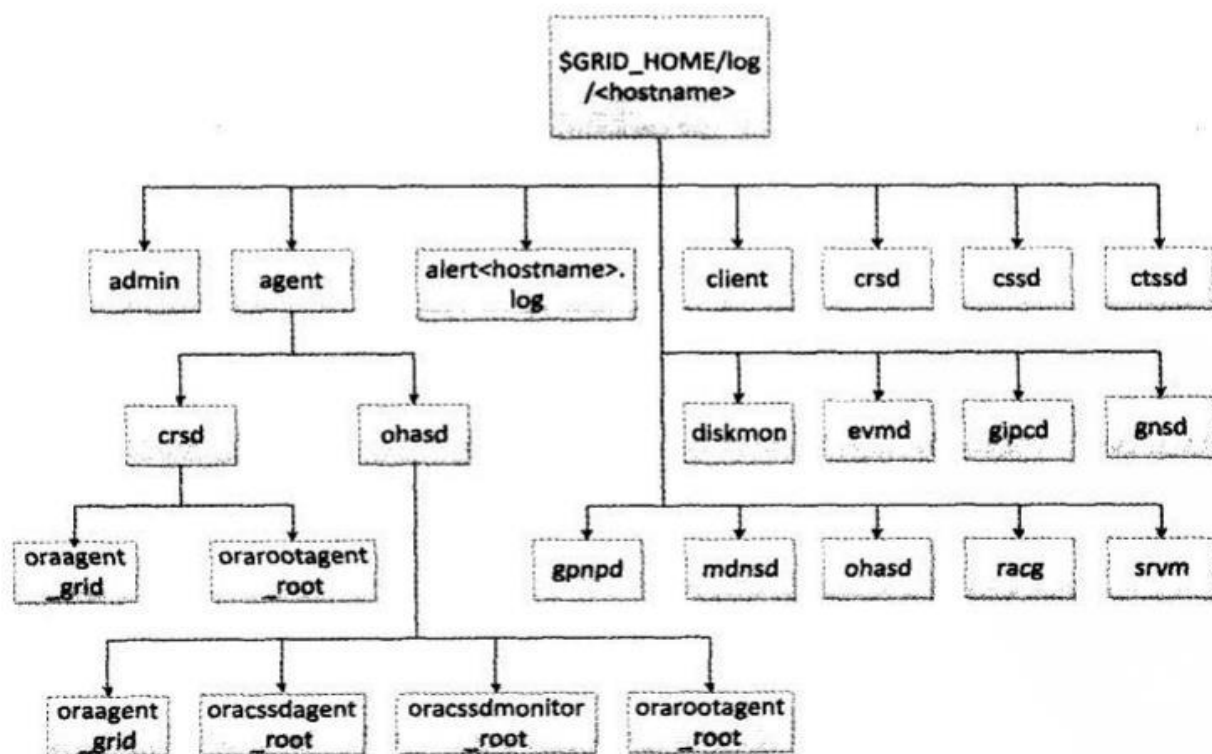


图3-10 Clusterware集群软件日志目录结构

表3-6对图3-10中目录存放的日志内容进行了详细的说明。

表3-6 Clusterware集群软件日志目录结构

| 日志目录或日志                         | 描述                                                 |
|---------------------------------|----------------------------------------------------|
| alert<hostname>.log             | 大多数Clusterware的问题都是通过查看它开始，它是检查、诊断Clusterware问题的入口 |
| admin                           |                                                    |
| agent                           | 代理进程记录的日志信息                                        |
| agent/crsd                      | CRSD进程的代理进程记录的日志信息                                 |
| agent/crsd/oraagent_grid        | CRSD进程的oraagent进程记录的日志信息                           |
| agent/crsd/orarootagent_root    | CRSD进程的orarootagent进程记录的日志信息                       |
| agent/ohasd                     | OHASD进程的代理进程记录的日志信息                                |
| agent/ohasd/oraagent_grid       | OHASD进程的oraagent进程记录的日志信息                          |
| agent/ohasd/oracssdagent_root   | OHASD进程的cssdagent进程记录的日志信息                         |
| agent/ohasd/oracssdmonitor_root | OHASD进程的cssdmonitor进程记录的日志信息                       |
| agent/ohasd/orarootagent_root   | OHASD进程的orarootagent进程记录的日志信息                      |
| client                          | OCR工具记录（OCRCONFIG、OCRCHECK、OCRDUMP）记录的日志信息         |
| crsd                            | CRSD进程记录的日志信息                                      |
| cssd                            | CSSD进程记录的日志信息                                      |
| ctssd                           | CTSSD进程记录的日志信息                                     |
| diskmon                         | diskmon进程记录的日志信息                                   |

|       |                                    |
|-------|------------------------------------|
| evmd  | EVMD进程记录的日志信息                      |
| gipcd | GIPCD进程记录的日志信息                     |
| gnsd  | GNSD进程记录的日志信息                      |
| gpnpd | GPMPD进程记录的日志信息                     |
| mdnsd | MDNSD进程记录的日志信息                     |
| ohasd | OHASD进程记录的日志信息                     |
| racg  | racgcut、racgevtf、racgmain工具记录的日志信息 |
| srvrm | 服务器管理日志信息                          |

根据 Clusterware 启动哪步时出现的问题来检查原因，熟悉启动流程以及诊断过程对我们分析解决问题非常有帮助。

### 6.1.3.2 12C 及以上版本集群日志

12C 及以上版本 Oracle 将集群日志都放在了 \$ORACLE\_BASE/diag/crs/`hostname`/crs/trace。集群警告日志就是：alert.log，其他进程日志都以 .trc 为结尾，该目录也可以在 adrci 中看到。

而 ASM SCAN 监听日志都属于 ADR 的管理范围内，通过 grid 用户的 ADRCI 工具可以实现 ASM SCAN 监听日志的查看。如下图所示，展示了 ASM SCAN 监听日志的 ADR 目录结构。

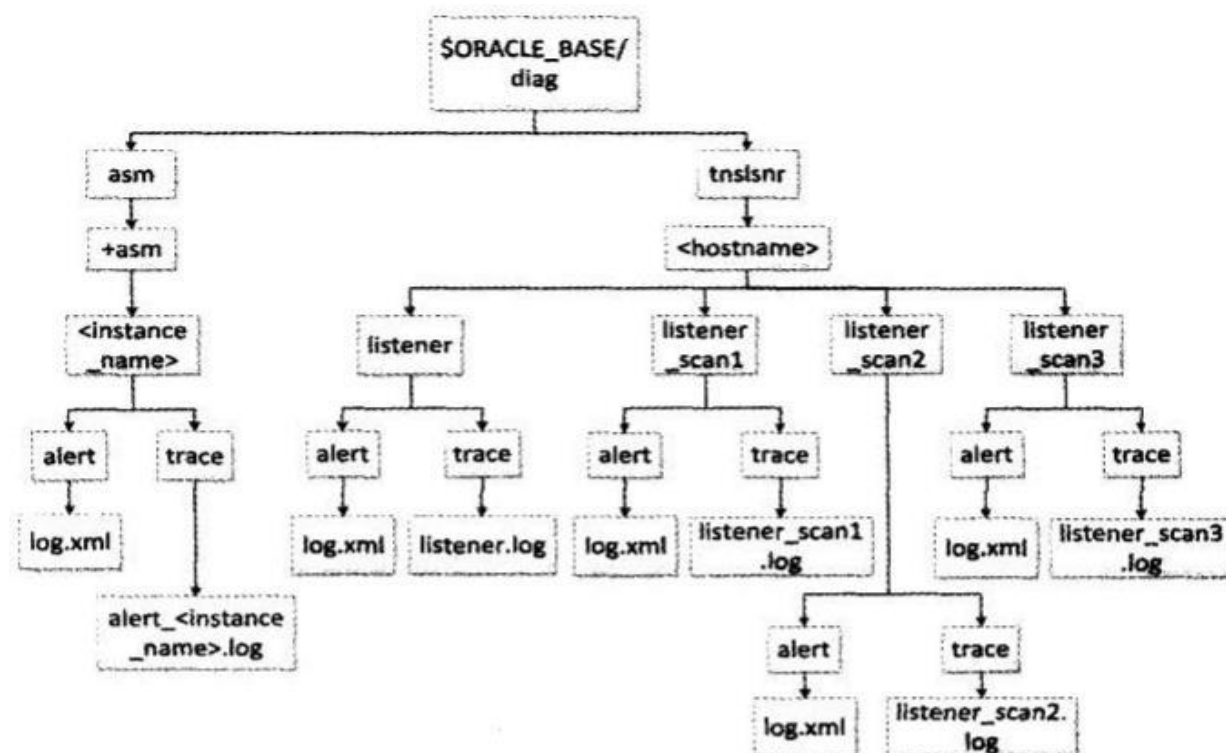


图3-11 ASM和SCAN监听的ADR目录结构

如上图所示，ASM 日志和 SCAN 监听日志都属于 ADR 的目录结构，能够使用 ADR 工具进行统一的管理。在 ASM 和 4 个监听的目录 都有 alert 和 trace 目录，分别保存了以 xml 文件形式和文本文件形式存储的日志信息。

### 6.1.11 RACDiag 信息收集

运行 racdiag.sql 执行。

在相应的目录下执行这个脚本。



需要注意的这个脚本已经包含如下脚本：

```
-- Taking Hang Analyze dumps
-- This may take a little while...
oradebug setmypid
oradebug unlimit
oradebug -g all hanganalyze 3
-- This part may take the longest, you can monitor bdump or udump to see
-- if the file is being generated.
oradebug -g all dump systemstate 267
```

所以如果运行 racdiag.sql 这个脚本，就不再需要做 sytemdump 以及 hanganalyze.

```
sqlplus "/ as sysdba"
sql> @racdiag.sql
```

性能影响：因为含有 systemdump，做执行时间可能会相对较长. 其执行时间受 systemdump 时间限制.

之后去 bdump 以及 udmp 检查执行时间点产生的文件.

参见： Script to Collect RAC Diagnostic Information (racdiag.sql) [ID 135714.1].

## 6.2 DataGuard

### 6.2.1 查看 DG 是否同步

使用 oracle 用户登录 DG 主机，查看归档日志情况：

```
SQL>select max(sequence#), thread#, dest_id, applied from v$archived_log group by
thread#, dest_id, applied order by thread#;
SQL>select max(sequence#), thread# from v$log group by thread#;
```

比较第二条语句查询出的 dg 主库最大日志号是否跟第一条语句查询出的记录。



```
[xtjk@stmt-rac-db1 /home/xtjk]$ sqlplus /nolog
SQL*Plus: Release 11.2.0.4.0 Production on Sun Aug 30 19:13:07 2020
Copyright (c) 1982, 2013, Oracle. All rights reserved.

SQL> conn xtjk
Enter password:
ERROR:
ORA-01017: invalid username/password; logon denied

SQL> conn xtjk
Enter password:
Connected.
SQL> select max(sequence#),thread#,dest_id,applied from v$archived_log group by thread#,dest_id,applied order by thread#;
```

| MAX(SEQUENCE#) | THREAD# | DEST_ID | APPLIED |
|----------------|---------|---------|---------|
| 102788         | 1       | 1       | NO      |
| 102788         | 1       | 2       | YES     |
| 102788         | 1       | 3       | YES     |
| 47079          | 2       | 1       | NO      |
| 47079          | 2       | 2       | NO      |
| 47078          | 2       | 2       | YES     |
| 47079          | 2       | 3       | NO      |
| 47078          | 2       | 3       | YES     |

```
8 rows selected.

SQL> select max(sequence#),thread# from v$log group by thread#;
```

| MAX(SEQUENCE#) | THREAD# |
|----------------|---------|
| 102789         | 1       |
| 47080          | 2       |

```
SQL>
```

APPLIED 代表了备库是否应用了日志，YES 为应用了、NO 为没有应用；MAX (sequence#) 代表了最大的日志号；Tread#代表哪一个主库；Dest\_ID 代表备库号，1 为主库、2 为备库 1、3 为备库 2。

上述例子中主库为一套 RAC (2 台双活的数据库主机)，带了 2 个 DG 备库，也就是说两台 RAC 主机的日志都要在备库应用。

- (1) 先看 RAC1 即 Thread#=1 的那几行，DEST\_ID=1 不用看，DEST\_ID=2 即备库 1 的已应用的最大归档日志号为 102788 跟第二条语句查询出的 Thread#=1 的最大日志号相差 1，正在同步中。
- (2) 接下来看 Thread#=1 和 DEST\_ID=3 的那一行，即备库 2 的已应用的最大归档日志号也为 102788，正在同步中。
- (3) 接下来看 RAC2 即 Thread#=2 的那几行，DEST\_ID=1 仍然不用看，DEST\_ID=2 即备库 1 的已应用的最大归档日志号为 47078 跟第二条语句查询出的 Thread#=2 的最大日志号相差 2，正在同步中。
- (4) 接下来看 Thread#=2 和 DEST\_ID=3 的那几行，即备库 2 的已应用的最大归档日志号为 47078 跟第二条语句查询出的 Thread#=2 的最大日志号相差 2，正在同步中。

### 6.2.2 日志应用问题 MRP 进程异常

MRP0 (Manager Recover Process)：负责日志应用，只要启用到日志应用才会有这个服务。在物理 standby 中是 MRP，在逻辑 standby 中是 LSP。

状态值：

WAIT\_FOR\_LOG：等待日志

APPLYING\_LOG：正在应用日志

#### 6.2.3.1 MRP 同步进程没有启动或者启动异常

- 确认MRP进程的状态

//确认MRP进程是否启动

```
SQL> SELECT PROCESS,STATUS, THREAD#,SEQUENCE#, BLOCK#, BLOCKS FROM
V$MANAGED_STANDBY ;
```

```
//检查standby当前应用状态，是否应用，是否实时应用
SQL> SELECT RECOVERY_MODE FROM V$ARCHIVE_DEST_STATUS WHERE RECOVERY_MODE!=' IDLE' ;
====
RECOVERY_MODE
-----
MANAGED REAL TIME APPLY      -- 表示实时应用
```

- 停止MRP进程

```
SQL> alter database recover managed standby database cancel;
```

- 启动MRP

- 执行下面命令启动 MRP 进程进行 apply log (非实时)

```
SQL> alter database recover managed standby database disconnect from session ;
```

- 执行下面命令启动 MRP 进程进行 apply log (实时)

```
SQL> alter database recover managed standby database using current logfile
disconnect from session;
```

注意：多个节点的 RAC 只有一个节点有 MRP 进程进行同步。

### 6.2.3.2 应用日志较慢

- 查看 redo 应用速度 (Active Apply Rate)

```
SQL> Col Values For A65
SQL> Col Recover_start For A21
SQL> Select To_char(START_TIME,'YYYY-MM-DD Hh24:Mi:ss')
"Recover_start",To_char(Item)||' = '||To_char(Sofar)||' '||To_char(Units)||' '||
To_char(TIMESTAMP,'YYYY-MM-DD Hh24:Mi') "Values" From V$Recovery_progress
Where Start_time=(Select Max(Start_time) From V$Recovery_progress);
```

- 在介质恢复或redo应用期间，都需要读取重做日志文件，默认都是串行恢复，我们可以在执行recover的时候加上parallel子句来指定并行度，提高读取和应用的性能，例如

```
SQL> alter database recover managed standby database parallel 128 disconnect from
session;
- 推荐parallel的值是#CPUs*2;
```

- 出现固定间隔apply lag
- apply lag 为对上次应用的间隔时间。每进行apply之后，该时间会清零
- 如果 apply lag 时间较大（5分钟以上，甚至20分钟），很有可能就是由于 standby redo log 一直没有被使用，所以每次是apply 主库传过来的归档日志。
- 由于归档日志生成与主库redo 量产生频率相关，所以间隔不一，在主库空闲时间，产生归档日志时间较长，导致apply lag变长。
- 经存储复制 (BCV 复制)后 的 Standby redo log 需要重建。

例

```
SQL> select group#, thread#, bytes/1024/1024 "SRL Size in MBytes", status from
v$standby_log order by 2,1;
```

| GROUP# | THREAD# | SRL Size in MBytes | STATUS |      |
|--------|---------|--------------------|--------|------|
| 13     | 1       | 1020 UNASSIGNED    | 《---   | 未被使用 |
| 14     | 1       | 1020 UNASSIGNED    |        |      |
| 15     | 1       | 1020 UNASSIGNED    |        |      |
| 16     | 1       | 1020 UNASSIGNED    |        |      |
| 17     | 1       | 1020 UNASSIGNED    |        |      |

- 实际查看 standby redo log 所在盘，确认权限正常，盘正常挂在，但却无法使用。怀疑由主库BCV复制后需要重建。

```
$ls -ltr /dev/rcrmaredo01
crw-rw---- 1 oracle dba          61,  1 Jul 23 13:48 /dev/rcrmaredo01
```

- 进行下列操作重新创建 standby redo log

#### 暂停 MRP0 进程

```
SQL> alter database recover managed standby database cancel ;
```

确认 standby redo log 的重建路径。可能是 DG 或者 LV;

```
SQL> select b.THREAD#, a.group#, a.type, a.member, b.bytes, b.bytes/1024/1024 as
MB, a.status from v$logfile a, v$standby_log b where a.group#=b.group# and
a.type='STANDBY';
```

将备库文件管理方式设置为手动

```
alter system set standby_file_management = manual scope=memory sid='*';
alter database drop standby logfile group 13;
alter database drop standby logfile group 13;
alter database drop standby logfile group 14;
alter database drop standby logfile group 15;
alter database drop standby logfile group 16;
alter database drop standby logfile group 17;
alter database drop standby logfile group 18;
alter database drop standby logfile group 19;

alter database drop standby logfile group 20;
alter database drop standby logfile group 21;
alter database drop standby logfile group 22;
alter database drop standby logfile group 23;
alter database drop standby logfile group 24;
alter database drop standby logfile group 25;
alter database drop standby logfile group 26;
ALTER DATABASE ADD standby LOGFILE THREAD 1
GROUP 14 '/dev/rzwbredo3g_52' SIZE 3070m,
```

```
GROUP 15 '/dev/rzwbredo3g_53' SIZE 3070m,
GROUP 16 '/dev/rzwbredo3g_54' SIZE 3070m,
GROUP 17 '/dev/rzwbredo3g_55' SIZE 3070m,
GROUP 18 '/dev/rzwbredo3g_56' SIZE 3070m,
GROUP 19 '/dev/rzwbredo3g_57' SIZE 3070m

ALTER DATABASE ADD standby LOGFILE THREAD 2
GROUP 20 '/dev/rzwbredo3g_58' SIZE 3070m,
GROUP 21 '/dev/rzwbredo3g_59' SIZE 3070m,
GROUP 22 '/dev/rzwbredo3g_60' SIZE 3070m,
GROUP 23 '/dev/rzwbredo3g_61' SIZE 3070m,
GROUP 24 '/dev/rzwbredo3g_62' SIZE 3070m,
GROUP 25 '/dev/rzwbredo3g_63' SIZE 3070m,
GROUP 26 '/dev/rzwbredo3g_64' SIZE 3070m
```

### 将备库管理方式设置回自动

```
alter system set standby_file_management = auto scope=memory sid='*';
SQL> /
```

| GROUP# | THREAD# | SRL | Size in MBytes | STATUS     |
|--------|---------|-----|----------------|------------|
| 13     | 1       |     | 1020           | ACTIVE     |
| 14     | 1       |     | 1020           | UNASSIGNED |
| 15     | 1       |     | 1020           | UNASSIGNED |

NAME VALUE

```
-----
apply lag +00 00:00:00
apply finish time +00 00:00:00.000
```

如果启动失败，则需要查看 MRP0 trace，进一步检查

#### 6.2.3.3 备库不能连接

检查备库各节点参数 remote\_listener 设置

#### 6.2.3.4 日志传输问题

##### RFS 进程没有正常启动

- RFS (Remote File Server) 进程：接收primary数据库的redo，保存在standby redo log (arch模式不写standby，直接保存归档)
- 状态值有：
  - WRITING：进程活跃，正在把redo写到归档日志中

- IDLE: 空闲, 没有日志同步
- RECEIVING: 收听网络通信
- OPENING: 正在打开归档日志

日志传输问题先检查 RFS 进程是否正常启动:

--确认RFS进程是否启动

```
SQL> SELECT PROCESS, STATUS, THREAD#, SEQUENCE#, BLOCK#, BLOCKS FROM
V$MANAGED_STANDBY ;
```

====

| PROCESS | STATUS    | THREAD# | SEQUENCE# | BLOCK# | BLOCKS |
|---------|-----------|---------|-----------|--------|--------|
| ARCH    | CLOSING   | 1       | 388       | 356352 | 886    |
| DGRD    | ALLOCATED | 0       | 0         | 0      | 0      |
| DGRD    | ALLOCATED | 0       | 0         | 0      | 0      |
| ARCH    | CLOSING   | 1       | 384       | 358400 | 1856   |
| ARCH    | CLOSING   | 1       | 386       | 358400 | 446    |
| ARCH    | CLOSING   | 1       | 387       | 354304 | 1200   |
| RFS     | IDLE      | 1       | 389       | 131446 | 1      |
| RFS     | IDLE      | 0       | 0         | 0      | 0      |
| RFS     | IDLE      | 1       | 0         | 0      | 0      |

--以上代表RFS是启动状态

如果未启动, 需要启动 listener , 如果无反应, 可以将主库执行

```
SQL> ALTER system SET log_archive_dest_state_2 = 'defer' ;
```

后再 enable, 备库的 RFS 进程就正常启动了。

### 6.2.3 MRP 进程正常, 但 DG 有 GAP

使用 oracle 用户登录备机查看日志有没有在应用:

```
SQL>select process, thread#, status, sequence#, BLOCK# from v$managed_standby;
```

```
SQL> select process,thread#,status,sequence#,BLOCK# from v$managed_standby;
```

| PROCESS | THREAD# | STATUS       | SEQUENCE# | BLOCK# |
|---------|---------|--------------|-----------|--------|
| ARCH    | 1       | CLOSING      | 443       | 342016 |
| DGRD    | 0       | ALLOCATED    | 0         | 0      |
| DGRD    | 0       | ALLOCATED    | 0         | 0      |
| ARCH    | 0       | CONNECTED    | 0         | 0      |
| ARCH    | 1       | CLOSING      | 444       | 344064 |
| ARCH    | 1       | CLOSING      | 414       | 335872 |
| RFS     | 0       | IDLE         | 0         | 0      |
| RFS     | 1       | IDLE         | 445       | 109912 |
| RFS     | 0       | IDLE         | 0         | 0      |
| RFS     | 0       | IDLE         | 0         | 0      |
| MRP0    | 1       | APPLYING_LOG | 445       | 109912 |

11 rows selected.

```
SQL> select process,thread#,status,sequence#,BLOCK# from v$managed_standby;
```

| PROCESS | THREAD# | STATUS       | SEQUENCE# | BLOCK# |
|---------|---------|--------------|-----------|--------|
| ARCH    | 1       | CLOSING      | 443       | 342016 |
| DGRD    | 0       | ALLOCATED    | 0         | 0      |
| DGRD    | 0       | ALLOCATED    | 0         | 0      |
| ARCH    | 0       | CONNECTED    | 0         | 0      |
| ARCH    | 1       | CLOSING      | 444       | 344064 |
| ARCH    | 1       | CLOSING      | 414       | 335872 |
| RFS     | 0       | IDLE         | 0         | 0      |
| RFS     | 1       | IDLE         | 445       | 109912 |
| RFS     | 0       | IDLE         | 0         | 0      |
| RFS     | 0       | IDLE         | 0         | 0      |
| MRP0    | 1       | APPLYING_LOG | 445       | 109912 |

11 rows selected.

```
SQL>
```

多次执行查看 MRP0 APPLYING\_LOG 那一行，块号应该在变化，即备机在同步着。  
如果没有在应用尝试重启 mrp 服务（基本上上海数据中心 DG 环境都为实时）

```
SQL>alter database recover managed standby database cancel;
SQL>alter database recover managed standby database using current logfile
disconnect from session;
```

如果还是没有应用，登录备机查看是否有 gap 并查看 alert 日志

```
SQL>select * from v$archive_gap;
$tail -1000f alert*
```

如果有 gap，到备机/arch 目录下查看归档日志是否存在，

(1) 如果存在则等一下再执行

```
SQL>select process,thread#,status,sequence#,BLOCK# from v$managed_standby;
```

(2) 如果不存在，则将缺少的归档日志从主机复制到备机，记得要修改拷贝过来的归档文件名跟备机其他归档日志格式一致，当然文件的属主、属组和权限也要一致

复制完了之后再执行：

```
SQL>alter database recover managed standby database cancel;
```

```
SQL>alter database recover managed standby database using current logfile disconnect from
session;
```

下面看看有没有应用日志了，并查看 GAP

```
SQL>select process,thread#,status,sequence#,BLOCK# from v$managed_standby;
SQL>select * from v$archive_gap;
```

如果还是没有应用日志并且 gap 还存在，则查看缺少的日志有没有注册在控制文件中（取一个范围将缺少的归档日志号包含在内即可）

```
SQL>select name from v$archived_log where sequence# between xxx and xxx;
```

查看缺少的日志有没有在控制文件中，如果没有要手动注册一下

例如：

```
SQL>alter database register logfile '/arch/实例名目录/1_XXXXX_XXXXXXX.dbf' ;
```

然后再

```
SQL>alter database recover managed standby database cancel;
SQL>alter database recover managed standby database using current logfile
disconnect from session;
SQL>select process,thread#,status,sequence#,BLOCK# from v$managed_standby;
```

## 7 常用工具或方法

### 7.1 主机资源类工具

#### 7.1.1 OSWatcher 工具

下载文档：Metalink Doc ID 301137.1

建议用最新版本进行部署。

Oswatcher /OSWBB 主要用于监控主机资源，如 CPU, 内存, 网络以及私有网络等. 其中私有网络需要单独配置.

需要说明的执行 OSWatcher 用户需要常见 OS 命令 的执行权限.  
如下所述.

OSW will need access to the OS utilities: *top*, *vmstat*, *iostat*, *mpstat*, *netstat*, and *andtraceroute*. These OS utilities need to be install on the system prior to running OSW. Execute permission on these utilities need to be granted to the user of OSW.

支持的平台有：

AIX

Tru 64

Solaris

HP-UNIX

Linux

可以使用如下类似脚本 方便进行搜集:

例如:

```
# cd <oswbb_dir>/archive
# find . -name '*18.07.16.1[4-5]00*' -print -exec zip
/archlog1/oswbb/archive/osw_20180716_14_15`hostname`.zip {} \;
```

name '\*18.07.16.1[4-5]00\*' 这个是表示 18 年 07 月 16 号 14 点到 15 点的监控信息被打包成一个 zip 文件

- 1) 需要注意的是 配置私有网络监控 需要手工配置, 具体参考下面.
- 2) 注意: 一般建议保存3天及以上, 如果保存2天, 遇到周末, 可能出现在周一取不到之前周五发生问题的诊断数据, 会对诊断工作带来不利.
- 3) 运行OSW 的时候, 最好不要将当前目录放置在"/" 根目录下.

用 nohup 在后台运行 osw: 可以参考如下方式为正确使用方式,

cd /your-osw-dir/osw/oswbb ->先定位到当前目录

nohup ./ startOSWbb.sh 30 168 &

#### ● 安装Oswatcher/OSWBB

1. tar zxvf 解压osw安装文件\*.tar.gz
2. osw相应的脚本工具

#### ● 运行Oswatcher/OSWBB

开始运行 osw

运行 startOSWbb.sh 脚本, 这个脚本包括两个参数

参数 1: 快照时间间隔, 单位为秒

参数 2: 快照记录保持的时间, 单位为小时

如, 用 nohup 在后台运行 osw:

cd /your-osw-dir/osw/oswbb

nohup ./ startOSWbb.sh 30 168 &

#### ● 查看osw运行数据/OSWBB运行数据

osw 运行后在 osw 的安装目录中会创建 archive 目录, 目录中包括 IOSTAT/VMWAT/TOP/等数据, 可以直接查看.



- 中止Oswatcher/OSWBB运行数据  
使用 stopOSWbb.sh 脚本

#### 例 1

表示 每 60s 搜集 1 次, 保存 10 小时

```
./startOSWbb.sh 60 10
```

#### 例 2

默认是每 30s 搜集 1 次, 保存 48 小时

```
./startOSWbb.sh
```

#### 例 3

表示 60s 搜集 1 次, 保存 10 小时, 后台运行.

```
nohup ./startOSWbb.sh 60 10 &
```

- 配置私有网络监控

如果需要监控私有网络, 可以使用 Exampleprivate.net 的参考, 建立 private.net , 并且给予执行权限, 并且使用下面的内容拷内进去, 之后运行 Oswatcher 即可 . 不同操作系统的建立方法不一样. 参见如下:

```
#####
#Solaris Example
#####
echo "zzz ***" `date`
traceroute -r -F node1
traceroute -r -F node2
#####
#HP Example
#####
echo "zzz ***" `date`
traceroute -r -F node1
traceroute -r -F node2
#####
#Linux Example
#####
echo "zzz ***" `date`
traceroute -r -F node1
traceroute -r -F node2
#####
```

```
#Tru64 Example
#####
echo "zzz ***" `date`
traceroute -r -f node1
traceroute -r -f node2
#####
#AIX Example
#####
echo "zzz ***" `date`
traceroute -r node1
traceroute -r node2
#####
# DO NOT DELETE THE FOLLOWING LINE!!!!!!!!!!!!!!!!!!!!!!
#
#####
rm locks/lock.file
```

## 7.2 SQL 优化类工具

### 7.2.1 SQLHC 工具

注：SQLHC 不需要安装，是 Oracle 提供的一种用来诊断 SQL 语句调优问题的工具，为最常用的 sql 性能分析工具。

优点：不需要安装。

缺点：其取的信息会比 SQLT 少一些，但考虑到客户实际全部部署的难度，

下载地址：SQL Tuning Health-Check Script (SQLHC) (Doc ID 1366133.1)

- 1) 登陆进系统
- 2) 将下载的 sqlhc.zip 放在常用的脚本目录下面
- 3) 以 SYSDBA 用户登陆，执行 sqlhc.sql 会出现如下提示，需要输入 2 个参数，1 个为 T。(表示购买了 Oracle Tuning pack)，之后输入相应的 SQLID 即可。

```
SQL> @sqlhc.sql
Parameter 1:
Oracle Pack License (Tuning, Diagnostics or None) [T|D|N] (required)
Enter value for 1: T
PL/SQL procedure successfully completed.

Parameter 2:
SQL_ID of the SQL to be analyzed (required)
Enter value for 2: djkbv8vkc64h
```

也可以直接输入:

For example:

```
# sqlplus / as sysdba
```

```
SQL> START sqlhc.sql T djkbvr8vkc64h
```

## 7.2.2 SQL Tuning advisor 工具

### ● 相关步骤

1. 创建一个 task --注意修改要优化的 sql\_id 以及 task\_name

```
set long 80000
```

```
col recs format a90
```

```
SET LONGCHUNKSIZE 1000
```

```
SET LINESIZE 200
```

```
set pages 2000
```

```
DECLARE
```

```
l_sql_tune_task_id VARCHAR2(100);
```

```
BEGIN
```

```
l_sql_tune_task_id := DBMS_SQLTUNE.create_tuning_task (
```

```
sql_id => '6cw0ux5g7c54k',
```

```
scope => DBMS_SQLTUNE.scope_comprehensive,
```

```
time_limit => 500,
```

```
task_name => '6cw0ux5g7c54k',
```

```
description => 'Tuning task1 for statement 6cw0ux5g7c54k');
```

```
DBMS_OUTPUT.put_line('l_sql_tune_task_id: ' || l_sql_tune_task_id);
```

```
END;
```

```
/
```

2. 执行 task 生成优化报告

```
EXEC DBMS_SQLTUNE.execute_tuning_task(task_name => '6cw0ux5g7c54k');
```

3. 查看优化报告

```
SELECT DBMS_SQLTUNE.REPORT_TUNING_TASK('6cw0ux5g7c54k') FROM DUAL;
```

4. 执行优化建议, 并确认优化结果

建议会以具体语句的形式给出, 有可能是建议你创建索引, 也有可能是让你绑定 profile 等。只需评估后直接执行即可。执行优化建议后确认是否得到了优化。

5. 删除 task

```
EXEC DBMS_SQLTUNE.drop_tuning_task(task_name => '6cw0ux5g7c54k');
```

### ● 示例

example:

sql\_id: 1j393agbufgvw

```
SELECT /*+ ordered */ O.NAME RSNAME, RU.NAME RSHEMA, RO.NAME RNAME, RM.PROPERTY, DECODE(RL.ECTX#, 0, DECODE(RM.ECTX#, 0, ' ', '"" || BU.NAME || '"/" || BO.NAME || '""),
' ') BONAME,

RM.RM_COMMENT FROM SYS.OBJ$ O, SYS.RULE_MAP$ RM, SYS.OBJ$ RO, SYS.RULE$ RL, SYS.USER$ RU, SYS.OBJ$ BO, SYS.USER$ BU WHERE O.OBJ# = :B1 AND O.OBJ# = RM.RS_OBJ#
AND RM.R_OBJ# = RL.OBJ# AND RL.OBJ# = RO.OBJ# AND RO.OWNER# = RU.USER# AND RM.ECTX# = BO.OBJ#(+) AND BO.OWNER# = BU.USER#(+) AND ((RU.NAME, RO.NAME, 59)
NOT IN (SELECT NE.OWNER, NE.NAME, NE.OBJ_TYPE FROM SYS.NOEXP$ NE))
```

1. execute tuning task for sql\_id 1j393agbufgvw as below and upload the report to SR

**创建一个 task**

**create tuning task:**

**set long 80000**

**col recs format a90**

**SET LONGCHUNKSIZE 1000**

**SET LINESIZE 200**

**set pages 2000**

**DECLARE**

**l\_sql\_tune\_task\_id VARCHAR2(100);**

**BEGIN**

**l\_sql\_tune\_task\_id := DBMS\_SQLTUNE.create\_tuning\_task (**

**sql\_id => '1j393agbufgvw',**

**scope => DBMS\_SQLTUNE.scope\_comprehensive,**

**time\_limit => 500,**

**task\_name => '1j393agbufgvw',**

**description => 'Tuning task1 for statement 1j393agbufgvw');**

**DBMS\_OUTPUT.put\_line('l\_sql\_tune\_task\_id: ' || l\_sql\_tune\_task\_id);**

**END;**

**/**

2. execute tuning task:

**EXEC DBMS\_SQLTUNE.execute\_tuning\_task(task\_name => '1j393agbufgvw');**

fetch tuning report:

3. **SELECT DBMS\_SQLTUNE.REPORT\_TUNING\_TASK('1j393agbufgvw') FROM DUAL;**

4. As the below result suggests there is a better execution plan found for the statement.

Accept the recommended SQL profile

**execute dbms\_sqltune.accept\_sql\_profile(task\_name => '1j393agbufgvw', task\_owner => 'SYS', replace => TRUE);**



```
| 0 | SELECT STATEMENT | | 11 | 1551 | 115K (1) | 00:23:01 |
|* 1 | FILTER | | | | |
|* 2 | HASH JOIN OUTER | | 11 | 1551 | 115K (1) | 00:23:01 |
| 3 | NESTED LOOPS OUTER | | 11 | 1375 | 115K (1) | 00:23:01 |
|* 4 | HASH JOIN | | 11 | 1056 | 115K (1) | 00:23:01 |
|* 5 | HASH JOIN | | 11 | 880 | 115K (1) | 00:23:01 |
| 6 | TABLE ACCESS FULL | RULE$ | 56120 | 493K | 555 (1) | 00:00:07 |
| 7 | MERGE JOIN CARTESIAN | | 30M | 2081M | 114K (1) | 00:22:51 |
| 8 | MERGE JOIN CARTESIAN | | 11 | 462 | 7 (15) | 00:00:01 |
| 9 | TABLE ACCESS BY INDEX ROWID | OBJ$ | 1 | 25 | 4 (0) | 00:00:01 |
|* 10 | INDEX RANGE SCAN | I_OBJ1 | 1 | | 3 (0) | 00:00:01 |
| 11 | BUFFER SORT | | 11 | 187 | 3 (34) | 00:00:01 |
| 12 | TABLE ACCESS BY INDEX ROWID | RULE_MAP$ | 11 | 187 | 2 (0) | 00:00:01 |
|* 13 | INDEX RANGE SCAN | I_RULE_MAP1 | 11 | | 1 (0) | 00:00:01 |
| 14 | BUFFER SORT | | 2795K | 77M | 114K (1) | 00:22:51 |
| 15 | INDEX FAST FULL SCAN | I_OBJ2 | 2795K | 77M | 10383 (1) | 00:02:05 |
| 16 | TABLE ACCESS FULL | USER$ | 467 | 7472 | 8 (0) | 00:00:01 |
| 17 | TABLE ACCESS BY INDEX ROWID | OBJ$ | 1 | 29 | 3 (0) | 00:00:01 |
|* 18 | INDEX RANGE SCAN | I_OBJ1 | 1 | | 2 (0) | 00:00:01 |
| 19 | TABLE ACCESS FULL | USER$ | 467 | 7472 | 8 (0) | 00:00:01 |
|* 20 | TABLE ACCESS FULL | NOEXP$ | 1 | 27 | 2 (0) | 00:00:01 |
```

Predicate Information (identified by operation id):

```
1 - filter( NOT EXISTS (SELECT /*+ FULL ("NE") */ 0 FROM "SYS"."NOEXP$" "NE" WHERE
"NE"."OBJ_TYPE"=59 AND "NE"."NAME"=:B1 AND "NE"."OWNER"=:B2))

2 - access("BO"."OWNER#"="BU"."USER#"(+))

4 - access("RO"."OWNER#"="RU"."USER#")

5 - access("RM"."R_OBJ#"="RL"."OBJ#" AND "RL"."OBJ#"="RO"."OBJ#")

10 - access("O"."OBJ#"=:B1)

13 - access("RM"."RS_OBJ#"=:B1)

18 - access("RM"."CTX#"="BO"."OBJ#"(+))

20 - filter("NE"."OBJ_TYPE"=59 AND "NE"."NAME"=:B1 AND "NE"."OWNER"=:B2)
```

2- Using SQL Profile

Plan hash value: 945621219

```
| Id | Operation | Name | Rows | Bytes | Cost (%CPU) | Time |
```

```
| 0 | SELECT STATEMENT | | 6 | 846 | 103 (0) | 00:00:02 |
|* 1 | FILTER | | | | |
```

```
| 2 | NESTED LOOPS OUTER | | 6 | 846 | 97 (0) | 00:00:02 |
|* 3 | HASH JOIN | | 6 | 750 | 91 (0) | 00:00:02 |
| 4 | NESTED LOOPS OUTER | | 6 | 600 | 87 (0) | 00:00:02 |
|* 5 | HASH JOIN | | 11 | 781 | 54 (0) | 00:00:01 |
| 6 | NESTED LOOPS | | 11 | 605 | 46 (0) | 00:00:01 |
| 7 | NESTED LOOPS | | 11 | 605 | 46 (0) | 00:00:01 |
| 8 | NESTED LOOPS | | 11 | 286 | 13 (0) | 00:00:01 |
| 9 | TABLE ACCESS BY INDEX ROWID| RULE_MAP$ | 11 | 187 | 2 (0) | 00:00:01 |
|* 10 | INDEX RANGE SCAN | I_RULE_MAP1 | 11 | | 1 (0) | 00:00:01 |
| 11 | TABLE ACCESS BY INDEX ROWID| RULE$ | 1 | 9 | 1 (0) | 00:00:01 |
|* 12 | INDEX UNIQUE SCAN | I_RULE | 1 | | 0 (0) | 00:00:01 |
|* 13 | INDEX RANGE SCAN | I_OBJ1 | 1 | | 2 (0) | 00:00:01 |
| 14 | TABLE ACCESS BY INDEX ROWID | OBJ$ | 1 | 29 | 3 (0) | 00:00:01 |
| 15 | TABLE ACCESS FULL | USER$ | 467 | 7472 | 8 (0) | 00:00:01 |
| 16 | TABLE ACCESS BY INDEX ROWID | OBJ$ | 1 | 29 | 3 (0) | 00:00:01 |
|* 17 | INDEX RANGE SCAN | I_OBJ1 | 1 | | 2 (0) | 00:00:01 |
| 18 | TABLE ACCESS BY INDEX ROWID | OBJ$ | 1 | 25 | 4 (0) | 00:00:01 |
|* 19 | INDEX RANGE SCAN | I_OBJ1 | 1 | | 3 (0) | 00:00:01 |
| 20 | TABLE ACCESS CLUSTER | USER$ | 1 | 16 | 1 (0) | 00:00:01 |
|* 21 | INDEX UNIQUE SCAN | I_USER# | 1 | | 0 (0) | 00:00:01 |
|* 22 | TABLE ACCESS FULL | NOEXP$ | 1 | 27 | 2 (0) | 00:00:01 |
```

### 7.2.3 SQL Monitor 工具

#### ● 主要步骤

1. 找到需要监控的 sql\_id

```
SQL> select sql_id,sql_text from v$sqlarea where sql_text like '%xxxxxx%';
```

2. 查看监控报告 一可以指定 txt 格式, html 格式等, 另外注意替换 sql\_id

```
set long 1000000
```

```
set longchunksize 100000
```

```
set linesize 1000
```

```
set pagesize 0
```

```
set trim on
```

```
set trimspool on
```

```
set echo off
```

```
set feedback off
```

```
spool report_sql_monitor.txt
```

```
select dbms_sqltune.report_sql_monitor(sql_id=>'3pw8cra3wj4mk', type=>'TEXT', report_level=>'ALL') as
```

```
report from dual;
spool off

或者 html 格式如下:
set long 1000000
set longchunksize 100000
set linesize 1000
set pagesize 0
set trim on
set trimspool on
set echo off
set feedback off
spool report_sql_monitor.html
select dbms_sqltune.report_sql_monitor(sql_id=>'3pw8cra3wj4mk', type=>'HTML', report_level=>'ALL') as
report from dual;
spool off
```

## ● 示例

### 1.生成txt格式报告

```
1. 确认需要监控的 sql_id
SQL> select sql_id,sql_text from v$sqlarea where sql_text='select count(1) from test.OBJECTS_PAR
a,test.OBJECTS_PAR b';
3pw8cra3wj4mk
select count(1) from test.OBJECTS_PAR a,test.OBJECTS_PAR b

SQL>

2. 查看 SQL_MONITOR 报告
SQL> set long 1000000
SQL> set longchunksize 100000
SQL> set linesize 1000
SQL> set pagesize 0
SQL> set trim on
SQL> set trimspool on
SQL> set echo off
SQL> set feedback off
SQL> spool report_sql_monitor.txt
SQL> select dbms_sqltune.report_sql_monitor(sql_id=>'3pw8cra3wj4mk', type=>'TEXT', report_level=>'ALL') as report from dual;
spool offSQL Monitoring Report

SQL Text
-----
select count(1) from test.OBJECTS_PAR a,test.OBJECTS_PAR b
```



#### Global Information

```

-----
Status           : DONE (ALL ROWS)
Instance ID      : 2
Session          : SYS (3196:32975)
SQL ID           : 3pw8cra3wj4mk
SQL Execution ID  : 33554432
Execution Started : 07/07/2022 17:00:30
First Refresh Time : 07/07/2022 17:00:39
Last Refresh Time : 07/07/2022 17:13:54
Duration         : 804s
Module/Action     : sqlplus@rac19c2 (TNS V1-V3)/-
Service          : SYS$USERS
Program          : sqlplus@rac19c2 (TNS V1-V3)
Fetch Calls      : 1
  
```

#### Global Stats

```

=====
| Elapsed | Cpu   | Other | Fetch | Buffer |
| Time(s) | Time(s) | Waits(s) | Calls | Gets |
=====
|      804 |    489 |    315 |      1 |   2848 |
=====
  
```

#### SQL Plan Monitoring Details (Plan Hash Value=3787850338)

| Id | Operation            | Name        | Rows    | Cost | Time      | Start  | Execs | Rows     | Mem   | Activity | Activity Detail |
|----|----------------------|-------------|---------|------|-----------|--------|-------|----------|-------|----------|-----------------|
|    |                      |             | (Estim) |      | Active(s) | Active |       | (Actual) | (Max) | (%)      | (# samples)     |
| 0  | SELECT STATEMENT     |             |         |      | 796       | +9     | 1     | 1        | .     |          |                 |
| 1  | SORT AGGREGATE       |             | 1       |      | 805       | +0     | 1     | 1        | .     |          |                 |
| 2  | MERGE JOIN CARTESIAN |             | 4G      | 69M  | 796       | +9     | 1     | 4G       | .     |          |                 |
| 3  | PARTITION RANGE ALL  |             | 62872   | 1092 | 796       | +9     | 1     | 62872    | .     |          |                 |
| 4  | TABLE ACCESS FULL    | OBJECTS_PAR | 62872   | 1092 | 796       | +9     | 4     | 62872    | .     |          |                 |
| 5  | BUFFER SORT          |             | 62872   | 69M  | 804       | +1     | 62872 | 4G       | 1MB   |          |                 |
| 6  | PARTITION RANGE ALL  |             | 62872   | 1091 | 1         | +9     | 1     | 62872    | .     |          |                 |
| 7  | TABLE ACCESS FULL    | OBJECTS_PAR | 62872   | 1091 | 1         | +9     | 4     | 62872    | .     |          |                 |

SQL>

## 2.生成html格式报告

```

1. 确认需要监控的 sql_id
SQL> select sql_id,sql_text from v$sqlarea where sql_text='select count(1) from test.OBJECTS_PAR
a,test.OBJECTS_PAR b';
3pw8cra3wj4mk
select count(1) from test.OBJECTS_PAR a,test.OBJECTS_PAR b

SQL>

2. 查看 SQL_MONITOR 报告
set long 1000000
set longchunksize 100000
set linesize 1000
set pagesize 0
set trim on
set trimspool on
set echo off
set feedback off
spool report_sql_monitor.html
select          dbms_sqltune.report_sql_monitor(sql_id=>'3pw8cra3wj4mk',          type=>'HTML',
report_level=>'ALL') as report from dual;
spool off

```

产生的html报告如下:

SQL> select dbms\_sqltune.report\_sql\_monitor(sql\_id=>'3pw8cra3wj4mk', type=>'HTML', report\_level=>'ALL') as report from dual;

SQL Monitoring Report

SQL Text

select count(1) from test.OBJECTS\_PAR a, test.OBJECTS\_PAR b

Global Information: DONE (ALL ROWS)

|                    |                                |             |               |
|--------------------|--------------------------------|-------------|---------------|
| Instance ID        | : 2                            |             |               |
| Session            | : SYS (3196:32975)             |             |               |
| SQL ID             | : 3pw8cra3wj4mk                | Buffer Gets | Database Time |
| SQL Execution ID   | : 33554432                     | 2848        | 804s          |
| Execution Started  | : 07/07/2022 17:00:30          |             | Wait Activity |
| First Refresh Time | : 07/07/2022 17:00:39          |             |               |
| Last Refresh Time  | : 07/07/2022 17:13:54          |             |               |
| Duration           | : 894s                         |             |               |
| Module/Action      | : sqplus@rac19c2 (TNS V1-V3)/- |             |               |
| Service            | : SYSDBGERS                    |             |               |
| Program            | : sqplus@rac19c2 (TNS V1-V3)   |             |               |
| Fetch Calls        | : 1                            |             |               |

SQL Plan Monitoring Details (Plan Hash Value=3787850338)

| ID | Operation            | Name        | Estimated Rows | Cost | Active Period (secs) | Execs | Rows  | Memory (Max) | Temp (Max) | IO Requests | CPU Activity | Wait Activity |
|----|----------------------|-------------|----------------|------|----------------------|-------|-------|--------------|------------|-------------|--------------|---------------|
| 0  | SELECT STATEMENT     |             |                |      |                      | 1     | 1     |              |            |             |              |               |
| 1  | SORT AGGREGATE       |             | 1              |      |                      | 1     | 1     |              |            |             |              |               |
| 2  | MERGE JOIN CARTESIAN |             | 4G             | 69M  |                      | 1     | 4G    |              |            |             |              |               |
| 3  | PARTITION RANGE ALL  |             | 62872          | 1092 |                      | 1     | 62872 |              |            |             |              |               |
| 4  | TABLE ACCESS FULL    | OBJECTS_PAR | 62872          | 1092 |                      | 4     | 62872 |              |            |             |              |               |
| 5  | BUFFER SORT          |             | 62872          | 69M  |                      | 62872 | 4G    | 1.5MB        |            |             |              |               |
| 6  | PARTITION RANGE ALL  |             | 62872          | 1091 |                      | 1     | 62872 |              |            |             |              |               |
| 7  | TABLE ACCESS FULL    | OBJECTS_PAR | 62872          | 1091 |                      | 4     | 62872 |              |            |             |              |               |

SQL> spool off

## 7.2.4 SQLT 工具

SQLTXPLAIN (简称 SQLT) 是 Oracle 提供的一种用来诊断 SQL 语句调优问题的工具。通常, 当用户遇到一个 SQL 语句调优问题时, Oracle Support 会要求提供很多的信息, 比如 SQL 语句, 10046/10053

trace, 对象统计信息, optimizer 信息等等。这些信息的收集是非常繁琐的事情, 而且需要用户具有深入的产品知识。由于信息收集不全, 或者客户不知道如何收集, 导致一些问题最终无法解决。Oracle 提供了 SQLT 这个工具来解决这个问题。

### 应用场景:

- 1) 收集单条SQL语句的信息, 包括:

| 序号 | 说明                           |
|----|------------------------------|
| 1  | SQL 文本                       |
| 2  | 执行计划(explain)                |
| 3  | 真实执行计划(row source operation) |
| 4  | 运行时的信息(时间, 记录数等等)            |
| 5  | 10046/10053 trace            |

- 2) 收集影响optimizer的统计信息(包括SYSTEM statistics和Object statistics)。

- 3) 收集所有可能会影响optimizer计算的参数:

| 序号 | 说明                    |
|----|-----------------------|
| 1  | 平台                    |
| 2  | 版本                    |
| 3  | NLS 信息                |
| 4  | 初始化参数                 |
| 5  | fix_control           |
| 6  | gather_statistics_job |

- 4) 如果Oracle Tuning Packs是可以使用的话, 自动调用SQL Tuning Advisor来生成优化报告。

- 5) 自动导出相关数据以方便客户/Oracle Support/Oracle Development创建 test case来重现客户的问题, 加速问题的处理。

- 6) 自动生成SQL Profile的脚本, 来固定SQL的执行计划。

### 风险说明:

需要创建专门的 schema 来管理 SQLT 的数据。目的是为了和用户数据进行隔离, 说明如下:

- 1) SQLT是用SQL和PL/SQL编写的，可以运行在各种不同版本操作系统的数据库。
- 2) 代码都是非加密的，可以清楚的看到操作的内容。
- 3) 不会泄露用户敏感数据。比如Column的最大/最小值还有histogram可以选择隐藏起来。
- 4) 安装在自己的schema下，不会影响用户数据。
- 5) 免费下载和使用，只需有一个可用的My Oracle Support license。

#### 下载说明：

- 1) 下载文档 ID:215187.1中的附件（需登陆My Oracle Support）。目前SQLT有两个不同的版本，一个是基于9.2/10.1的，另一个是基于10.2以后的；
- 2) 解压缩文件夹中的sqlt\_instructions.html或者SQLTXPLAIN.pdf。在解压缩文件夹下的doc目录下也可以找到相同的信息(PPT)。
- 3) 在绝大多数的情况下，SQLT包含了诊断SQL语句调优问题的所有必要信息。因此，建议尽可能安装并用它来收集信息，以便于问题更有效率的解决。
- 4) 如果您在使用SQLT的过程中遇到问题，或者是有任何改进的建议或意见，可以在文档ID:215187.1下添加英文评论，您的评论会直接反馈给作者。

## 7.3 日志收集类工具

### 7.3.1 TFA 工具

Oracle 公司为了解决用户反馈在故障诊断的过程中，需要使用各种命令，收集各种各样的繁琐的日志，费时费力，于是 Oracle 在 11.2 的版本之后，推出了 Trace File Analyzer Collector (TFA)。专门用来收集 GridInfrastructure/RAC 环境下的诊断日志的工具，它可以用非常简单的命令协助用户收集 RAC 里的日志，以便进一步进行诊断；TFA 是类似 diagcollection 的一个 oracle 集群日志收集器，而且 TFA 比 diagcollection 集中和自动化的诊断信息收集能力更强大。

因为内容较大，详细的搜集方法请见如下文档。



SSC TFA快速安装使用指南v1.0.docx

### 7.3.2 ADRCI 工具

从 11G 开始，alert 和 trace 的信息以一种新的形式产生，并存储在 Automatic Diagnostic Repository (ADR)中。

Oracle11g 提供了一些工具和方法来自动识别和收集与 **problem** (数据库环境中的一个严重错误) 或 **incident** (问题的发生) 相关的跟踪文件。这个功能被称为事件打包服务 (IPS)。Support Workbench (GUI) 和 ADRCI (命令行) 实用工具是事件打包服务的两个接口。数据库中每出现一个严重错误 (问题) 都会创建一个事件。

使用方法如下:

To check for the error reported in the ADR:

With the 11g / 12c environment being set, from the OS command prompt invoke the ADRCI utility:

```
$ adrci
```

Let ADRCI know which Oracle environment to investigate:

```
adrci> show home
```

--show home will list all the ADR homes. Set the proper database ADR home for use with ADRCI.

```
adrci> set homopath database_home
```

Check for the problems reported:

```
adrci> show problem
```

ADR Home = /opt/oracle/diag/rdbms/<DBNAME>/<DBNAME>:

\*\*\*\*\*

| PROBLEM_ID | PROBLEM_KEY | LAST_INCIDENT | LASTINC_TIME                         |
|------------|-------------|---------------|--------------------------------------|
| 1          | ORA 1578    | 18104         | 2009-06-01 22:06:19.501207<br>+10:00 |

1 rows fetched

You can have multiple problems reported. Create a package for the one which needs to be investigated.

### 3. Check for the incidents reported for the problem which needs to be packaged

Each occurrence of the problem is called an 'incident'. The instance alert file will show all the incidents reported. Each incident will be associated with a unique incident id in the ADR.

From the ADRCI prompt, execute the *SHOW INCIDENT* command for the problem to be investigated, by specifying the associated problem key. You can get the problem key from the *SHOW PROBLEM* command:

```
adrci> show incident -p "problem_key='ORA 1578'"
```

ADR Home = /opt/oracle/diag/rdbms/<DBNAME>/<DBNAME>:

\*\*\*\*\*

| INCIDENT_ID | PROBLEM_KEY | CREATE_TIME                          |
|-------------|-------------|--------------------------------------|
| 18147       | ORA 1578    | 2009-06-01 22:02:08.805002<br>+10:00 |

1 rows fetched

You can see multiple incidents reported for the same problem.

Note the incident number for which you need to create a package.

#### 4. Package the trace files invoking IPS.

Generate the package using the *IPS PACK* command and store the generated package in a directory of your choice. The example below shows the package for the incident will be created in the /tmp directory.

```
adrci> ips pack incident 18147 in /tmp
```

Generated package 9 in file /tmp/ORA1578\_20090602113045\_COM\_1.zip, mode complete

If at this step the ADRCI utility complains that the incident was flood-controlled and that no package can be generated for it, then instead of choosing the most recent incident to be packaged, choose the first incident that occurred after an instance startup.

Examples of *IPS PACK* include:

```
ips pack problem 100 in /tmp
```

--Generates the package for the problem id 100 in /tmp

```
ips pack incident 6439 in /tmp
```

--Generates the package for the incident id 6439 in /tmp

```
ips pack problemkey "ORA 1578"
```

--Generates the package for the problem with the problem\_key 'ORA 1578'

```
ips pack seconds 8
```

--Generates the package with the incidents occurred in last 8 seconds.

```
ips pack time '2007-05-01 10:00:00.00' to '2007-05-01 23:00:00.00'
```

```
--Generates the package with the incidents occurred between the times '2007-05-01 10:00:00.00' and
'2007-05-01 23:00:00.00'
```

示例:

```
oracle@slesn1:~> adrci
adrci> show home
ADR Homes:
diag/rdbms/rac19c/rac19c1
diag/rdbms/db19c/rac19c1
diag/rdbms/db19c/db19c1
adrci> set homopath diag/rdbms/rac19c/rac19c1      <<<<<<选择正确的 oracle home
adrci> show problem

ADR Home = /home/oracle/oracle/diag/rdbms/rac19c/rac19c1:
*****

PROBLEM_ID                                PROBLEM_KEY
LAST_INCIDENT          LASTINC_TIME
-----
1                      ORA 7445 [kclollnumiter]          264241
2022-05-06 16:31:12.329000 +08:00
1 row fetched

adrci> show incident -p "problem_key='ORA 7445 [kclollnumiter]'"

ADR Home = /home/oracle/oracle/diag/rdbms/rac19c/rac19c1:
*****

INCIDENT_ID          PROBLEM_KEY                                CREATE_TIME
-----
216241                ORA 7445 [kclollnumiter]          2022-05-06
16:19:32.743000 +08:00
264241                ORA 7445 [kclollnumiter]          2022-05-06
16:31:12.329000 +08:00
2 rows fetched

adrci> ips pack incident 264241 in /tmp      <<<<<<<<<< 收集 incident id=264241 期间产生的日志
Generated package 1 in file /tmp/ORA7445kc_20220714120903_COM_1.zip, mode complete

adrci> ips pack problem 1 in /tmp      <<<<<<<<收集 problem id=1 的错误即 ORA 7445 [kclollnumiter]期间的日志
Generated package 2 in file /tmp/ORA7445kc_20220714121312_COM_1.zip, mode complete
adrci>
```

```
adrci> ips pack problemkey 'ORA 7445 [kclollnumiter]' in /tmp <<<<<<<<收集 ORA 7445 [kclollnumiter]期间的日志
Generated package 3 in file /tmp/ORA7445kc_20220714121343_COM_1.zip, mode complete
adrci>
adrci> ips pack time '2022-07-14 12:00:00' to '2022-07-14 13:00:00' in /tmp <<<<<<<收集某时间段的日志
Generated package 4 in file /tmp/IPSPKG_20220714134007_COM_1.zip, mode complete

adrci>
```

## 7.4 数据库巡检类工具

### 7.4.1 OraCheck 工具

ORAchK 之前被称为 RACcheck，后来它的检查范围进行了扩展，改名为 ORAchK，它是在数据库系统进行健康检查的一个专用工具，这个工具主要用来检查软件的配置是否符合要求以及一些最佳实践是否被应用了。通过这个工具，用户可以很方便地、自动化地对自己的系统进行健康检查和评估。

ORAchK 能够检查的软件主要有：OS、Oracle Clusterware (CRS)、Grid Infrastructure environment (GI)、Automatic Storage Management (ASM) 以及 Real Application Clusters (RAC)、单机的数据库、Golden Gate。

详细下载地址和安装方法见：

ORAchK - Health Checks for the Oracle Stack (Doc ID 1268927.2)

**需要注意：**

需要 OS 侧安装 BASH Shell 程序包。

此外有发现客户自行将已装好的 SHELL 包中拿出可执行文件 Shell 放至未安装 BASH 的机器作为 Workaround，因此工具为非 Oracle 官方提供，建议客户自行测试有效性。

### 7.4.2 oratop 工具

#### 工具描述：

ORATOP 是一个用来监控 Oracle 数据库性能的工具，它是使用 C 开发的工具利用 Oracle OCI API 来查询数据库内部的视图并通过类似 OS 工具「top」的介面来呈现，可以支援 single instance 或是 RAC 的数据库。

#### 应用场景：

- 1) 平台支持：



- 支持linux/unix client端连接
- 可以几乎realtime 监控
- 查找top session/event/sql plan/tbs/asm/dg等等
- 基于console,文本方式

- Supported Oracle Releases:
  - ✓ 11gR2 (11.2.0.3, 11.2.0.4)
  - ✓ 12cR2 (12.1.0.1, 12.1.0.2, 12.2.0.1...
  - ✓ ...

## 下载说明:

orator is now distributed and integrated with Trace File Analyzer Collector (TFA) along with all the other tools that are a part of the Support Tools Bundle.

So rather than downloading orator separately the preferred method for obtaining orator is to [download the latest version of TFA](#).

This integration means that anytime TFA is upgraded (PSUs, download from MOS, etc) the latest version of orator and the other Support Tools of the bundle will also be obtained.

Orator 已集成在 TFA 中, 所以, 无需单独下载安装 orator, TFA 安装后就会带有 orator。同样, TFA 升级完成后, orator 也会得到升级。

下载和使用方法可参考文档: **1500864.1**

## 使用方法:

使用方法参考如下:

```
. 改名.
$ mv orator* orator

2. 授予 execute 权限
$ chmod 755 orator

3. 设置对应环境变量 (如果用户已经设置无需再次配置)
$ export TERM=xterm    #or vt100
```

```
$ export ORACLE_HOME=<11.2 database home>
$ export LD_LIBRARY_PATH=$ORACLE_HOME/lib
$ export PATH=$ORACLE_HOME/bin:$PATH
$ export ORACLE_SID=<local 11.2 database SID to be monitored> #only needed if connecting to a local
database
```

#### 4. 运行

Invoke the tool to monitor a local database as follows (interval can be any interval in seconds):

```
$ ./oratop -i 10 / as sysdba
```

#### 5. 无需 dba 权限

如果使用非 DBA 权限的用户，则需要授予如下权限

```
grant select on v_$instance to <username>;
grant select on v_$parameter to <username>;
grant select on v_$statname to <username>;
grant select on v_$wait_chains to <username>;
grant select on gv_$process to <username>;
grant select on gv_$session to <username>;
grant select on gv_$sysstat to <username>;
grant select on gv_$sesstat to <username>;
grant select on gv_$osstat to <username>;
grant select on gv_$sga to <username>;
grant select on gv_$system_event to <username>;
grant select on gv_$session_event to <username>;
grant select on gv_$session_wait to <username>;
grant select on gv_$sysmetric to <username>;
```

#### 6. 可以使用 alias

```
$ ./oratop -i 10 username/password@tns_alias
$ ./oratop -i 10 system/manager@tns_alias
```

#### 7. 退出按” Q” 键, 或者, CTRL-C.

### 示例:

主要的画面输出:

| Instance                                                                            | DB CPU | Host Load | I/O  | Mem  | Active Sessions | Sessions | TXN           | RESP | DB Time Ratio |                   |            |      |     |      |      |     |                     |  |  |  |  |
|-------------------------------------------------------------------------------------|--------|-----------|------|------|-----------------|----------|---------------|------|---------------|-------------------|------------|------|-----|------|------|-----|---------------------|--|--|--|--|
| <div>prctop6: 1641 stma 21:02:44 op 4.6d, 7 ins, 289G mt, 22 sn, 3 us, 89% db</div> |        |           |      |      |                 |          |               |      |               | HEADER (Section1) |            |      |     |      |      |     |                     |  |  |  |  |
| ID                                                                                  | SQL    | HLU       | MBPS | IOBL | %FR             | PGAU     | ASL           | ASL  | ASW           | ASP               | ARS        | USN  | IPS | ULPS | SSKI | UBL | DBW                 |  |  |  |  |
| 2                                                                                   | 94     | 26        | 1    | 2m   | 60              | 2G       | 19            | 1    | 0             | 0                 | 23.5       | 34   | 0   | 817  | 28m  | 96  | 4                   |  |  |  |  |
| 1                                                                                   | 90     | 25        | 1    | 0m   | 56              | 1G       | 25            | 1    | 0             | 0                 | 22.0       | 31   | 0   | 639  | 33m  | 98  | 2                   |  |  |  |  |
| 7                                                                                   | 87     | 22        | 0    | 0m   | 60              | 785M     | 18            | 3    | 0             | 0                 | 21.2       | 31   | 0   | 956  | 22m  | 99  | 1                   |  |  |  |  |
| 3                                                                                   | 87     | 21        | 1    | 1m   | 60              | 1G       | 21            | 3    | 0             | 0                 | 21.0       | 31   | 0   | 759  | 27m  | 99  | 1                   |  |  |  |  |
| 6                                                                                   | 53     | 13        | 0    | 1m   | 60              | 1G       | 14            | 0    | 0             | 0                 | 12.8       | 23   | 0   | 794  | 16m  | 100 | 0                   |  |  |  |  |
| DATABASE (Section2)                                                                 |        |           |      |      |                 |          |               |      |               |                   |            |      |     |      |      |     |                     |  |  |  |  |
| EVENT (Cumulative) AVG: TOT WAITS TIME(s) AVG_MS PCT WAIT_CLASS                     |        |           |      |      |                 |          |               |      |               |                   |            |      |     |      |      |     |                     |  |  |  |  |
| DB CPU                                                                              |        |           |      |      |                 |          | 621685        |      |               |                   |            | 99   |     |      |      |     |                     |  |  |  |  |
| Disk file Mirror Read                                                               |        |           |      |      |                 |          | 624411        |      |               |                   |            | 2346 |     |      |      |     | 3.8 0 User I/O      |  |  |  |  |
| os thread startup                                                                   |        |           |      |      |                 |          | 1755          |      |               |                   |            | 1066 |     |      |      |     | 607.2 0 Concurrency |  |  |  |  |
| control file parallel write                                                         |        |           |      |      |                 |          | 113221        |      |               |                   |            | 500  |     |      |      |     | 4.4 0 System I/O    |  |  |  |  |
| control file sequential read                                                        |        |           |      |      |                 |          | 958051        |      |               |                   |            | 427  |     |      |      |     | 0.4 0 System I/O    |  |  |  |  |
| DB EVENTS (Section3)                                                                |        |           |      |      |                 |          |               |      |               |                   |            |      |     |      |      |     |                     |  |  |  |  |
| ID                                                                                  | SID    | SPID      | USR  | PROG | PGA             | OPN      | SQLID/BLOCKER | E/T  | STATUS        | STE               | WAIT_EVENT | M/T  |     |      |      |     |                     |  |  |  |  |
| 4                                                                                   | 2038   | 26013     | AR   | S000 | 86M             | SEL      | 8xbbtxc5j901h | 20s  | ACTIVE        | CPU               | wa for cpu | 143m |     |      |      |     |                     |  |  |  |  |
| 4                                                                                   | 2056   | 26054     | AR   | S014 | 77M             | SEL      | 8xbbtxc5j901h | 20s  | ACTIVE        | CPU               | wa for cpu | 107m |     |      |      |     |                     |  |  |  |  |
| 4                                                                                   | 2031   | 32157     | AR   | S007 | 72M             | SEL      | 8xbbtxc5j901h | 18s  | ACTIVE        | CPU               | wa for cpu | 104m |     |      |      |     |                     |  |  |  |  |
| 4                                                                                   | 2027   | 32273     | AR   | S022 | 94M             | SEL      | 8xbbtxc5j901h | 22s  | ACTIVE        | CPU               | wa for cpu | 101m |     |      |      |     |                     |  |  |  |  |
| 4                                                                                   | 2042   | 32257     | AR   | S038 | 84M             | SEL      | 6a1n94p5thb26 | 12s  | ACTIVE        | CPU               | wa for cpu | 100m |     |      |      |     |                     |  |  |  |  |
| 4                                                                                   | 2028   | 32078     | AR   | S029 | 105M            | SEL      | 1fc63hhym257  | 7s   | ACTIVE        | CPU               | wa for cpu | 99m  |     |      |      |     |                     |  |  |  |  |
| 4                                                                                   | 2033   | 26021     | AR   | S001 | 8M              | SEL      | f984r1a4w72bb | 4s   | ACTIVE        | CPU               | wa for cpu | 94m  |     |      |      |     |                     |  |  |  |  |
| 3                                                                                   | 1789   | 29539     | AR   | S013 | 85M             | SEL      | 8xbbtxc5j901h | 11s  | ACTIVE        | CPU               | wa for cpu | 85u  |     |      |      |     |                     |  |  |  |  |
| PROCESSES (Section4)                                                                |        |           |      |      |                 |          |               |      |               |                   |            |      |     |      |      |     |                     |  |  |  |  |

## 8 常用信息收集方法

### 8.1 收集 AWR 报告

Oracle 的 AWR 报告是一个很有用的工具，基本上每次故障或者性能问题都要收集。该报告是通过对比两次快照(snapshot)收集到的统计信息，来生成报表数据的，默认 Oracle 每隔一个小时收集一次 snapshot，如果想手动收集可以执行如下步骤：

```
exec dbms_workload_repository.create_snapshot();
```

收集 AWR 步骤：

使用oracle用户登录

输入：@\${ORACLE\_HOME}/rdbms/admin/awrrpt.sql

Specify Type: text <<默认HTML网页格式，可选TEXT文本，一般html格式就可以

Num\_days: <<最近多少天的AWR快照变化

Begin\_snapshot: <<开始时间的快照编号

End\_snapshot: <<结束时间的快照编号

Report name: <<报告名称

当如果需要收集的实例已经宕机，可以通过下面方法在另一个活着的实例收集宕机实例的 AWR 和 ASH 报告：

```
SELECT output FROM TABLE(DBMS_WORKLOAD_REPOSITORY.AWR_REPORT_HTML( 3616491197, 1, 61143, 61144) );
SELECT output FROM TABLE( DBMS_WORKLOAD_REPOSITORY.ASH_REPORT_HTML( 3616491197, 1, to_date('2010-07-23 11:45:02','yyyy-mm-dd hh24:mi:ss'), to_date('2010-07-23 12:00:24','yyyy-mm-dd hh24:mi:ss')) );
```

## 8.2 收集 ASH 报告

Oracle 的 ASH 报告是发生性能问题时常收集的一种工具，用于收集某一段时间内性能信息。

步骤：

```
使用oracle用户登录
输入: @?/rdbms/admin/ashrpt.sql
Enter value for report_type: <<一般选html
Defaults to -15 mins <<输入开始时间
Enter value for begin_time <<输入开始时间，可以输入-5即5分钟前，或者17:05这样的具体时间
Enter value for duration: <<持续时间，单位为分钟
Enter value for report_name: <<报告名称，回车保持默认即可
```

## 8.3 收集 ASH Dump 数据

--作用是什么？

步骤：

```
使用oracle用户登录
先创建临时存放ash dump数据的表（导出后可以删除）
Create table xyyh_ash_20201223 TABLESPACE USERS as
  select * from dba_hist_active_sess_history
  where sample_time>=to_date('2020-12-23 10:00:00','yyyy-mm-dd hh24:mi:ss')
  and sample_time<=to_date('2020-12-23 13:50:00','yyyy-mm-dd hh24:mi:ss')
  ;
Create Table xyyh_m_sqltext tablespace users As
With tmp As
  (Select /*+ materialize */Distinct sql_id
   From dba_hist_active_sess_history
   Where sample_time Between
        to_timestamp('2020-12-23 10:00:00','yyyy/mm/dd hh24:mi:ss') And
        to_timestamp('2020-12-23 13:50:00','yyyy/mm/dd hh24:mi:ss'))
Select * From dba_hist_sqltext Where sql_id In (Select * From tmp)
;
-- 导出
exp userid='/' as sysdba\ file=ashdata.dmp tables='xyyh_ash_20201223','
xyyh_m_sqltext'
```

开始时间和结束时间按照实际情况调整，注意需要包含故障时段

功能:

- ASH dump 数据用于分析一段时间内 session 的活动情况。

## 8.4 收集 ADDM 信息

使用oracle用户登录  
输入: @?/rdbms/admin/addmrpt.sql

## 8.5 标准 Trace 收集/Alert 信息收集

RAC

1. 每个节点的Alert.log
2. 每个节点的LMS[0|9] trace file.
3. 每个节点的LCK trace file
4. 每个节点的LMON trace文件
5. 每个节点的LMD0 trace文件
6. 每个节点的DIAGA trace 文件
7. Alert log中提到的Trace 文件.

如果是非 RAC

1. 单Instance的Alert.log
2. Alert log中提到的Trace 文件.

性能影响: 无, 只是简单提取数据.

## 8.6 TFA 日志收集

TFA Collector 目的:

Oracle 公司为了解决用户反馈在故障诊断的过程中, 需要使用各种命令, 收集各种各样的繁琐的日志, 费时费力, 于是 Oracle 在 11.2 的版本之后, 推出了 Trace File Analyzer Collector (TFA)。专门用来收集 Grid Infrastructure/RAC 环境下的诊断日志的工具, 它可以用非常简单的命令协助用户收集 RAC 里的日志, 以便进一步进行诊断; TFA 是类似 diagcollection 的一个 oracle 集群日志收集器, 而且 TFA 比 diagcollection 集中和自动化的诊断信息收集能力更强大。

注: TFA 在 Oracle11.2.0.4 以后 (但是 12.1.0.1 并没有包含) 已经是 GI 软件中默认自带的工具了

但是建议下载最新的 Oracle Trace File Analyzer with database support tools bundle, 他定期会有更新并修复一些 bug, 所以推荐使用最新版本。

最新版本可以参考 MOS 文档: Autonomous Health Framework (AHF) - Including TFA and ORAchk/EXAchk (Doc ID 2550798.1)

使用 TFA 收集信息命令如下：

```
- 建议使用root用户进行收集 （建议两个节点都进行收集）
$GI_HOME/tfa/bin/tfactl diagcollect -from "Mar/15/2019 08:00:00" -to " Mar/15/2019 11:00:00"
- 例子表示，收集2019年3月15日8点到11点的集群相关日志内容
```

## 8.7 OS 日志信息搜集

相关信息如下：

Linux: /var/log/messages

Solaris: /var/adm/messages

HP-UX: /var/adm/syslog/syslog.log

Tru64: /var/adm/messages

AIX: Please upload the output of the "errpt" command and the "errpt -a" command, from both nodes

Windows: Save Application Log and System Log as txt files Using Event Viewer

性能影响：无，只是简单提取数据.

## 8.8 OSWatcher 日志收集

Oswatcher/OSWBB 主要用于监控主机资源，如 CPU, 内存, 网络以及私有网络等. 其中私有网络需要单独配置.

可以使用如下类似脚本 方便进行搜集：

例如：

```
# cd <oswbb_dir>/archive
# find . -name '*18.07.16.1[4-5]00*' -print -exec zip
/archlog1/oswbb/archive/osw_20180716_14_15`hostname`.zip {} \;
```

name '\*18.07.16.1[4-5]00\*' 这个是表示 18 年 07 月 16 号 14 点到 15 点的监控信息被打包成一个 zip 文件

## 8.9 HangAnalyze 信息搜集

搜集方法如下：

如果 Instance hang 的状态，这时候可以考虑如下的方法进行：

如果 sqlplus / as sysdba 登录有问题，可以考虑使用 sqlplus -prelim / as sysdba (11g 以前版本)

Show a way to connect to a database through Sqlplus when all other connection methods are

hanging. Using a Sqlplus preliminary connection you will be able to connect to the database since no session is actually created, but you will have limited access to the SGA. This will help in capturing diagnostic information like hanganalyze and systemstate dumps to aid in problem resolution.

```
sqlplus -prelim / as sysdba

sqlplus / as sysdba
SQL> oradebug setmypid
Statement processed.
SQL> oradebug unlimit
Statement processed.
SQL> oradebug hanganalyze 3
等 2 分钟.
SQL>oradebug hanganalyze 3
等 2 分钟
SQL> oradebug tracefile_name
Hang Analysis in /usr/app/oracle/admin/myrac/bdump/myrac1_diag_7056.trc
SQL> exit
```

性能影响：正常情况，2-3 分钟左右应该能完成一次 Hang Analyze..

需要注意：在 11.2 以上版本，不能使用 -prelim 搜集 HangAnalysis 信息。

**Note:** From 11.2.0.2 onwards, hanganalyze will not produce output under a sqlplus "preliminary connection" since it requires a process state object and a session state object. If a hanganalyze is attempted, the trace will output:

```
HANG ANALYSIS:

ERROR: Can not perform hang analysis dump without a process state object and a session state object.
( process=(nil), sess=(nil) )
```

## 8.10 SystemState 信息搜集

Systemstate 包含数据库系统的状态信息，搜集方法如下：

详情请参考：How to Collect Diagnostics for Database Hanging Issues (Doc ID 452358.1)

```
非 RAC 情况:
sqlplus '/ as sysdba'
oradebug setmypid
oradebug unlimit
oradebug hanganalyze 3
-- Wait one minute before getting the second hanganalyze
oradebug hanganalyze 3
```

```
oradebug tracefile_name  
exit
```

```
sqlplus '/ as sysdba'  
oradebug setmypid  
oradebug unlimit  
oradebug dump systemstate 266  
oradebug dump systemstate 266  
oradebug tracefile_name  
exit
```

(此外 Systemdump 较低的级别为 10, 如果 266/267 做不出来, 可以考虑用 level 10. Level 10 相对执行速度较快.)

```
SQL> conn / as sysdba  
SQL> oradebug setmypid  
SQL> oradebug unlimit  
SQL> oradebug dump systemstate 10;  
等 2 分钟  
SQL> oradebug dump systemstate 10;  
SQL> oradebug tracefile_name  
SQL> exit
```

#### RAC 情况: .

第 1 种: collection commands for Hanganalyze and Systemstate: **RAC with fixes for bug 11800959** ((fixed in 11.2.0.2 DB PSU5, 11.2.0.3 and above))**and Bug 11827088** (Fixed in 11.2.0.2.4 Database Patch Set Update and 11.2.0.3 above)

```
sqlplus '/ as sysdba'  
oradebug setorapname reco  
oradebug unlimit  
oradebug -g all hanganalyze 3  
oradebug -g all hanganalyze 3  
oradebug -g all dump systemstate 266  
oradebug -g all dump systemstate 266  
exit
```

第 2 种: Collection commands for Hanganalyze and Systemstate: **RAC without fixes for Bug 11800959** ((fixed in 11.2.0.2 DB PSU5, 11.2.0.3 and above))**and Bug 11827088** (Fixed in 11.2.0.2.4 Database Patch Set Update and 11.2.0.3 above)

```
sqlplus '/ as sysdba'  
oradebug setorapname reco
```



```
oradebug unlimit
oradebug -g all hanganalyze 3
oradebug -g all hanganalyze 3
oradebug -g all dump systemstate 258
oradebug -g all dump systemstate 258
exit
```

第三种：也可以每个节点分开做(2 个节点分开做)。

```
sqlplus '/ as sysdba'
oradebug setorapname reco
oradebug unlimit
oradebug hanganalyze 3
oradebug hanganalyze 3
oradebug dump systemstate 258
oradebug dump systemstate 258
exit
```

### 性能影响：

由于 System dump 等于系统整体的 dump，耗时相对要大一些，其和系统 CPU/内存/I/O 负载，会话数，SGA 大小等方面都有关系。一般 systemdump 266 /267 采样需要消耗时间参考如下：

某省级移动公司 BOSS 系统大型数据库有 4000-5000 个会话 SGA 64G

节点 1 dump 时间大概 50 分钟左右/次，dump 大小 1.5G/次，2 次大概 3G 左右。

节点 2 dump 时间要少一些，30 分钟左右/次，dump 出来的大小约 1G/次。

但需要说明的：此值仅为参考值，因为客户实际的环境和负载不同，具体时间范围可能有所变化。

此外 Systemdump 较低的级别为 10，如果 266/267 做不出来，可以考虑用 level 10。Level 10 相对执行速度较快。

## 8.11 errorstack 信息收集

针对特定进程的 **errorstack** 信息进行收集（oracle support 指定收集）

```
SQL> oradebug setospid <pid of spinning process>
SQL> oradebug unlimit
SQL> oradebug dump errorstack 3
等待 2-3 分钟左右
SQL> oradebug dump errorstack 3
SQL> oradebug tracefile_name
SQL>exit
```

## 8.12 10046 事件

10046 事件主要跟踪会话执行的 SQL 的情况，跟踪会话中 SQL 的执行计划及等待事件可以通过 TKPROF,TRCA 等工具格式化 TRC 文件

可以在 sqlplus 中收集本 session 执行的 10046 trace

```
alter session set tracefile_identifier='10046';
alter session set timed_statistics = true;
alter session set statistics_level=all;
alter session set max_dump_file_size = unlimited;
alter session set events '10046 trace name context forever,level 12';
-- Execute the queries or operations to be traced here --
--select * from dual;
alter session set events '10046 trace name context off';
```

也可以在 sqlplus 中使用 oradebug，生成其他 session 的 10046 trace

该场景主要是生成并找出会话 (sid,serial#) =' 99,5929' 的10046信息

```
sqlplus / as sysdba
SQL> select b.pid,b.spid from v$session a ,v$process b where a.paddr=b.addr and a.sid=&SID and
a.serial#=&SERIAL;
Enter value for sid: 99
Enter value for serial: 5929
old   1: select b.pid,b.spid from v$session a ,v$process b where a.paddr=b.addr and a.sid=&SID and
a.serial#=&SERIAL
new   1: select b.pid,b.spid from v$session a ,v$process b where a.paddr=b.addr and a.sid=99 and
a.serial#=5929

      PID SPID
-----
51 13238462

SQL> oradebug setorapid 51
Oracle pid: 51, Unix process pid: 13238462, image: oracle@ktttestdb001r (TNS V1-V3)

SQL> oradebug event 10046 trace name context forever,level 12

SQL> oradebug event 10046 trace name context off;
Statement processed.
```

```
SQL> oradebug tracefile_name
/u01/app/oracle/diag/rdbms/abs/abs1/trace/abs1_oracle_13238462.trc

SQL> oradebug close_trace;
Statement processed.

kttestdb001r:/home/oracle$tkprof /u01/app/oracle/diag/rdbms/abs/abs1/trace/abs1_oracle_13238462.trc
/home/oracle/1.txt
TKPROF: Release 11.2.0.4.0 - Development on Wed Jul 6 03:02:30 2016
Copyright (c) 1982, 2011, Oracle and/or its affiliates. All rights reserved.
```

查看/home/oracle/1.txt 里面关于该 SQL 的信息即可

### 8.13 10053 事件

10053 事件主要分析优化器的行为及执行计划产生的依据,便于查找执行计划错误的原因,发现表分析数据中存在的问题

可以在 sqlplus 中收集本 session 执行的 10053 trace

```
SQL> ALTER SESSION SET EVENTS '10053 trace name context forever, level 1';
Session altered.

-- set up binds.

SQL> variable a number
SQL> variable b varchar2(10)

-- assign values to binds

SQL> begin
2 :a:=20;
3 :b:='CLERK';
4 end;
5 /

SQL> select empno, ename, mgr
2 from emp
3 where deptno = :a
4 and job = :b
```

/

```
SQL> ALTER SESSION SET EVENTS '10053 trace name context off';
SQL> SELECT value FROM v$diag_info WHERE name='Default Trace File';
```

也可以在 sqlplus 中使用 oradebug, 生成其他 session 的 10053 trace

该场景主要是生成并找出会话 (sid,serial#) = ' 42,6229' 的 10053 信息

```
sqlplus / as sysdba
```

```
SQL> select sid,serial# from v$session where sid=42;
```

| SID | SERIAL# |
|-----|---------|
| 42  | 6229    |

```
SQL> select b.pid,b.spid from v$session a ,v$process b where a.paddr=b.addr and a.sid=&SID and
a.serial#=&SERIAL;
```

```
Enter value for sid: 42
```

```
Enter value for serial: 6229
```

```
old 1: select b.pid,b.spid from v$session a ,v$process b where a.paddr=b.addr and a.sid=&SID and
a.serial#=&SERIAL
```

```
new 1: select b.pid,b.spid from v$session a ,v$process b where a.paddr=b.addr and a.sid=42 and
a.serial#=6229
```

| PID SPID   |
|------------|
| 49 7012488 |

```
SQL> oradebug setorapid 49
```

```
Oracle pid: 49, Unix process pid: 7012488, image: oracle@ktttestdb001r (TNS V1-V3)
```

```
SQL> oradebug event 10053 trace name context forever,level 1;
Statement processed.
```

```
SQL> oradebug tracefile_name
```

```
/u01/app/oracle/diag/rdbms/abs/abs1/trace/abs1_ora_7012488.trc
```

```
SQL> oradebug event 10053 trace name context off;
Statement processed.
```

```
SQL> oradebug close_trace;
Statement processed.
```

查看相关的 trace 文件即可

## 9 附加内容 – 相关故障解决分析参考

### 9.1 ORA-4031

#### 【故障现象】

oracle 进程认为内存无法分配，严重时可能导致整个实例 hang。

#### 【可能故障原因】

- 物理内存耗尽
- Shared pool 大小不合理
- Shared pool 子池个数不合理
- 内存碎片
- 过高的硬解析/没有绑定变量

#### 【应急措施】

```
Sun Jan 22 16:26:11 EAT 2019
Errors in file /oracle/app/oracle/admin/xxxxx2/bdump/xxxxx22_j006_8166.trc:
ORA-12012: 自动执行作业 2776 出错
ORA-04031: 无法分配 4120 字节的共享内存 ("shared pool","select
o.owner#,o.name,o.nam...", "Typecheck", "kgghteInit")
ORA-06512: 在 "XXXXXX.XX_TS_DIS_ID_EVENT", line 47
ORA-06512: 在 line 1
```

一般应急状况下，处理的第一时间不用去检查共享池碎片，意义不大。而且 4031 错误同样有可能导致整个数据库实例 hang，所以应该去检查什么组件大小比较异常（有条件的话同类似的其他库环境去对比）

```
Select t.*
  From (Select Name,
              bytes / (1024 * 1024) "MB",
              round(bytes / (Select Value From v$parameter t
                              Where t.name = 'shared_pool_size') * 100,
                    2) || '%' "USED%"
        From v$sgastat
        Where pool = 'shared pool'
        Order By 2 Desc) t
 Where rownum < 20;
```

根据当前组件大小状态，判断是哪部分出了问题，如果是可以回收空间的组件，条件允许

的话通过 flush shared\_pool 来处理。比如 sql area、Ccursor 等占据主要大小，可能与硬解析有关，可以通过下面的查询确认：

```
SET pages 10000
SET linesize 200
column FORCE_MATCHING_SIGNATURE format 99999999999999999999
with c as
  (select force_matching_signature, count(*) cnt
   from v$sqlarea
   where force_matching_signature != 0
   group by force_matching_signature
   having count(*) > 20),
sq as
  (select sql_text,
         force_matching_signature,
         row_number() over(partition by force_matching_signature order by sql_id desc) p
   from v$sqlarea s
   where force_matching_signature in
        (select force_matching_signature from c))
select sq.sql_text, sq.force_matching_signature, c.cnt "unshared count"
  from c, sq
 where sq.force_matching_signature = c.force_matching_signature
    and sq.p = 1
 order by c.cnt desc;
```

如果是无法回收的 perm 组件，则必须重启实例。临时的一个解决方法是，内存空间足够的情况下，内存中调大 shared pool，等到临时窗口后再重启实例恢复。

```
Select pool, Sum(bytes) / (1024 * 1024) "MB"
  From v$sghostat
 Where pool = 'shared pool'
 Group By pool;

alter system set shared_pool_size = XXXXM sid='xxxx' scope=memory;
```

在重启或者刷新 shared\_pool 前，收集相关信息留做进一步的根因诊断。

- 建议从 MOS 430473.1 下载脚本，收集发生问题时候的信息
- 尝试收集 AWR，如果夯住则中断掉

```
SQL> @$ORACLE_HOME/rdbms/admin/awrrpt.sql
可以通过下面方法在另一个好的实例收集AWR
SELECT output FROM
TABLE(DBMS_WORKLOAD_REPOSITORY.AWR_REPORT_HTML(      3616491197, 1, 61143,
61144) );
```

- 对于 4031 的深入以及代码层面分析，Oracle 后台研发一般需要 heapdump 或者 library cache dump 进行分析。

1) Heapdump 的信息：命令如下：

```
sqlplus /nolog
connect / as sysdba
oradebug setmypid
oradebug unlimit
oradebug dump heapdump 536871426
oradebug tracefile_name
oradebug close_trace
```

影响分析：从经验来看，一般 Heapdump 执行不会对系统造成太多的影响，影响基本可以忽略。如果产生影响，也是系统产生 4031 后影响结果，非本身该操作影响。

2) 根据需要可以做 library cache dump. 命令 如下：

```
ALTER SYSTEM SET EVENTS 'IMMEDIATE TRACE NAME LIBRARY_CACHE LEVEL 16';
```

影响分析：从经验来看，library cache dump 可能会对系统有些少许的影响(依据系统负载不同而影响不同)，如果故障期间在非业务高峰期，建议可以使用。

备注：执行 Heapdump 以及 library cache dump 有必要可以联系 oracle 支持进行指导。

### 【后续分析】

补充收集 Alert file/trace file、AWR/ASH、OSW。进一步分析；必要时需要寻求 oracle support 的支持。

## 9.2 SQL 锁问题处理

### 9.2.1 查看被锁对象信息

```
set linesize 180
col username for a15
col lock_level for a15
col owner for a10
col object_name for a15
col object_type for a15
col machine for a15
col program for a20
col osuser for a15
col terminal for a10
SELECT s.username,
decode(l.type, 'TM', 'TABLE LOCK',
'TX', 'ROW LOCK',
NULL) LOCK_LEVEL,
```

```
o.owner, o.object_name, o.object_type,  
s.sid, s.serial#, s.terminal, s.machine, s.program, s.osuser  
FROM v$session s, v$lock l, dba_objects o  
WHERE l.sid = s.sid  
AND l.id1 = o.object_id(+)  
AND s.username is NOT NULL  
and owner=' &OWNER' ;
```

### 9.2.2 查看会话阻塞信息

```
set linesize 180  
col username for a15  
col event for a35  
select  
a.sid, a.serial#, a.USERNAME, a.status, a.BLOCKING_SESSION_STATUS, a.BLOCKING_INSTANCE, a.BLOCKING_SESSION, a.event, a.SECONDS_IN_WAIT      from      v$session      a      where  
a.BLOCKING_SESSION_STATUS = 'VALID' ;
```

### 9.2.3 解锁

```
通过与业务人员沟通确定可以 kill 阻塞会话后，再 kill 该会话（即 3.2.2 的  
blocking_session）  
alter system kill session 'SID,SERIAL#';
```